

Manual de Login y Autenticación — Laravel + JWT + MySQL

PHP / Laravel 11 · MySQL 8 · JWT · .env

Guía Completa de Implementación

Agosto 2026

Parte 1 — Fundamentos del Proyecto, Setup y Arquitectura

1. Introducción al Manual

Propósito

Stack Tecnológico

¿Qué Aprenderá el Lector?

Prerrequisitos

2. ¿Por Qué Laravel? (Decisión de Arquitectura)

Comparativa: Laravel vs Symfony vs Slim vs PHP Puro

Ventajas de Laravel para Autenticación

El Ecosistema de Autenticación en Laravel

Decisión para Este Manual: JWT Puro con `tymon/jwt-auth`

3. Arquitectura de Autenticación (Conceptual)

Diagrama Conceptual del Sistema

Flujo de Autenticación Completo

Modelo de Capas

Dónde Vive la Lógica de Negocio

Stateless JWT: Qué Significa

Access Token vs Refresh Token

4. Decisiones de Seguridad Fundamentales

Password Hashing: bcrypt

JWT Signing Algorithm: HS256

Claims del JWT

Token Storage en el Frontend

HTTPS como Requisito No Negociable

CORS Configuration

5. Estructura del Proyecto

Creación del Proyecto

Estructura de Directorios Relevante para Auth

Responsabilidad de Cada Componente

6. Setup Inicial Paso a Paso

Paso 1: Crear el Proyecto

Paso 2: Verificar .env y Clave de Aplicación

Paso 3: Verificar que el Proyecto Corre

Paso 4: Limpiar la Estructura por Defecto

Paso 5: Crear los Directorios de la Estructura

7. Convenciones y Buenas Prácticas

Naming Conventions

Principios SOLID Aplicados a la Autenticación

DRY en Lógica de Auth

Tipado Estricto

Uso de Enums

Manejo de Excepciones

Response Standardization

8. Resumen y Puente a la Parte 2

Lo Construido en Esta Parte

Lo Que Viene en la Parte 2

Referencias para Partes Posteriores

Parte 2: Variables de Entorno y Base de Datos

1. Introducción

2. El archivo `.env` — Teoría y Práctica

2.1 ¿Qué es `.env` ?

2.2 `.env.example` vs `.env`

2.3 Variables de entorno vs archivos de configuración de Laravel

2.4 `php artisan config:cache` — cuándo y por qué

3. Variables de Entorno para Autenticación

3.1 Bloque Database

3.2 Bloque JWT

3.3 Bloque App

3.4 Variables adicionales de seguridad

4. Configuración de MySQL

4.1 Crear la base de datos

4.2 Crear un usuario dedicado (buena práctica)

4.3 Probar la conexión

5. La Migración de Users — Diseño de la Tabla

5.1 Crear la migración

5.2 Análisis columna por columna

6. El Modelo User

6.1 `extends Authenticatable`

6.2 `implements JWTSubject`

6.3 SoftDeletes

6.4 \$fillable vs \$guarded

6.5 \$hidden

6.6 casts() — la magia de Laravel 11

7. Los Enums de Usuario

7.1 UserRole

7.2 UserStatus

7.3 ¿Por qué Backed Enums (: string)?

7.4 Ventajas sobre constantes de clase

8. Ejecución y Verificación

8.1 Ejecutar migraciones

8.2 Verificar con migrate:status

8.3 Probar con Tinker

8.4 (Opcional) Crear UserFactory

9. Seguridad en la Capa de Datos

9.1 SQL Injection: protección automática de Eloquent

9.2 Mass Assignment: \$fillable como protección

9.3 Timing Attacks en login

9.4 Hasheo automático con cast 'hashed'

10. Resumen y Puente a la Parte 3

Lo construido en esta parte

Decisiones de diseño que las Partes 3-6 deben conocer

Lo que viene en la Parte 3

Parte 3: Configuración de JWT y Modelo de Tokens

1. Introducción a JWT

1.1 ¿Qué es JSON Web Token?

1.2 ¿Por qué JWT para APIs?

1.3 JWT vs OAuth2 vs SAML — ¿Cuándo usar cada uno?

1.4 JWT NO es un mecanismo de sesiones

2. Instalación del Paquete JWT-Auth

2.1 Composer require

2.2 Service Providers en Laravel 11

2.3 Publicar configuración

3. Configuración de config/jwt.php — Explicación Exhaustiva

3.1 secret

3.2 keys (solo para algoritmos asimétricos)

3.3 ttl — Time To Live del Access Token

3.4 refresh_ttl — Ventana de Refresh

3.5 algo — Algoritmo de Firma

- 3.6 `required_claims` — Claims Obligatorios
- 3.7 `persistent_claims`
- 3.8 `lock_subject`
- 3.9 `leeway`
- 3.10 `blacklist_enabled`
- 3.11 `blacklist_grace_period`
- 3.12 `decrypt_cookies`
- 3.13 `providers`
- 4. Generar `JWT_SECRET`
 - 4.1 El comando `jwt:secret`
 - 4.2 Qué hace exactamente
 - 4.3 Verificación
 - 4.4 ADVERTENCIA DE PRODUCCIÓN
- 5. Implementar `JWTSubject` en el Modelo `User`
 - 5.1 El código
 - 5.2 `getJWTIdentifier()` — El claim `sub`
 - 5.3 `getJWTCustomClaims()` — Claims personalizados
 - 5.4 ADVERTENCIA DE SEGURIDAD SOBRE CLAIMS PERSONALIZADOS
- 6. Configurar el Guard en `config/auth.php`
 - 6.1 El archivo `config/auth.php` modificado
 - 6.2 Explicación por sección
- 7. Anatomía de un Token JWT en esta Aplicación
 - 7.1 Token completo (ejemplo)
 - 7.2 Header decodificado
 - 7.3 Payload decodificado
 - 7.4 Explicación de cada claim en este payload
 - 7.5 Relación con `required_claims`
- 8. Estrategia de Tokens: Access + Refresh
 - 8.1 Access Token
 - 8.2 Refresh Token
 - 8.3 Rotación de Refresh Tokens (Refresh Token Rotation)
 - 8.4 Flujo Completo de Refresh
- 9. Almacenamiento de Refresh Tokens — Diseño de la Tabla `refresh_tokens`
 - 9.1 Esquema SQL
 - 9.2 Explicación columna por columna
 - 9.3 Índices
 - 9.4 FK con `ON DELETE CASCADE`
 - 9.5 Alternativa evaluada: Redis vs Base de Datos
 - 9.6 Estrategia de limpieza

10. Verificación de la Configuración

- 10.1 Limpiar caché de configuración
- 10.2 Probar generación de token con Tinker
- 10.3 Decodificar el token en jwt.io

11. Resumen y Puente a Parte 4

- 11.1 Lo instalado y configurado en esta parte
- 11.2 Estado actual del sistema
- 11.3 Lo que viene en la Parte 4

Decisiones Vinculantes para Partes 4-6

Parte 4: Registro de Usuarios

1. Introducción

- 1.1 Dónde estamos
- 1.2 Objetivo de esta parte
- 1.3 Qué construiremos

2. Migración de `refresh_tokens`

- 2.1 Crear la migración
- 2.2 Análisis columna por columna
- 2.3 Índices — la clave del rendimiento
- 2.4 Ejecutar la migración

3. Modelo `RefreshToken`

- 3.1 Explicación del modelo

4. Actualización del Modelo `User`

5. `RegisterRequest` — Validación de Entrada

- 5.1 Crear el `FormRequest`
- 5.2 Explicación de cada regla
- 5.3 `messages()` — Personalización de errores
- 5.4 ¿Qué pasa cuando la validación falla?

6. `AuthService::register()` — Lógica de Negocio

- 6.1 Explicación detallada

7. `RegisterController` — Punto de Entrada HTTP

- 7.1 Explicación

8. Ruta de Registro

- 8.1 Explicación

9. Testing del Registro

- 9.1 Prueba manual con curl
- 9.2 Prueba con Postman / Insomnia
- 9.3 Pruebas con PHPUnit
- 9.4 Casos de prueba negativos

9.5 Ejecutar los tests

10. Seguridad Específica del Registro

10.1 Rate Limiting

10.2 Password Strength

10.3 Email Uniqueness y Timing Attacks

10.4 SQL Injection

10.5 XSS en el nombre

10.6 Mass Assignment — `$fillable` y `role`

11. Resumen y Puente a la Parte 5

11.1 Lo construido en esta parte

11.2 Flujo completo de registro

11.3 Lo que viene en la Parte 5

Decisiones Vinculantes para Partes 5-6

Parte 5: Login, Autenticación y Middleware JWT

1. Introducción

1.1 Dónde estamos

1.2 Objetivo de esta parte

1.3 Qué construiremos

2. `LoginRequest` — Validación de Credenciales

2.1 Crear el `FormRequest`

2.2 Explicación de cada regla y las reglas AUSENTES

2.3 `messages()` — Personalización de errores

3. `AuthService::login()` — Lógica de Negocio

3.1 El método completo

3.2 Explicación detallada, paso a paso

4. Excepciones de Dominio

4.1 `AuthenticationException`

4.2 `AccountInactiveException`

4.3 Explicación del diseño

5. `LoginController` — Punto de Entrada HTTP

5.1 Explicación

6. Ruta de Login y Grupo de Rutas Públicas

6.1 Agrupación de rutas públicas

6.2 Explicación

7. Middleware `JwtAuthenticate` — Protección de Rutas

7.1 Crear el middleware

7.2 Explicación detallada del flujo

7.3 Registrar el middleware en el Kernel

7.4 ¿Por qué NO usar el middleware incluido en `tymon/jwt-auth` ?

8. Rutas Protegidas

8.1 Agrupación bajo `jwt.auth`

8.2 Explicación

9. `AuthService::refreshTokens()` — Rotación de Tokens con Detección de Reuse Attack

9.1 El método completo

9.2 Explicación detallada de cada paso

10. `AuthService::logout()` — Cierre de Sesión

10.1 El método completo

10.2 Explicación

11. `RefreshTokenRequest` — Validación del Refresh Token

11.1 Explicación

12. `RefreshTokenController` y `LogoutController`

12.1 `RefreshTokenController`

12.2 `LogoutController`

12.3 Explicación

13. Endpoint `/me` — Usuario Autenticado

13.1 `UserController`

13.2 Explicación

14. Testing del Login, Middleware, Refresh y Logout

14.1 Archivo de tests

14.2 Explicación de los tests

14.3 Ejecutar los tests

15. Resumen y Puente a Parte 6

15.1 Lo construido en esta parte

15.2 El `AuthService` completo

15.3 Flujo completo de autenticación

15.4 Lo que viene en la Parte 6

Decisiones Vinculantes para Parte 6

Parte 6: Seguridad Avanzada, CORS, Rate Limiting, Limpieza Programada y Testing Integral

1. Introducción

1.1 Dónde estamos

1.2 Objetivo de esta parte

1.3 Qué NO construiremos en esta parte

2. Configuración de CORS (Cross-Origin Resource Sharing)

2.1 Qué es CORS y por qué importa

2.2 Flujo de una petición CORS

2.3 Configuración en Laravel 11

- 2.4 Explicación de cada clave
- 2.5 Verificación de CORS
- 3. Rate Limiting Avanzado
 - 3.1 ¿Qué es rate limiting y por qué es crítico?
 - 3.2 Rate Limiting por IP vs por Usuario
 - 3.3 Rate limiters personalizados
 - 3.4 Aplicar los rate limiters personalizados en las rutas
 - 3.5 Strategy de rate limiting en producción
- 4. HTTP Security Headers
 - 4.1 Crear el middleware de security headers
 - 4.2 Registrar el middleware globalmente
 - 4.3 Explicación de cada header
- 5. Manejo de Errores Global
 - 5.1 Configuración en `bootstrap/app.php`
 - 5.2 Explicación de cada sección
- 6. Logging de Eventos de Seguridad
 - 6.1 Eventos a registrar
 - 6.2 Niveles de log (PSR-3)
 - 6.3 Estructura de los logs
 - 6.4 Estrategia de monitoreo recomendada
- 7. Comando de Limpieza de Refresh Tokens (Scheduled Task)
 - 7.1 El problema
 - 7.2 Crear el comando
 - 7.3 Explicación del comando
 - 7.4 Programar el comando
 - 7.5 Configurar el cron en el servidor
- 8. Factories para Testing
 - 8.1 UserFactory completa
 - 8.2 Explicación de cada estado
- 9. Testing Integral de Autenticación
 - 9.1 Archivos de test
 - 9.2 LoginTest (expandido desde Parte 5)
 - 9.3 RegisterTest (expandido desde Parte 4)
 - 9.4 SecurityTest (NUEVO)
 - 9.5 Ejecutar todos los tests
- 10. Checklist de Seguridad para Producción
 - ☒ Ambiente y Configuración
 - ☒ Optimización de Laravel
 - ☒ CORS

- ☑ Rate Limiting
- ☑ Security Headers
- ☑ Base de Datos
- ☑ Contraseñas y Autenticación
- ☑ JWT Específico
- ☑ Monitoreo y Logs
- ☑ Tests

11. Preparación para Producción — Resumen de Comandos

12. Conclusión del Manual Completo

12.1 Lo Construido en las 6 Partes

12.2 Stack Final

12.3 El lector ahora tiene

12.4 Próximos pasos recomendados

12.5 Referencias y lecturas adicionales

Decisiones Vinculantes para Futuras Extensiones

Parte 1 — Fundamentos del Proyecto, Setup y Arquitectura

1. Introducción al Manual

Propósito

Este manual es una guía completa para implementar un sistema de autenticación y autorización robusto, seguro y mantenible sobre PHP/Laravel. Está diseñado para desarrolladores con experiencia básica-intermedia en PHP que necesitan llevar su sistema de login al siguiente nivel — desde cero hasta un sistema listo para producción con seguridad avanzada y testing automatizado.

No es un tutorial de “copiar y pegar”. Cada decisión de arquitectura, cada patrón y cada línea de código están justificadas con razonamiento técnico. Al terminar, el lector no solo tendrá un sistema funcional, sino que entenderá el POR QUÉ de cada pieza.

Stack Tecnológico

Componente	Tecnología	Versión Mínima
Lenguaje	PHP	8.2+
Framework	Laravel	11.x
Base de Datos	MySQL	8.0+
Autenticación	JWT (JSON Web Tokens)	tymon/jwt-auth
Configuración	<code>.env</code>	—
Dependencias	Composer	2.x
Assets Frontend	Node.js / npm	20.x LTS

¿Qué Aprenderá el Lector?

Al completar las partes de este manual, el lector será capaz de:

- Configurar un proyecto Laravel desde cero con autenticación JWT stateless.
- Diseñar una arquitectura en capas que separe responsabilidades: Controllers, FormRequests, Services, Repositories y Models.
- Implementar registro, login, logout, refresh de tokens y verificación de email.
- Proteger rutas con middleware JWT personalizado y políticas de autorización.
- Aplicar seguridad avanzada: rate limiting, CORS, headers de seguridad, protección contra ataques comunes (CSRF, XSS, SQL Injection, timing attacks).
- Escribir tests unitarios y de feature que validen cada flujo de autenticación.

- Integrar el sistema con un frontend SPA (React/Vue) o aplicación mobile.

Prerrequisitos

Antes de comenzar, asegúrese de tener instalado:

- **PHP 8.2+** con las extensiones: `bcmath`, `ctype`, `fileinfo`, `json`, `mbstring`, `openssl`, `pdo`, `pdo_mysql`, `tokenizer`, `xml`.
- **Composer 2.x** para gestión de dependencias PHP.
- **MySQL 8.0+** como motor de base de datos.
- **Node.js 20 LTS** y **npm** para compilación de assets básicos (solo si se usa el frontend incluido de Laravel).

Conocimientos esperados: sintaxis PHP moderna, conceptos básicos de Laravel (rutas, controladores, migraciones), HTTP y REST, fundamentos de bases de datos relacionales.

2. ¿Por Qué Laravel? (Decisión de Arquitectura)

Comparativa: Laravel vs Symfony vs Slim vs PHP Puro

Framework	Curva de Aprendizaje	Autenticación	ORM/DB	Middleware	Ecosistema
Laravel	Media	Nativa + paquetes	Eloquent (activo)	Pipeline integrado	Extenso
Symfony	Alta	Security component	Doctrine (data mapper)	Event dispatcher	Muy extenso
Slim	Baja	Manual	Manual	Middleware stack	Mínimo
PHP Puro	—	Manual	Manual	Manual	Inexistente

Laravel ofrece el mejor balance entre potencia y productividad. Symfony es más explícito y modular — ideal para proyectos enterprise con requisitos muy específicos. Slim y PHP puro requieren implementar todo desde cero, lo cual es inviable para un sistema de auth que debe ser seguro y mantenible.

Ventajas de Laravel para Autenticación

- **Hashing integrado:** `Hash::make()` usa bcrypt por defecto con factor de trabajo configurable. No hay que tocar `password_hash()` manualmente ni preocuparse por salts.
- **Middleware Pipeline:** Cada request atraviesa una cadena de middlewares. La autenticación JWT se inserta como un middleware más, sin contaminar los controladores.

```
Request → CORS → Auth (JWT) → Rate Limiter → Controller → Response
```

- **Eloquent ORM:** El modelo `User` se integra directamente con JWT para resolver la identidad del usuario autenticado en cada request.
- **Validación declarativa:** Los FormRequests permiten validar datos de entrada antes de que lleguen a la lógica de negocio. Las reglas son legibles, reutilizables y testables.
- **Rate Limiting nativo:** `RateLimiter` de Laravel permite proteger endpoints de login y registro contra fuerza bruta

sin dependencias externas.

El Ecosistema de Autenticación en Laravel

Laravel ofrece tres caminos principales para autenticación. Elegir el correcto ES una decisión de arquitectura que condiciona todo lo demás:

Solución	Tipo de Auth	Caso de Uso Ideal	Estado
Sanctum	Token/Sesión	SPA (React, Vue) mismo dominio; Mobile con tokens simples	Stateful o stateless
Passport	OAuth2	APIs que necesitan OAuth2 completo (third-party, scopes complejos)	OAuth2 server
JWT puro	JWT stateless	APIs REST stateless para SPA + mobile; microservicios	Stateless

Sanctum es ideal cuando el frontend vive en el mismo dominio (cookies de sesión) o cuando los tokens pueden almacenarse en base de datos (no es JWT real, es un token opaco).

Passport implementa un servidor OAuth2 completo. Es overkill si no se necesita el protocolo OAuth2 — su complejidad es considerable y requiere entender grants, scopes y flujos OAuth.

Decisión para Este Manual: JWT Puro con `tymon/jwt-auth`

Se eligió **JWT puro** con el paquete `tymon/jwt-auth` por las siguientes razones técnicas:

- Stateless:** El servidor no almacena sesiones. Cada token contiene toda la información necesaria para identificar al usuario. Esto elimina consultas a base de datos para verificar sesiones en cada request.
- Mobile-friendly:** Los tokens JWT viajan en el header `Authorization: Bearer <token>`, lo cual funciona nativamente en cualquier cliente HTTP (Axios, Fetch, Retrofit, Alamofire).
- SPA-ready:** El token puede almacenarse en httpOnly cookies o en memoria, y enviarse en cada request sin configuración adicional.
- Escalabilidad horizontal:** Al no depender de sesiones en servidor, múltiples instancias de la API pueden validar tokens sin compartir estado.

El paquete `tymon/jwt-auth` es el estándar de facto para JWT en Laravel, con más de una década de madurez y mantenimiento activo.

3. Arquitectura de Autenticación (Conceptual)

Diagrama Conceptual del Sistema

```

sequenceDiagram
    participant C as Cliente (SPA/Mobile)
    participant R as Router (api.php)
    participant M as JWT Middleware
    participant FR as FormRequest
    participant S as AuthService
    participant DB as MySQL

    Note over C,DB: === FLUJO DE REGISTRO ===
    C->>R: POST /api/auth/register
    R->>FR: Validar datos (RegisterRequest)
    FR->>R: Datos válidos
    R->>S: register(data)
    S->>DB: INSERT user
    DB-->>S: User creado
    S-->>R: User + Tokens
    R-->>C: 201 {user, access_token, refresh_token}

    Note over C,DB: === FLUJO DE LOGIN ===
    C->>R: POST /api/auth/login
    R->>FR: Validar credenciales (LoginRequest)
    FR->>R: Datos válidos
    R->>S: login(email, password)
    S->>DB: SELECT user WHERE email = ?
    DB-->>S: User
    S->>S: Hash::check(password)
    S-->>R: User + Tokens
    R-->>C: 200 {user, access_token, refresh_token}

    Note over C,DB: === PETICIÓN AUTENTICADA ===
    C->>R: GET /api/user (Authorization: Bearer <access_token>)
    R->>M: Verificar JWT
    M->>M: Decodificar + validar firma + exp
    M-->>R: User autenticado (auth()->user())
    R-->>C: 200 {user data}

    Note over C,DB: === REFRESH DE TOKEN ===
    C->>R: POST /api/auth/refresh (Bearer <refresh_token>)
    R->>M: Verificar refresh token
    M->>R: User autenticado
    R->>S: refresh()
    S->>R: Nuevos tokens
    R-->>C: 200 {access_token, refresh_token}

    Note over C,DB: === LOGOUT ===
    C->>R: POST /api/auth/logout (Bearer <access_token>)
    R->>M: Verificar JWT
    M->>R: User autenticado
    R->>S: logout()
    S->>S: Invalidar token (blacklist)
    S-->>R: OK
    R-->>C: 200 {message: "Sesión cerrada"}

```

Flujo de Autenticación Completo

El sistema implementa cinco flujos principales:

1. **Registro:** El cliente envía `name`, `email`, `password`, `password_confirmation`. El sistema valida, crea el usuario, y devuelve tokens JWT.
2. **Login:** El cliente envía `email` y `password`. El sistema verifica credenciales y devuelve tokens JWT.
3. **Peticiones autenticadas:** Cada request incluye el access token en el header `Authorization: Bearer <token>`. El middleware JWT lo decodifica, valida firma, expiración, y resuelve el usuario.
4. **Refresh de token:** Cuando el access token expira, el cliente envía el refresh token para obtener un nuevo par de tokens sin re-autenticarse.
5. **Logout:** El access token se invalida (se agrega a una blacklist). El refresh token también se invalida.

Modelo de Capas

La arquitectura sigue el principio de separación de responsabilidades en capas bien definidas:

HTTP Layer (Controllers, Middleware)	← Entrada/Salida HTTP
Validation Layer (FormRequests)	← Validación de datos
Service Layer (AuthService, UserService)	← Lógica de negocio
Data Layer (Models, Repositories)	← Persistencia
Infrastructure (MySQL, Redis, Mail)	← Servicios externos

Cada capa tiene una responsabilidad exclusiva:

- **Controllers:** Recibir la request HTTP, delegar al Service Layer, devolver respuesta HTTP. Nunca contienen lógica de negocio.
- **FormRequests:** Validar datos de entrada (authorize + rules). Si falla la validación, Laravel retorna 422 automáticamente.
- **Service Layer:** Toda la lógica de negocio vive aquí. El `AuthService` orquesta registro, login, refresh y logout. Los controllers solo llaman métodos del service.
- **Models/Repositories:** Acceso a datos. Eloquent Models para operaciones CRUD básicas. Si se necesita desacoplar más, se agrega un Repository Pattern.

Dónde Vive la Lógica de Negocio

Regla de oro: La lógica de negocio NUNCA vive en controllers. Los controllers son adaptadores HTTP — traducen requests a llamadas de servicio y respuestas de servicio a responses HTTP.

Un controller correcto se ve así:

```
final class LoginController extends Controller
{
    public function __construct(
        private readonly AuthService $authService
    ) {}

    public function __invoke(LoginRequest $request): JsonResponse
    {
        $result = $this->authService->login(
            email: $request->validated('email'),
            password: $request->validated('password')
        );

        return response()->json($result, 200);
    }
}
```

No hay `if`, no hay lógica de negocio, no hay acceso a base de datos. Solo delegación.

Stateless JWT: Qué Significa

Stateless significa que el servidor no mantiene ningún estado de sesión entre requests. Toda la información necesaria para autenticar al usuario está dentro del token JWT.

Implicaciones:

- El servidor no consulta la base de datos para verificar si la sesión es válida (solo verifica la firma y expiración del

token).

- Cualquier instancia del servidor puede validar el token sin compartir estado.
- La invalidación inmediata de tokens (logout) requiere una blacklist, lo cual reintroduce algo de estado.

Ventajas:

- Escalabilidad horizontal sin sticky sessions ni almacenamiento compartido de sesiones.
- Menor latencia: no hay consulta a base de datos por request autenticado.

Desventajas:

- El token no puede invalidarse remotamente sin blacklist.
- Si un token es robado, es válido hasta que expire.
- El payload del token viaja en cada request (overhead de ~1-2 KB).

Access Token vs Refresh Token

Característica	Access Token	Refresh Token
Propósito	Autenticar requests	Obtener nuevos access tokens
Lifetime típico	15-60 minutos	7-30 días
Contiene	Claims del usuario	Claims mínimos (solo ID)
Se envía en	Cada request autenticado	Solo en endpoint de refresh
Almacenamiento FE	Memoria (recomendado)	httpOnly cookie
Blacklist	Sí (en logout)	Sí (en logout)

Estrategia de rotación: Cuando se usa el refresh token para obtener un nuevo access token, el refresh token también se rota — se emiten ambos tokens nuevos y el refresh token anterior se invalida. Esto limita la ventana de ataque si un refresh token es comprometido.

4. Decisiones de Seguridad Fundamentales

Password Hashing: bcrypt

Laravel usa **bcrypt** como algoritmo de hashing por defecto. Para este manual se mantiene bcrypt por:

- **Madurez:** bcrypt existe desde 1999. Ha resistido dos décadas de escrutinio criptográfico.
- **Factor de trabajo configurable:** Laravel establece el cost factor en 10 por defecto (configurable en `config/hashing.php`). Cada incremento en 1 duplica el tiempo de cómputo.
- **Resistencia a GPU:** bcrypt fue diseñado específicamente para ser ineficiente en hardware paralelo (GPUs, FPGAs).
- **Compatibilidad universal:** Soportado en todas las versiones de PHP sin extensiones adicionales.

Argon2 vs bcrypt: Argon2 (ganador del Password Hashing Competition 2015) es teóricamente superior — resistencia a GPU y a ataques side-channel. Sin embargo, bcrypt sigue siendo la opción más compatible y madura. Si el proyecto requiere Argon2id, Laravel lo soporta en `config/hashing.php` con un cambio de una línea.

```
// Laravel abstrae el algoritmo – cambiar de bcrypt a argon2 es una línea de config
'default' => env('HASH_DRIVER', 'bcrypt'),
```

JWT Signing Algorithm: HS256

Algoritmo	Tipo	Clave	Caso de Uso
HS256	Simétrico	Secreto compartido	Servicio monolítico, single-server
RS256	Asimétrico	Par público/privado	Microservicios, múltiples consumidores

Para este manual se usa **HS256** porque:

- Es el más simple: una sola clave secreta (`JWT_SECRET` en `.env`).
- El mismo servidor que emite los tokens los valida (single-server).
- Menor overhead que RS256 (clave más corta, firma más rápida).

Se usará RS256 si en el futuro se requiere que múltiples servicios independientes validen tokens sin compartir el secreto.

Claims del JWT

Un JWT está compuesto por claims (afirmaciones) en su payload. Los claims estándar y custom para este sistema:

```
{
  "sub": "42", // Subject: ID del usuario
  "iat": 1690000000, // Issued At: timestamp de emisión
  "exp": 1690003600, // Expiration: timestamp de expiración
  "jti": "unique-token-id", // JWT ID: identificador único del token
  "token_type": "access", // Custom: tipo de token (access | refresh)
  "role": "user" // Custom: rol del usuario
}
```

Regla de mínimo privilegio: El access token incluye `role` para decisiones rápidas de autorización sin consultar la base de datos. El refresh token incluye SOLO `sub`, `iat`, `exp`, y `token_type: "refresh"` — nunca incluye `role` ni otros datos sensibles.

Token Storage en el Frontend

Método	XSS-safe	JS-accesible	Auto-envío	Vulnerable a CSRF
httpOnly cookie	✓	✗	✓	✓ (mitigable)
localStorage	✗	✓	✗	✗
Memory (variable)	✓	✓	✗	✗
sessionStorage	✗	✓	✗	✗

Recomendación para este manual:

- **Access Token:** Almacenar en memoria (variable JavaScript). Se pierde al cerrar la pestaña, lo cual es deseable para tokens de corta duración. Se envía manualmente en el header `Authorization: Bearer <token>`.
- **Refresh Token:** Almacenar en **httpOnly cookie** con flags `Secure`, `SameSite=Strict`, y `Path=/api/auth/refresh`. Esto lo protege de XSS (no es accesible desde JavaScript) y limita su exposición.

HTTPS como Requisito No Negociable

En producción, **HTTPS es obligatorio**. Sin HTTPS:

- Los tokens JWT viajan en texto plano por la red.
- Las credenciales de login son visibles para cualquier intermediario.
- Los httpOnly cookies con flag `Secure` no funcionan sin HTTPS.

El entorno de desarrollo local (`localhost`) es la única excepción aceptable.

CORS Configuration

Cross-Origin Resource Sharing (CORS) es el mecanismo que permite o deniega requests desde orígenes diferentes al de la API. Es crítico en APIs porque:

- La API (`api.tudominio.com`) y el frontend SPA (`app.tudominio.com`) son orígenes distintos.
- Sin CORS, el navegador bloquea todas las requests cross-origin por defecto (same-origin policy).

Configuración recomendada:

- `Access-Control-Allow-Origin` : Solo los orígenes explícitamente autorizados, NUNCA `*`.
- `Access-Control-Allow-Methods` : Solo los métodos HTTP usados por la API (`GET, POST, PUT, DELETE`).
- `Access-Control-Allow-Headers` : `Content-Type, Authorization, X-Requested-With`.
- `Access-Control-Allow-Credentials` : `true` (necesario para cookies httpOnly).
- `Access-Control-Max-Age` : Cachear preflight por 1 hora (3600s) para reducir requests.

5. Estructura del Proyecto

Creación del Proyecto

```
composer create-project laravel/laravel paradise-api
```

Estructura de Directorios Relevante para Auth

```

paradise-api/
├── app/
│   ├── Enums/
│   │   ├── UserRole.php           ← Roles de usuario
│   │   └── UserStatus.php        ← Estados de usuario (active, banned, etc.)
│   ├── Exceptions/
│   │   └── Auth/
│   │       ├── InvalidCredentialsException.php
│   │       └── TokenExpiredException.php
│   ├── Http/
│   │   ├── Controllers/
│   │   │   └── Auth/
│   │   │       ├── LoginController.php
│   │   │       ├── RegisterController.php
│   │   │       ├── LogoutController.php
│   │   │       ├── RefreshTokenController.php
│   │   │       └── MeController.php
│   │   ├── Middleware/
│   │   │   └── JwtAuthenticate.php ← Middleware JWT personalizado
│   │   ├── Requests/
│   │   │   └── Auth/
│   │   │       ├── LoginRequest.php
│   │   │       ├── RegisterRequest.php
│   │   │       └── RefreshTokenRequest.php
│   ├── Models/
│   │   └── User.php               ← Modelo de usuario
│   ├── Services/
│   │   ├── AuthService.php        ← Lógica de autenticación
│   │   └── Contracts/
│   │       └── AuthServiceInterface.php
│   └── config/
│       ├── auth.php               ← Guards, providers, defaults
│       └── jwt.php                ← Configuración de JWT (Parte 3)
├── database/
│   └── migrations/
│       └── xxxx_create_users_table.php
├── routes/
│   └── api.php                    ← Rutas públicas y protegidas
├── .env                           ← Variables de entorno
├── .env.example                   ← Template de variables
└── composer.json

```

Responsabilidad de Cada Componente

Directorio/Archivo	Responsabilidad
app/Enums/	Enumeraciones tipadas (PHP 8.1+) para roles, estados y constantes de dominio
app/Exceptions/Auth/	Excepciones específicas del dominio de autenticación
app/Http/Controllers/Auth/	Controladores de entrada HTTP para cada endpoint de auth
app/Http/Middleware/JwtAuthenticate	Intercepta requests, valida el JWT, inyecta el usuario autenticado en la request
app/Http/Requests/Auth/	Validación de datos de entrada para cada endpoint de auth
app/Models/User.php	Modelo Eloquent del usuario, implementa <code>JWTSubject</code>
app/Services/AuthService.php	Lógica de negocio pura: registro, login, refresh, logout
config/auth.php	Configuración de guards (api → jwt) y providers (users → eloquent)
config/jwt.php	Configuración de JWT: TTL, algoritmo, claims, blacklist (se crea en Parte 3)
routes/api.php	Definición de rutas: públicas (login, register) y protegidas (user, logout)
.env	Secretos y configuración por entorno

6. Setup Inicial Paso a Paso

Paso 1: Crear el Proyecto

```
composer create-project laravel/laravel paradise-api
cd paradise-api
```

Esto descarga Laravel 11.x, instala dependencias, genera el archivo `.env` desde `.env.example` y crea la clave de aplicación.

Paso 2: Verificar .env y Clave de Aplicación

El archivo `.env` contiene las variables de entorno. Laravel lo genera automáticamente desde `.env.example`. En esta parte solo se verifica su existencia — la configuración específica para base de datos y JWT se cubre en la Parte 2.

```
php artisan key:generate
```

Qué hace: Genera una clave criptográfica aleatoria de 32 caracteres y la almacena en `APP_KEY` dentro de `.env`.

Por qué es crítico: Esta clave se usa para: - Encriptar sesiones y cookies. - Encriptar datos con `Crypt::encrypt()`. - Firmar tokens CSRF. - Derivar claves para el cifrado interno de Laravel.

Sin `APP_KEY`, cualquier dato encriptado por Laravel es inaccesible — incluyendo sesiones existentes y valores encriptados en base de datos.

Paso 3: Verificar que el Proyecto Corre

```
php artisan serve
```

Esto inicia el servidor de desarrollo en `http://localhost:8000`. Abrir esa URL en el navegador debe mostrar la página de bienvenida de Laravel.

Para detener: `Ctrl+C`.

Paso 4: Limpiar la Estructura por Defecto

Laravel incluye archivos de ejemplo que no se usarán en este sistema de autenticación. Eliminarlos mantiene el proyecto limpio:

```
# Eliminar migraciones por defecto que no se usarán
rm database/migrations/0001_01_01_000000_create_users_table.php
rm database/migrations/0001_01_01_000001_create_cache_table.php
rm database/migrations/0001_01_01_000002_create_jobs_table.php

# Eliminar modelos por defecto si existen
# El modelo User se reescribirá con la interfaz JWTSubject

# Eliminar controladores por defecto
rm app/Http/Controllers/*.php -f
```

Nota: Se eliminan las migraciones por defecto porque se crearán migraciones personalizadas con todos los campos necesarios para el sistema de auth (roles, tokens, timestamps). Esto se hace en la Parte 2.

Paso 5: Crear los Directorios de la Estructura

```
mkdir -p app/Enums
mkdir -p app/Exceptions/Auth
mkdir -p app/Http/Controllers/Auth
mkdir -p app/Http/Requests/Auth
mkdir -p app/Services/Contracts
```

7. Convenciones y Buenas Prácticas

Naming Conventions

Elemento	Convención	Ejemplo
Controladores	PascalCase + <code>Controller</code>	<code>LoginController</code>
FormRequests	PascalCase + <code>Request</code>	<code>LoginRequest</code>
Services	PascalCase + <code>Service</code>	<code>AuthService</code>
Métodos en services	camelCase, verbos de acción	<code>login()</code> , <code>register()</code> , <code>refreshTokens()</code>
Métodos en controllers	<code>__invoke()</code> (single-action)	<code>__invoke(LoginRequest \$r)</code>
Rutas API	kebab-case, plural para recursos	<code>/api/auth/login</code> , <code>/api/auth/register</code>
Tablas	snake_case, plural	<code>users</code>
Migraciones	snake_case, descriptivo	<code>create_users_table</code>
Enums	PascalCase, casos en PascalCase	<code>UserRole::Admin</code> , <code>UserStatus::Active</code>

Todo el código sigue **PSR-12** (PHP Standards Recommendation 12), que es el estándar de codificación que Laravel adopta.

Principios SOLID Aplicados a la Autenticación

Principio	Aplicación en el Sistema de Auth
Single Responsibility	Cada clase tiene una razón para cambiar: <code>LoginController</code> solo maneja login HTTP
Open/Closed	El <code>AuthService</code> se extiende sin modificar — nuevas estrategias de auth como interfaces
Liskov Substitution	Los contracts (<code>AuthServiceInterface</code>) garantizan que cualquier implementación sea sustituible
Interface Segregation	Interfaces pequeñas y específicas, no una interfaz monolítica de “auth”
Dependency Inversion	Controllers dependen de abstracciones (<code>AuthServiceInterface</code>), no de implementaciones concretas

DRY en Lógica de Auth

- La generación de tokens (access + refresh) es un método privado reutilizado en login y register.
- La estructura de respuesta JSON es un método privado del service, no se repite en cada controller.
- La validación de contraseñas (complejidad, confirmación) se define UNA vez en `RegisterRequest`.

Tipado Estricto

Cada archivo `.php` del sistema de auth comienza con:

```
declare(strict_types=1);
```

Esto obliga a PHP a no hacer coerción automática de tipos. Un `string` no se convierte a `int` implícitamente — el código falla con un `TypeError` explícito. Esto elimina bugs sutiles donde `"42"` se convierte mágicamente a `42`.

Uso de Enums

Los roles y estados de usuario se modelan con PHP Enums (disponibles desde PHP 8.1):

```
enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';
    case Moderator = 'moderator';
}

enum UserStatus: string
{
    case Active = 'active';
    case Inactive = 'inactive';
    case Banned = 'banned';
}
```

Ventajas sobre constantes de clase o strings mágicos:

- Type safety: `UserRole::Admin` es un tipo distinto, no un string cualquiera.
- Autocompletado en el IDE.
- No se puede asignar un valor inválido accidentalmente.
- Los backed enums (`: string`) se serializan automáticamente a la base de datos.

Manejo de Excepciones

El sistema distingue dos tipos de excepciones:

1. **Excepciones de dominio** (`app/Exceptions/Auth/`): Representan errores de negocio. Ejemplo: `InvalidCredentialsException` se lanza cuando las credenciales no coinciden. El Service Layer las lanza, el Controller (o el Exception Handler de Laravel) las convierte a respuestas HTTP.
2. **Excepciones HTTP**: Laravel las maneja automáticamente. Ejemplo: `ModelNotFoundException` → 404.

El `Handler` de excepciones de Laravel se configura para mapear excepciones de dominio a códigos HTTP:

```
// InvalidCredentialsException siempre es 401 Unauthorized
// TokenExpiredException siempre es 401 con un código interno específico
```

Response Standardization

TODAS las respuestas de la API siguen la misma estructura JSON:

```

// Éxito
{
  "data": {
    "user": { "id": 42, "name": "...", "email": "..."},
    "access_token": "eyJ...",
    "refresh_token": "eyJ...",
    "token_type": "Bearer",
    "expires_in": 3600
  }
}

// Error
{
  "error": {
    "code": "invalid_credentials",
    "message": "Las credenciales proporcionadas son incorrectas.",
    "details": {}
  }
}

// Error de validación (Laravel lo genera automáticamente)
{
  "message": "Los datos proporcionados no son válidos.",
  "errors": {
    "email": ["El campo email es obligatorio."],
    "password": ["La contraseña debe tener al menos 8 caracteres."]
  }
}

```

El contrato de respuesta es: - `data` siempre presente en éxito, contiene el payload de la respuesta. - `error` siempre presente en error, contiene `code` (string snake_case, identificador único) y `message` (string, descripción legible). - Los errores de validación mantienen el formato estándar de Laravel para compatibilidad con librerías frontend.

8. Resumen y Puente a la Parte 2

Lo Construido en Esta Parte

En la Parte 1 se establecieron los fundamentos conceptuales y estructurales del sistema de autenticación:

- Se definió el stack tecnológico y se justificó cada elección.
- Se analizó por qué Laravel con JWT puro es la decisión arquitectónica correcta para este caso de uso.
- Se diseñó la arquitectura en capas y los cinco flujos principales de autenticación.
- Se documentaron todas las decisiones de seguridad: bcrypt, HS256, claims JWT, token storage, CORS, HTTPS.
- Se definió la estructura de directorios y la responsabilidad de cada componente.
- Se creó el proyecto Laravel base y se limpió la estructura.
- Se establecieron las convenciones de código, naming, tipado, enums, excepciones y formato JSON de respuesta.

Lo Que Viene en la Parte 2

La Parte 2 cubre la configuración del entorno y la base de datos: - Configuración del archivo `.env` con todas las variables necesarias para el sistema de auth. - Creación de la base de datos MySQL. - Diseño del schema de la tabla `users` con todos los campos requeridos. - Migración y configuración de la conexión a base de datos. - Verificación de la conexión y primeras pruebas con Tinker.

Referencias para Partes Posteriores

Las siguientes decisiones tomadas en esta parte deben ser respetadas en las partes subsiguientes:

Decisión	Detalle
Namespace base	App\Services\AuthService y App\Http\Controllers\Auth*
Estructura JSON de respuesta	{ "data": {...} } para éxito, { "error": { "code": "...", "message": "..." } } para error
Nombres de métodos del Service	login() , register() , refreshTokens() , logout()
Enums	UserRole y UserStatus en App\Enums\
FormRequest naming	LoginRequest , RegisterRequest , RefreshTokenRequest en App\Http\Requests\Auth\
Algoritmo JWT	HS256 con clave secreta en JWT_SECRET (.env)
Token storage FE recomendado	Access token en memoria, refresh token en httpOnly cookie
Lifetime tokens	Access token: 15 min; Refresh token: 7 días (configurable en jwt.php)
Middleware JWT	JwtAuthenticate en App\Http\Middleware\
Guarda en auth.php	Guard api con driver jwt y provider users (modelo App\Models\User)
PSR-12 + tipado estricto	declare(strict_types=1) en todos los archivos

Parte 2: Variables de Entorno y Base de Datos

1. Introducción

En la [Parte 1](#) sentamos las bases del proyecto: instalamos Laravel 11, definimos la estructura de directorios (`App\Services\AuthService` , controllers single-action en `App\Http\Controllers\Auth\` , FormRequests en `App\Http\Requests\Auth\` , middleware JWT en `App\Http\Middleware\`), y establecimos las convenciones de respuesta JSON y tipado estricto que gobernarán todo el desarrollo. El esqueleto está listo.

El objetivo de **esta segunda parte** es triple:

1. **Configurar todas las variables de entorno** que el sistema de autenticación necesita — desde la conexión a base de datos hasta los parámetros JWT.
2. **Conectar Laravel con MySQL 8** creando la base de datos y verificando la conexión.
3. **Diseñar y crear la tabla** `users` mediante migraciones, junto con el modelo `User` , los enums `UserRole` y `UserStatus` , y los casts necesarios.

Al terminar esta parte tendremos la capa de datos completamente preparada para que la Parte 3 pueda instalar y configurar el paquete JWT sin fricción.

2. El archivo `.env` — Teoría y Práctica

2.1 ¿Qué es `.env` ?

El archivo `.env` es la implementación de Laravel del tercer factor de la metodología [The Twelve-Factor App](#): **almacenar la configuración en el entorno**. La premisa es simple pero poderosa: el código debe ser el mismo en desarrollo, staging y producción; lo que cambia entre entornos son los valores de configuración (credenciales de base de datos, claves secretas, URLs de API, etc.).

Laravel carga este archivo al iniciar la aplicación usando la librería [vlucas/phpdotenv](#), que parsea las líneas `CLAVE=valor` y las inyecta en las variables de entorno del proceso PHP (`$_ENV` y `getenv()`).

¿Por qué no hardcodear credenciales en `config/database.php` ?

```
// ✗ JAMÁS hagas esto:
'mysql' => [
    'password' => 'miSuperPassword123',
],

// ✔ Esto es lo correcto:
'mysql' => [
    'password' => env('DB_PASSWORD', ''),
],
```

Si el password está hardcodeado, cada desarrollador que clone el repo ve las credenciales. Si cambian, hay que modificar el código. Si el repo es público (o se vuelve público por accidente), las credenciales quedan expuestas para siempre en el historial de Git.

2.2 `.env.example` VS `.env`

Archivo	¿Se commitea?	Propósito
<code>.env</code>	NO	Contiene valores reales (passwords, keys, secrets). Es único por entorno y por desarrollador.
<code>.env.example</code>	Sí	Es una plantilla con todas las claves necesarias y valores de ejemplo/documentación. Sirve como referencia para que cualquier dev sepa qué variables debe definir.

Flujo típico al clonar un proyecto:

```
cd .env.example .env
php artisan key:generate
# Editar .env con los valores reales de tu entorno
```

Laravel incluye `.env` en `.gitignore` por defecto. **Nunca** elimines esa línea. Si por accidente alguna vez commiteaste un `.env` con secretos reales, considera esas credenciales **comprometidas** y rótalas inmediatamente.

2.3 Variables de entorno vs archivos de configuración de Laravel

Laravel tiene una capa de indirección entre `.env` y el código de la aplicación: los archivos dentro de `config/`. Observa cómo funciona:

```
// config/database.php
'mysql' => [
    'driver'      => 'mysql',
    'host'        => env('DB_HOST', '127.0.0.1'),
    'port'        => env('DB_PORT', '3306'),
    'database'    => env('DB_DATABASE', 'forge'),
    'username'    => env('DB_USERNAME', 'forge'),
    'password'    => env('DB_PASSWORD', ''),
    // ...
],
```

El helper `env('DB_HOST', '127.0.0.1')` lee de `.env` y, si la variable no existe, usa el segundo argumento como fallback. Luego, en cualquier parte del código usas `config('database.connections.mysql.host')`, **nunca** `env('DB_HOST')` directamente.

2.4 `php artisan config:cache` — cuándo y por qué

```
php artisan config:cache
```

Este comando fusiona todos los archivos de `config/` en un único archivo PHP compilado en `bootstrap/cache/config.php`. Beneficios:

- **Rendimiento:** elimina decenas de llamadas a `env()` y lecturas de archivos en cada request.
- **Atomicidad:** la configuración se carga de una sola pieza.

Regla de oro: después de ejecutar `config:cache`, la función `env()` **siempre retorna** `null` (porque las variables ya se leyeron y se “cachearon” en el archivo compilado). Por eso:

NUNCA uses `env()` fuera de los archivos de `config/`.

Si necesitas un valor de entorno en un Service, Controller o Command, agrégalo a un archivo de configuración existente (o crea uno nuevo en `config/`) y accede vía `config('clave')`.

En desarrollo (`APP_ENV=local`) no se suele cachear la configuración porque cambia con frecuencia. En producción, se ejecuta como parte del deploy.

3. Variables de Entorno para Autenticación

3.1 Bloque Database

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=paradise_roasters
DB_USERNAME=root
DB_PASSWORD=
```

Variable	Propósito	Valor recomendado
<code>DB_CONNECTION</code>	Driver de base de datos. Laravel soporta <code>mysql</code> , <code>pgsql</code> , <code>sqlite</code> , <code>sqlsrv</code> .	<code>mysql</code> para este proyecto
<code>DB_HOST</code>	Dirección del servidor MySQL. <code>127.0.0.1</code> si está en la misma máquina. En producción suele ser una IP privada o un nombre de host del proveedor de cloud.	<code>127.0.0.1</code> (dev), IP/hostname (prod)
<code>DB_PORT</code>	Puerto de MySQL. El estándar es <code>3306</code> .	<code>3306</code>
<code>DB_DATABASE</code>	Nombre de la base de datos. Usar snake_case, sin espacios ni caracteres especiales.	<code>paradise_roasters</code> o el nombre de tu app
<code>DB_USERNAME</code>	Usuario de conexión. En desarrollo suele ser <code>root</code> por simplicidad.	<code>root</code> (dev), usuario dedicado (prod)
<code>DB_PASSWORD</code>	Contraseña del usuario. En blanco solo si usas root sin password en dev local. En producción: contraseña fuerte, almacenada en un gestor de secretos.	(vacío en dev local), valor seguro en prod

¿Por qué MySQL y no SQLite/PostgreSQL?

- **SQLite** es excelente para pruebas y desarrollo ligero, pero no soporta concurrencia real de escritura — dos requests simultáneos que intenten registrar usuarios pueden bloquearse mutuamente. Para producción con múltiples usuarios, necesitas un motor cliente-servidor.
- **PostgreSQL** es una alternativa perfectamente válida y muchos equipos la prefieren. La decisión aquí es MySQL

porque es lo definido en el stack del proyecto. Laravel abstrae la diferencia — cambiar de motor más adelante es mayormente transparente si usas Eloquent y migraciones.

- **MySQL 8** trae soporte nativo de JSON, window functions, CTEs recursivas, y mejor manejo de `utf8mb4` que su predecesor. Es la versión mínima recomendada.

Charset y Collation

En `config/database.php`, Laravel ya configura:

```
'charset' => 'utf8mb4',  
'collation' => 'utf8mb4_unicode_ci',
```

Término	Significado
utf8mb4	La implementación completa de UTF-8 en MySQL. El <code>utf8</code> simple de MySQL (<code>utf8mb3</code>) solo soporta el Basic Multilingual Plane (3 bytes por carácter), dejando fuera emojis (🤪,) y caracteres de escrituras asiáticas extendidas. <code>utf8mb4</code> usa hasta 4 bytes por carácter y cubre TODO el estándar Unicode.
utf8mb4_unicode_ci	Collation que define cómo se comparan y ordenan los strings. El sufijo <code>_ci</code> significa case-insensitive : <code>'usuario@email.com'</code> = <code>'USUARIO@EMAIL.COM'</code> para búsquedas y constraints UNIQUE. La variante <code>unicode</code> usa el algoritmo de comparación oficial de Unicode; <code>general</code> es más rápido pero menos preciso en ordenamiento multilingüe.

Para autenticación esto es crítico: el índice UNIQUE sobre `email` con collation case-insensitive evita que alguien registre `Usuario@Email.com` y `usuario@email.com` como cuentas diferentes.

3.2 Bloque JWT

```
JWT_SECRET=  
JWT_TTL=15  
JWT_REFRESH_TTL=10080
```

Variable	Significado	Valor recomendado
<code>JWT_SECRET</code>	Clave criptográfica usada para firmar los tokens con HMAC-SHA256. Es el equivalente JWT de <code>APP_KEY</code> . Si se filtra, cualquiera puede forjar tokens y suplantar usuarios.	Debe ser un string aleatorio largo (al menos 256 bits/32 bytes). El comando <code>php artisan jwt:secret</code> lo genera automáticamente. Nunca lo escribas a mano ni uses palabras de diccionario.
<code>JWT_TTL</code>	Time To Live del access token, en minutos . Pasado este tiempo, el token expira y debe renovarse con un refresh token.	<code>15</code> minutos. Es el balance estándar entre seguridad (ventana de exposición corta si un token se filtra) y usabilidad (no estar renovando cada 30 segundos). Para SPAs y mobile apps es común usar 15-60 minutos.
<code>JWT_REFRESH_TTL</code>	Tiempo de vida del refresh token, en minutos . El refresh token es un token de larga duración que permite obtener nuevos access tokens sin pedir credenciales de nuevo.	<code>10080</code> (7 días). Suficiente para que un usuario no tenga que loguearse cada día, pero acotado para que sesiones abandonadas no persistan indefinidamente.

¿Por qué access token de corta duración + refresh token de larga duración?

Este patrón de dos tokens minimiza el riesgo: si un access token se filtra (ej: queda en logs del frontend, se intercepta en una red), solo es válido por 15 minutos. El refresh token, que vive más tiempo, idealmente **nunca** sale del servidor o se almacena de forma más segura (httpOnly cookie). En la Parte 5 implementaremos la rotación de refresh tokens para mayor seguridad.

En la Parte 3 generaremos `JWT_SECRET` con:

```
php artisan jwt:secret
```

Este comando añade automáticamente la clave al archivo `.env`:

```
JWT_SECRET=base64:g7XpL9mNqR...restoDelHash
```

3.3 Bloque App

```
APP_NAME="Paradise Roasters"
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8000
```

Variable	Relevancia para autenticación
APP_NAME	Se usa en notificaciones y correos (ej: "Paradise Roasters - Restablecer contraseña"). No es crítico para el flujo de login, pero aparece en respuestas de email si implementas verificación o reseteo.
APP_ENV	local para desarrollo, production para producción. Afecta APP_DEBUG y el nivel de detalle en errores. En producción, los errores de autenticación nunca deben exponer stack traces ni detalles de infraestructura.
APP_DEBUG	true en desarrollo (ver errores detallados en Postman/Insomnia), false en producción. Si APP_DEBUG=true en producción, un 500 de login podría mostrar consultas SQL, paths del servidor, y credenciales en los mensajes de error — catastrófico para seguridad.
APP_URL	URL base de la aplicación. Si implementarías email verification o password reset, Laravel usaría APP_URL para generar los links de verificación. Para una API pura sin frontend server-rendered, no es tan crítico, pero es buena práctica tenerlo bien definido.

3.4 Variables adicionales de seguridad

```
SESSION_DRIVER=file
CORS_ALLOWED_ORIGINS=http://localhost:3000
```

Variable	Explicación
SESSION_DRIVER	Laravel por defecto inicia sesiones incluso en APIs. Con JWT no dependemos de sesiones — el estado de autenticación viaja en el token, no en una cookie de sesión en el servidor. Dejamos file como valor explícito para documentar la decisión, pero en producción puedes configurarlo como array (en memoria) o null para que Laravel no toque el storage de sesiones. La arquitectura es stateless .
CORS_ALLOWED_ORIGINS	Si tu frontend Vue/React/Next.js corre en localhost:3000, necesitas permitir ese origen en las respuestas CORS. Laravel 11 usa el middleware HandleCors y el archivo config/cors.php. Sin esta configuración, el navegador bloqueará las peticiones de login desde el frontend. En producción, aquí irá la URL real de tu SPA.

4. Configuración de MySQL

4.1 Crear la base de datos

Conéctate a MySQL (como root o con un usuario administrador) y ejecuta:

```
CREATE DATABASE paradise_roasters
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

Puedes verificarla con:

```
SHOW CREATE DATABASE paradise_roasters;
```

El resultado debe mostrar `CHARACTER SET utf8mb4` y `COLLATE utf8mb4_unicode_ci`.

¿Por qué definir charset y collation a nivel de base de datos?

Es una defensa en profundidad: aunque Laravel lo especifique en la conexión, definir el default de la base de datos evita que futuras herramientas de administración, backups restaurados a mano, o migraciones ejecutadas desde otro cliente creen objetos con el charset incorrecto (`latin1`), que es el default de MySQL y **no** soporta caracteres como `ñ`, `é`, o emojis).

4.2 Crear un usuario dedicado (buena práctica)

En desarrollo local con `root` sin password es aceptable, pero **en producción nunca uses root**. Crea un usuario específico con permisos acotados a la base de datos de la aplicación:

```
CREATE USER 'pr_app'@'localhost' IDENTIFIED BY 'unaContraseñaMuyFuerte123!';
GRANT ALL PRIVILEGES ON paradise_roasters.* TO 'pr_app'@'localhost';
FLUSH PRIVILEGES;
```

Buenas prácticas para el password en producción

Mínimo 16 caracteres, con mayúsculas, minúsculas, números y símbolos

Almacenarlo en un gestor de secretos (HashiCorp Vault, AWS Secrets Manager, Doppler, etc.), no en `.env` directamente si el equipo es grande

Rotarlo periódicamente (cada 90 días como mínimo)

No usar el mismo password que `APP_KEY` o `JWT_SECRET`

Luego, actualiza `.env` con este usuario:

```
DB_USERNAME=pr_app
DB_PASSWORD=unaContraseñaMuyFuerte123!
```

4.3 Probar la conexión

Sin migraciones todavía, podemos verificar que Laravel llega a MySQL:

```
php artisan migrate:status
```

Deberías ver un error similar a:

```
Migration table not found.
```

Esto **confirma** que la conexión funciona — Laravel se conectó a MySQL, buscó la tabla `migrations`, no la encontró, y te lo dijo claramente. La crearemos en la sección 8 al ejecutar `php artisan migrate`.

Si en cambio ves `SQLSTATE[HY000] [1045] Access denied` o `SQLSTATE[HY000] [2002] Connection refused`, revisa credenciales, host y puerto en tu `.env`.

5. La Migración de Users — Diseño de la Tabla

5.1 Crear la migración

```
php artisan make:migration create_users_table
```

Laravel detecta el nombre `create_users_table` y genera el esqueleto con `Schema::create('users', ...)` automáticamente. Sobrescribe el contenido del archivo generado en `database/migrations/` con el siguiente código completo:

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->string('role')->default('user');
            $table->string('status')->default('active');
            $table->rememberToken();
            $table->timestamps();
            $table->softDeletes();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('users');
    }
};
```

5.2 Análisis columna por columna

```
$table->id()
```

Equivalente a `$table->bigIncrements('id')`. Genera:

```
`id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY
```

- **BIGINT** (8 bytes, rango $\approx 9.2 \times 10^{18}$) en vez de **INT** (4 bytes, rango $\approx 2.1 \times 10^9$). ¿Por qué? Una aplicación con millones de usuarios alcanzaría el límite de **INT** más rápido de lo que parece. El costo de almacenamiento extra (4 bytes por fila) es insignificante comparado con el costo de migrar una tabla de **INT** a **BIGINT** en producción con millones de registros. Es **escalabilidad preventiva**.
- **UNSIGNED** : los IDs no deberían ser negativos. Duplica el rango positivo disponible.
- **AUTO_INCREMENT** : MySQL asigna el siguiente valor secuencial automáticamente.

```
$table->string('name')
```

Genera `VARCHAR(255) NOT NULL`.

- `255` es el default de Laravel para `string()`. Es más que suficiente para nombres reales (el nombre más largo registrado tiene ~200 caracteres; un `VARCHAR(255)` cubre cualquier caso práctico).
- **Validación de longitud mínima**: MySQL no la impone. La responsabilidad de rechazar un nombre vacío o de 1 carácter está en la capa de validación (`RegisterRequest` en la Parte 4), no en la base de datos. La DB debe ser flexible; la aplicación, estricta.
- **NOT NULL**: un usuario sin nombre no tiene sentido en este dominio. Si necesitaras registro solo con email, usarías `nullable()`.

```
$table->string('email')->unique()
```

Genera:

```
`email` VARCHAR(255) NOT NULL,
UNIQUE KEY `users_email_unique` (`email`)
```

- **UNIQUE** : a nivel de base de datos, **garantiza** que no puedan existir dos filas con el mismo email, incluso si dos requests llegan simultáneamente. El índice UNIQUE es atómico — MySQL rechaza el segundo INSERT duplicado. Esto es defensa en profundidad: aunque el código falle, la integridad de datos está protegida en la capa más baja.
- **Índice automático**: MySQL crea implícitamente un índice B-tree sobre la columna `email` para hacer eficiente el constraint UNIQUE. Ese mismo índice acelera `SELECT ... WHERE email = ?` — que es exactamente la consulta del login. **No necesitas añadir `->index()` manualmente.**
- **Collation case-insensitive**: como MySQL usa `utf8mb4_unicode_ci`, la comparación `WHERE email = 'USUARIO@email.com'` encuentra `usuario@email.com`. Sin esto, `Usuario@Email.com` y `usuario@email.com` serían emails diferentes para la base de datos — una pesadilla de soporte.
- **¿Longitud del email?** El RFC 5321 permite hasta 254 caracteres en un email completo. `VARCHAR(255)` cubre el caso máximo.

```
$table->timestamp('email_verified_at')->nullable()
```

- **Tipo `TIMESTAMP`** : almacena fecha y hora en UTC internamente (Laravel convierte a la zona horaria de la app al leer).
- `nullable()` : un usuario recién registrado no tiene email verificado todavía. `NULL` significa “no verificado”; un valor no-nulo significa “verificado en fecha X”.
- **No lo implementaremos en este manual**, pero la columna está lista para cuando quieras añadir verificación por email. Es más fácil incluirla ahora que añadir una migración nueva después.

```
$table->string('password')
```

Genera `VARCHAR(255) NOT NULL`.

“¿Por qué 255 caracteres si bcrypt genera hashes de exactamente 60?”

Buena observación. bcrypt produce strings de 60 caracteres:

```
$2y$12$LJ3m4ys3GZfnYMz8kVsKa0mxAZjFPqFq7dQ8eY0.XyK0mWxj501Nu
```

Pero:

Razón	Detalle
Argon2	El algoritmo Argon2id (recomendado por OWASP y PHP $\geq$ 7.3) genera hashes de ~96 caracteres. Si en el futuro Laravel cambia el driver de hashing por defecto, nuestra tabla lo soporta sin migración.
Futuros algoritmos	No sabemos qué algoritmo será estándar en 5 años. Podría generar hashes de 120 o 200 caracteres. <code>VARCHAR(255)</code> es barato en almacenamiento y nos da margen.
Convención de Laravel	El propio Laravel usa <code>\$table->string('password')</code> (255) en sus migraciones por defecto. Seguir la convención del framework evita sorpresas.

NUNCA guardes contraseñas en texto plano. Ni siquiera temporalmente, ni siquiera en logs. Laravel 11 con el cast `'password' => 'hashed'` lo maneja automáticamente (sección 6), pero si alguna vez escribes lógica manual de asignación de password, siempre usa `Hash::make()` o `bcrypt()`.

```
$table->string('role')->default('user')
```

- **Tipo** `VARCHAR` con default `'user'`. Los valores provienen del enum PHP `UserRole` (sección 7): `'admin'` y `'user'`.
- **¿Por qué no el tipo ENUM nativo de MySQL?**

```
-- Esto es lo que NO vamos a usar:  
role ENUM('user', 'admin') NOT NULL DEFAULT 'user'
```

Razones para evitarlo:

1. **Añadir valores requiere ALTER TABLE.** Si mañana necesitas un rol `moderator`, `editor` o `super_admin`, un ENUM de MySQL te obliga a `ALTER TABLE ... MODIFY COLUMN ...`
`ENUM('user', 'admin', 'moderator', 'editor', 'super_admin')` — una operación que en tablas grandes puede bloquear escrituras durante minutos.
2. **El orden de los valores importa.** Si cambias el orden de los miembros del ENUM o eliminas uno del medio, MySQL reenumera los índices numéricos internos, lo que puede corromper datos silenciosamente.
3. **Acoplamiento DB-aplicación.** La lógica de “qué roles existen y qué pueden hacer” pertenece al código PHP (Enums, Políticas, Gates), no al schema de la base de datos.

Con `VARCHAR` + validación PHP + cast de Enum en Eloquent, añadir un nuevo rol es instantáneo: agregas un case al enum PHP y listo. La base de datos no se entera.

```
$table->string('status')->default('active')
```

- Mismo razonamiento que `role`. Valores posibles (definidos en `UserStatus`, sección 7): `'active'`, `'inactive'`, `'banned'`.
- `active`: usuario normal, puede loguearse.
- `inactive`: cuenta desactivada (por el usuario o por un admin). El login debe rechazarlo con un mensaje específico.

- `banned` : cuenta bloqueada por violación de términos. Similar a `inactive` pero con semántica distinta (útil para auditoría y UI).

`$table->rememberToken()`

Genera `VARCHAR(100) NULL`. Es parte del sistema nativo de Laravel para la funcionalidad “Remember Me” en sesiones web tradicionales.

- En este manual **no lo usaremos** (es una API stateless con JWT), pero mantenerlo no cuesta nada y mantiene compatibilidad con herramientas de Laravel como `Auth::login()` si alguna vez necesitas un panel admin con sesiones.
- Si estás 100% seguro de que nunca usarás autenticación basada en sesiones, puedes eliminarlo. Personalmente recomiendo dejarlo: no ocupa espacio significativo y eliminar columnas nativas del framework puede romper paquetes de terceros que asuman su existencia.

`$table->timestamps()`

Genera dos columnas:

```
`created_at` TIMESTAMP NULL
`updated_at` TIMESTAMP NULL
```

- `created_at` : se establece automáticamente al insertar (`CURRENT_TIMESTAMP` por defecto en MySQL).
- `updated_at` : se actualiza automáticamente en cada `UPDATE` (Eloquent lo maneja).
- **Utilidad en autenticación:** saber cuándo se registró un usuario (para estadísticas, rate limiting, detección de cuentas spam creadas en lote) y cuándo fue su último cambio (último cambio de password, actualización de perfil). En la Parte 5 podríamos añadir auditoría extra, pero con `created_at` y `updated_at` ya tenemos lo mínimo indispensable.

`$table->softDeletes()`

Genera:

```
`deleted_at` TIMESTAMP NULL
```

El trait `SoftDeletes` de Eloquent convierte las operaciones `delete()` en `UPDATE deleted_at = NOW()` en vez de `DELETE FROM`. El usuario “eliminado” sigue en la base de datos, pero Eloquent lo excluye automáticamente de todas las consultas normales.

Implicaciones para autenticación:

Situación	Comportamiento
Usuario activo (<code>deleted_at IS NULL</code>)	Puede loguearse normalmente.
Usuario con soft-delete (<code>deleted_at IS NOT NULL</code>)	<code>User::where('email', \$email)->first()</code> no lo encuentra (Eloquent añade <code>WHERE deleted_at IS NULL</code> automáticamente). El login devuelve “credenciales inválidas”.
Mismo email registrado de nuevo	Como el email sigue en la tabla con <code>deleted_at = NOW()</code> , el índice UNIQUE lo bloquea . Necesitas manejar este caso: o restauras el usuario soft-deleted, o usas un índice UNIQUE compuesto <code>UNIQUE(email, deleted_at)</code> que es el patrón correcto.

Atención con índices UNIQUE + softDeletes: MySQL no ignora valores `NULL` en índices UNIQUE (permite múltiples `NULL`). Sin embargo, si necesitas permitir que un email “eliminado” se re-registre, deberías modificar la migración para usar un índice compuesto. En este manual usaremos el índice UNIQUE simple por simplicidad — si un usuario se elimina, su email no puede reutilizarse a menos que se restaure la cuenta o se purgue físicamente.

6. El Modelo User

Crea (o sobrescribe) el archivo `app/Models/User.php` con el siguiente contenido:

```

<?php

declare(strict_types=1);

namespace App\Models;

use App\Models\UserRole;
use App\Models\UserStatus;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Notifications\Notifiable;
use Tymon\JWTAuth\Contracts\JWTSubject;

class User extends Authenticatable implements JWTSubject
{
    use Notifiable, SoftDeletes;

    protected $fillable = [
        'name',
        'email',
        'password',
        'role',
        'status',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password'          => 'hashed',
            'role'               => UserRole::class,
            'status'             => UserStatus::class,
        ];
    }

    /**
     * Get the identifier that will be stored in the subject claim of the JWT.
     *
     * @return mixed
     */
    public function getJWTIdentifier()
    {
        // Se implementará en la Parte 3
        return $this->getKey();
    }

    /**
     * Return a key value array, containing any custom claims to be added to the JWT.
     *
     * @return array
     */
    public function getJWTCustomClaims()
    {
        // Se implementará en la Parte 3
        return [];
    }
}

```

6.1 extends Authenticatable

`Illuminate\Foundation\Auth\User` (alias `Authenticatable`) extiende el Model de Eloquent añadiendo los métodos que Laravel necesita internamente para autenticación:

Método	Propósito
<code>getAuthIdentifier()</code>	Retorna la PK del usuario (el <code>id</code>). Lo usa el guard JWT para identificar al usuario autenticado.
<code>getAuthPassword()</code>	Retorna el hash del password. Lo usa <code>Hash::check()</code> en el intento de login.
<code>getRememberToken()</code> / <code>setRememberToken()</code>	Para sesiones web tradicionales (no las usaremos).

Al extender esta clase en vez de `Model` directamente, nuestro modelo es compatible con guards, Gates, Policies, y cualquier paquete de autenticación.

6.2 `implements JWTSubject`

La interfaz `Tymon\JWTAuth\Contracts\JWTSubject` requiere dos métodos:

- `getJWTIdentifier()`: qué valor poner en el claim `sub` del JWT. Típicamente, `$this->getKey()` (el `id` del usuario).
- `getJWTCustomClaims()`: array asociativo de claims personalizados que se incluirán en el payload del token. Útil para roles, permisos, o metadatos.

La implementación actual es un placeholder — los métodos están definidos con return values que funcionan (ver Parte 3 para la versión final), pero la lógica de claims personalizados (rol, status) la refinaremos en la Parte 3 cuando configuremos el guard JWT.

¿Por qué declararlo ahora y no en la Parte 3? Porque el modelo necesita esta interfaz para que el guard JWT lo reconozca como sujeto de token, y queremos que el modelo esté completo antes de instalar y configurar el paquete. Así evitamos el clásico error de “Class User does not implement JWTSubject” al primer intento de login.

6.3 `SoftDeletes`

El trait añade automáticamente:

- Scope global:** `WHERE deleted_at IS NULL` en todas las consultas de Eloquent (select, update, delete).
- Scopes locales:** `User::withTrashed()`, `User::onlyTrashed()`.
- Métodos:** `$user->trashed()` (bool), `$user->restore()`.
- Eventos:** `deleting` → `soft-delete`, `restoring` → `deleted_at = null`.

En el contexto de autenticación: un usuario soft-deleted **no** será encontrado por `User::where('email', ...)->first()` en el intento de login. Para el sistema, es como si no existiera. En la Parte 5 evaluaremos si queremos dar un mensaje específico (“esta cuenta fue eliminada”) o tratarlo como credenciales inválidas genéricas (recomendado por seguridad: no revelar si una cuenta existe o no).

6.4 `$fillable` vs `$guarded`

```
protected $fillable = [
    'name',
    'email',
    'password',
    'role',
    'status',
];
```

Esto es una **whitelist explícita**: solo estas columnas pueden asignarse masivamente. Cualquier intento de asignar otra columna vía `User::create($data)` o `$user->update($data)` es **silenciosamente ignorado** por Eloquent.

La alternativa `$guarded` (blacklist):

```
// Si usaras $guarded, dirías "todas las columnas
// pueden asignarse masivamente excepto estas":
protected $guarded = ['id', 'created_at', 'updated_at', 'deleted_at'];
```

¿Por qué `$fillable` es mejor?

1. **Principio de mínimo privilegio:** solo expones lo que necesitas. Si añades una columna `is_admin` o `balance` a la tabla en el futuro, `$guarded` la expondría automáticamente a mass assignment. `$fillable` la mantiene protegida hasta que explícitamente la añadas.
2. **Documentación implícita:** leer `$fillable` te dice exactamente qué campos espera recibir un `RegisterRequest` o `UpdateProfileRequest`.
3. **Seguridad por defecto:** el día que estás cansado y olvidas actualizar `$guarded`, `$fillable` te protege. En seguridad defensiva, el default debe ser “no” a menos que explícitamente digas “sí”.

6.5 `$hidden`

```
protected $hidden = [
    'password',
    'remember_token',
];
```

Cuando serializas un modelo a JSON (respuesta de API, `->toArray()`, `->toJson()`), estas columnas se omiten:

```
// ✓ Lo que el frontend recibe:
{
    "id": 1,
    "name": "Carlos",
    "email": "carlos@email.com",
    "role": "user",
    "status": "active",
    "created_at": "2025-01-15T10:30:00.000000Z",
    "updated_at": "2025-06-01T14:22:00.000000Z"
}

// ✗ Lo que NUNCA debe ver:
{
    "password": "$2y$12$LJ3m4ys3GZfnYMz8kVsKa0mxAZjFPqFq7dQ8eY0.XyK0mwXj501Nu",
    "remember_token": "aB3xK9..."
}
```

Esto es crítico para seguridad: exponer el hash del password es un desastre — aunque bcrypt sea resistente a fuerza bruta, revela información (cost factor, algoritmo) que facilita ataques dirigidos. `$hidden` es tu última línea de defensa si accidentalmente retornas `$user` en una respuesta JSON.

6.6 `casts()` — la magia de Laravel 11

```
protected function casts(): array
{
    return [
        'email_verified_at' => 'datetime',
        'password'         => 'hashed',
        'role'              => UserRole::class,
        'status'            => UserStatus::class,
    ];
}
```

Cast 'password' => 'hashed'

Nuevo en Laravel 10+ (mejorado en 11). Lo que hace:

```
// Sin el cast:
$user->password = Hash::make('plaintext123');

// Con el cast 'password' => 'hashed':
$user->password = 'plaintext123'; // Se hashea automáticamente al guardar
```

¿Qué pasa internamente? El cast `hashed` envuelve la asignación con `Hash::make()`. Se ejecuta cuando el modelo se guarda (`save()`, `create()`, `update()`, `fill()` + `save()`). Si el password ya está hasheado (empieza con `$2y$`), no lo re-hashea — detecta que ya es un hash bcrypt válido y lo deja pasar.

Esto elimina una fuente común de bugs: olvidar `Hash::make()` al crear usuarios en seeders, factories, tests o comandos artisan.

Nota para Laravel 10.x: el cast `'hashed'` existe desde Laravel 10, pero la sintaxis exacta de `protected function casts(): array` (método, no propiedad) es de Laravel 11. En Laravel 10 usarías:

```
protected $casts = [
    'password' => 'hashed',
];
```

Cast UserRole::class y UserStatus::class

Los Backed Enums de PHP 8.1 se integran con Eloquent automáticamente:

```
// Al leer de la DB:
$user = User::first();
// $user->role es un objeto UserRole, no un string
if ($user->role === UserRole::Admin) { /* ... */ }

// Al escribir:
$user->role = UserRole::User; // Se guarda como 'user' en la DB
$user->role = 'admin';        // También funciona: se convierte a UserRole::Admin

// Al comparar con strings directamente NO funciona sin cast:
// $user->role === 'admin' // false (es UserRole::Admin, no string)
// $user->role->value === 'admin' // true
```

7. Los Enums de Usuario

7.1 UserRole

Crea el archivo `app/Enums/UserRole.php`:

```
<?php

declare(strict_types=1);

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';
}
```

7.2 UserStatus

Crea el archivo `app/Enums/UserStatus.php`:

```
<?php

declare(strict_types=1);

namespace App\Enums;

enum UserStatus: string
{
    case Active = 'active';
    case Inactive = 'inactive';
    case Banned = 'banned';
}
```

7.3 ¿Por qué Backed Enums (: string)?

Un Backed Enum asigna un valor escalar (string o int) a cada caso. Esto es necesario para serialización a base de datos:

```
UserRole::Admin->value // 'admin'
UserRole::from('user') // UserRole::User
UserRole::tryFrom('super') // null (no lanza excepción)
```

Un enum puro (`enum Suit { case Hearts; }`) no tiene `.value` ni `.from()` — solo existe como identidad en memoria PHP. No puede persistirse a una columna VARCHAR automáticamente.

7.4 Ventajas sobre constantes de clase

El enfoque antiguo (pre-PHP 8.1):

```
class UserRole {
    const ADMIN = 'admin';
    const USER = 'user';
}
```

Problemas:

Constantes de clase	Backed Enums
<code>UserRole::ADMIN</code> es un string, no un tipo. Cualquier función que reciba <code>string \$role</code> acepta cualquier string.	<code>UserRole</code> es un tipo . Una función <code>assignRole(UserRole \$role)</code> rechaza strings arbitrarios en tiempo de compilación.
No hay forma de iterar todos los roles posibles.	<code>UserRole::cases()</code> devuelve un array de todos los casos. Perfecto para reglas de validación.
No hay pattern matching exhaustivo.	<code>match (\$role) { UserRole::Admin => ..., UserRole::User => ... }</code> — si olvidas un caso, PHP lanza <code>UnhandledMatchError</code> .
Sin métodos propios.	<code>UserRole</code> puede tener métodos: <code>canManageUsers(): bool</code> , <code>label(): string</code> , etc.

8. Ejecución y Verificación

8.1 Ejecutar migraciones

```
php artisan migrate
```

Salida esperada:

```
Migrating: 2014_10_12_000000_create_users_table
Migrated: 2014_10_12_000000_create_users_table (123.45ms)
```

8.2 Verificar con migrate:status

```
php artisan migrate:status
```

Salida esperada:

```
+-----+-----+-----+-----+
| Ran? | Migration                                     | Batch |
+-----+-----+-----+-----+
| Yes  | 2014_10_12_000000_create_users_table         | 1      |
+-----+-----+-----+-----+
```

8.3 Probar con Tinker

```
php artisan tinker
```

Dentro de Tinker:

```
// Crear un usuario de prueba
$user = User::create([
    'name'      => 'Carlos Prueba',
    'email'     => 'carlos@prueba.com',
    'password'  => 'password123',
    'role'      => UserRole::Admin,
    'status'    => UserStatus::Active
]);

// Verificar que el password está hashado
echo $user->password;
// Deberías ver algo como: $2y$12$LJ3m4ys3GZfnYMz8kVsKa0mxAZjFPqFq7dQ8eY0...

// Verificar casts de enum
echo get_class($user->role); // App\Enums\UserRole
echo $user->role->value;      // admin

echo get_class($user->status); // App\Enums\UserStatus
echo $user->status->value;      // active

// Verificar que el usuario se persiste correctamente
$found = User::where('email', 'carlos@prueba.com')->first();
echo $found->name; // Carlos Prueba
```

8.4 (Opcional) Crear UserFactory

Si quieres generar usuarios de prueba rápidamente, crea o modifica `database/factories/UserFactory.php` :

```
<?php

declare(strict_types=1);

namespace Database\Factories;

use App\Models\UserRole;
use App\Models\UserStatus;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Facades\Hash;

class UserFactory extends Factory
{
    public function definition(): array
    {
        return [
            'name' => fake()->name(),
            'email' => fake()->unique()->safeEmail(),
            'email_verified_at' => now(),
            'password' => Hash::make('password'), // Factory no usa el cast 'hashed' directamente
            'role' => UserRole::User,
            'status' => UserStatus::Active,
        ];
    }
}
```

Desde Tinker:

```
User::factory()->count(5)->create();
User::count(); // 6 (los 5 nuevos + el de prueba anterior)
```

9. Seguridad en la Capa de Datos

9.1 SQL Injection: protección automática de Eloquent

Toda consulta a través del Query Builder de Laravel usa **prepared statements** con parámetros enlazados (bindings):

```
// ✔ Seguro – Eloquent usa PDO prepared statements
User::where('email', $email)->first();

// El SQL que llega a MySQL es:
// SELECT * FROM users WHERE email = ? AND deleted_at IS NULL
// El valor de $email se envía como parámetro separado, no interpolado
```

El **riesgo real** está en consultas raw con interpolación manual:

```
// ✗ PELIGROSO – SQL Injection directa
$susers = DB::select("SELECT * FROM users WHERE email = '$email'");

// ✗ IGUAL DE PELIGROSO – DB::raw() no escapa por sí solo
User::whereRaw("email = '$email'")->get();
```

La forma segura de usar raw queries es con bindings nombrados o posicionales:

```
// ✓ Seguro – bindings posicionales
User::whereRaw('email = ?', [$email])->get();

// ✓ Seguro – bindings nombrados
DB::select('SELECT * FROM users WHERE email = :email', ['email' => $email]);
```

Regla práctica: si ves concatenación de strings (`.`) con input de usuario dentro de `DB::raw()`, `whereRaw()`, `orderByRaw()`, o `selectRaw()`, detente. Hay una forma mejor con bindings.

9.2 Mass Assignment: `$fillable` como protección

Sin `$fillable`, un atacante podría enviar un campo extra en el JSON del registro:

```
// POST /api/auth/register
{
  "name": "Atacante",
  "email": "atacante@evil.com",
  "password": "password123",
  "role": "admin" // ~ Campo malicioso
}
```

Si el modelo tuviera `$guarded = []` o no definiera `$fillable`, ese `'role' => 'admin'` se asignaría. Con nuestro `$fillable` explícito que **incluye** `'role'`, el campo se acepta pero pasa por validación del `RegisterRequest` (Parte 4) que lo rechazará. La combinación de `$fillable` + FormRequest validation es defensa en profundidad.

CVEs históricos de mass assignment en Laravel han ocurrido cuando: - Se usaba `$guarded = []` (sin blacklist). - Se olvidaba definir `$fillable` o `$guarded` (el default de Laravel 4 era permitir todo). - Paquetes de terceros asignaban `Input::all()` directamente al modelo.

Siempre define `$fillable` con la lista exacta de columnas asignables.

9.3 Timing Attacks en login

Un timing attack mide el tiempo que tarda el servidor en responder para inferir información. Por ejemplo, si la comparación de passwords usa `===` de PHP:

```
// ✗ Vulnerable a timing attack: strcmp para temprano si difiere el primer byte
if ($userInputPassword === $storedHash) { ... }
```

Laravel usa `Hash::check()`, que internamente emplea `password_verify()` de PHP (o `hash_equals()` para comparaciones genéricas). Ambas funciones son **timing-attack-safe**: comparan carácter por carácter y **siempre recorren el string completo**, sin cortocircuitar ante diferencias. El tiempo de respuesta es constante independientemente de si el hash coincide o no.

En la Parte 5 veremos cómo todo el flujo de login debe ser timing-safe: email inválido y password incorrecta deben tomar el mismo tiempo para no filtrar qué usuarios existen en la base de datos.

9.4 Hasheo automático con cast `'hashed'`

El cast `'password' => 'hashed'` previene el error humano más común en autenticación:

```
// ✖ Error humano clásico: olvidar Hash::make()
$user->password = $request->password; // Guarda "password123" en texto plano
$user->save();

// ✔ Con el cast 'hashed', esto automáticamente se convierte en:
$user->password = $request->password;
// Internamente: $this->attributes['password'] = Hash::make('password123')
$user->save();
```

El cast es tu **red de seguridad** — incluso si un desarrollador nuevo o cansado olvida `Hash::make()`, el modelo lo hace automáticamente. Solo debes tener cuidado en contextos donde no se usa Eloquent (raw `DB::table('users')->insert([...])`), donde el cast no aplica.

10. Resumen y Puente a la Parte 3

Lo construido en esta parte

Componente	Archivo	Estado
Variables de entorno	<code>.env</code>	Configuradas (DB, JWT, App, seguridad)
Base de datos MySQL	Servidor MySQL 8	Creada con charset <code>utf8mb4</code>
Tabla <code>users</code>	Migración <code>create_users_table</code>	Creada con 11 columnas, índices, soft deletes
Modelo <code>User</code>	<code>app/Models/User.php</code>	Extiende <code>Authenticatable</code> , implements <code>JWTSubject</code> , casts de enum y hashed password
Enum <code>UserRole</code>	<code>app/Enums/UserRole.php</code>	<code>Admin</code> y <code>User</code>
Enum <code>UserStatus</code>	<code>app/Enums/UserStatus.php</code>	<code>Active</code> , <code>Inactive</code> , <code>Banned</code>

Decisiones de diseño que las Partes 3-6 deben conocer

A continuación, un resumen taxativo de **todo lo que las siguientes partes del manual deben asumir como verdad establecida**. Esto incluye nombres exactos de columnas, clases, métodos, constantes, y decisiones arquitectónicas que **no deben alterarse** en las Partes 3, 4, 5 y 6.

Base de datos

Elemento	Valor exacto
Nombre de la tabla	users
Columna PK	id (BIGINT UNSIGNED AUTO_INCREMENT)
Columna de email	email (VARCHAR(255), UNIQUE, NOT NULL)
Columna de password	password (VARCHAR(255), NOT NULL)
Columna de nombre	name (VARCHAR(255), NOT NULL)
Columna de rol	role (VARCHAR(255), NOT NULL, DEFAULT 'user')
Columna de estado	status (VARCHAR(255), NOT NULL, DEFAULT 'active')
Columna de soft delete	deleted_at (TIMESTAMP, NULLABLE)
Timestamps	created_at , updated_at
Collation	utf8mb4_unicode_ci (case-insensitive para email)

Modelo User

Elemento	Valor exacto
Namespace	App\Models\User
Clase base	Illuminate\Foundation\Auth\User (alias Authenticatable)
Interfaces	Tymon\JWTAuth\Contracts\JWTSubject
Traits	Notifiable , SoftDeletes
\$fillable	['name', 'email', 'password', 'role', 'status']
\$hidden	['password', 'remember_token']
Cast password	'hashed'
Cast role	UserRole::class
Cast status	UserStatus::class
getJWTIdentifier()	Retorna \$this->getKey()
getJWTCustomClaims()	Retorna [] (se enriquecerá en Parte 3)

Enums

Enum	Namespace	Valores
UserRole	App\Enums\UserRole	Admin = 'admin' , User = 'user'
UserStatus	App\Enums\UserStatus	Active = 'active' , Inactive = 'inactive' , Banned = 'banned'

Convenciones de código

Regla	Detalle
<code>declare(strict_types=1)</code>	En cada archivo PHP del proyecto
Respuesta JSON éxito	<code>{ "data": { ... } }</code>
Respuesta JSON error	<code>{ "error": { "code": "string_id", "message": "string" } }</code>
Controllers	Single-action (<code>__invoke</code>) en <code>App\Http\Controllers\Auth\</code>
FormRequests	<code>LoginRequest</code> , <code>RegisterRequest</code> , <code>RefreshTokenRequest</code> en <code>App\Http\Requests\Auth\</code>
Service	<code>App\Services\AuthService</code>
Middleware JWT	<code>App\Http\Middleware\JwtAuthenticate</code>
Guard en <code>config/auth.php</code>	<code>api</code> → driver <code>jwt</code> , provider <code>users</code> → <code>App\Models\User</code>

Variables de entorno

Variable	Valor para desarrollo
<code>DB_CONNECTION</code>	<code>mysql</code>
<code>DB_HOST</code>	<code>127.0.0.1</code>
<code>DB_PORT</code>	<code>3306</code>
<code>DB_DATABASE</code>	<code>paradise_roasters</code>
<code>JWT_SECRET</code>	Se generará con <code>php artisan jwt:secret</code> en la Parte 3
<code>JWT_TTL</code>	<code>15</code> (minutos)
<code>JWT_REFRESH_TTL</code>	<code>10080</code> (minutos, 7 días)

Lo que viene en la Parte 3

En la [Parte 3: Instalación y Configuración de JWT](#):

1. Instalar el paquete `tymon/jwt-auth` .
2. Publicar el archivo de configuración `config/jwt.php` .
3. Configurar el guard `api` con driver `jwt` en `config/auth.php` .
4. Generar `JWT_SECRET` con `php artisan jwt:secret` .
5. Implementar correctamente los métodos `getJWTIdentifier()` y `getJWTCustomClaims()` del modelo `User` .
6. Crear el middleware `JwtAuthenticate` .
7. Verificar que el guard JWT funciona emitiendo un token desde Tinker.

Nota para el revisor de la Parte 3: el modelo `User` ya está completo (implementa `JWTSubject`, tiene `SoftDeletes`, casts de enum, y password hashed). Los enums `UserRole` y `UserStatus` están definidos y en uso. La migración `create_users_table` ya fue ejecutada. Solo falta instalar el paquete JWT, configurarlo, y refinar los métodos de `JWTSubject` según las necesidades que surjan al probar.

Parte 3: Configuración de JWT y Modelo de Tokens

1. Introducción a JWT

1.1 ¿Qué es JSON Web Token?

JWT (JSON Web Token) es un estándar abierto definido en la [RFC 7519](#) que define un formato compacto y autónomo para transmitir información entre partes como un objeto JSON. La información transmitida está **firmada digitalmente**, lo que permite verificar que no ha sido alterada.

Un JWT se compone de tres partes separadas por puntos (.):

HEADER.PAYLOAD.SIGNATURE

Cada parte es una cadena codificada en Base64URL:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJEsIm5hbWUiOiJKb2huIERvZSIsImldCI6MTY5MjAwMDAwMCwiZXhwIjoxNjk5MDAwOTAwfw.4fRNqLw2GvJk1Xmzyg RBdJYKtyH0NKz3BgNkHX7g

Header

Contiene metadatos sobre el token: el algoritmo de firma y el tipo de token.

Campo	Significado
alg	Algoritmo criptográfico usado para firmar el token. <code>HS256</code> = HMAC-SHA256, una firma simétrica (misma clave para firmar y verificar).
typ	Tipo de token. Siempre <code>"JWT"</code> (en mayúsculas).

Payload

Contiene los **claims** (declaraciones o afirmaciones) sobre una entidad (normalmente el usuario) y datos adicionales. Existen tres tipos de claims:

Tipo	Descripción	Ejemplos
Registered claims	Claims predefinidos por la RFC 7519. No son obligatorios pero sí recomendados.	<code>iss</code> (issuer), <code>sub</code> (subject), <code>exp</code> (expiration), <code>iat</code> (issued at), <code>nbf</code> (not before), <code>jti</code> (JWT ID)
Public claims	Claims definidos por la aplicación que usen namespaces para evitar colisiones.	<code>role</code> , <code>name</code> , <code>https://miapp.com/permissions</code>
Private claims	Claims acordados entre dos partes específicas.	<code>user_id</code> , <code>account_type</code>

Ejemplo de payload decodificado:

```
{
  "iss": "http://localhost:8000",
  "iat": 1692000000,
  "exp": 1692000900,
  "nbf": 1692000000,
  "jti": "a1b2c3d4e5f6",
  "sub": 1,
  "role": "admin",
  "name": "John Doe"
}
```

Signature

La firma se calcula aplicando el algoritmo especificado en el header sobre la concatenación del header y el payload codificados:

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret
)
```

La firma garantiza dos cosas: 1. **Integridad:** si alguien modifica el payload o el header, la firma no coincidirá y el token será rechazado. 2. **Autenticidad:** solo quien posee el `secret` puede generar firmas válidas.

ADVERTENCIA CRÍTICA: La firma **NO encripta** el contenido. El payload es Base64URL-encoded, lo que significa que **cualquiera puede decodificarlo y leerlo**. JWT garantiza que el contenido no ha sido alterado, NO que sea confidencial. Si necesitas confidencialidad, debes usar JWE (JSON Web Encryption) o transmitir el JWT siempre por HTTPS.

1.2 ¿Por qué JWT para APIs?

Ventaja	Explicación
Stateless (sin estado)	El servidor no necesita almacenar sesiones. Toda la información necesaria para autenticar y autorizar está dentro del token. Esto elimina la necesidad de consultar una base de datos de sesiones en cada petición.
Escalabilidad horizontal	Al no depender de estado en el servidor, cualquier instancia puede validar el token sin necesidad de compartir sesiones entre nodos. No necesitas Redis/memcached para sticky sessions.
Auto-contenido	El token transporta claims que evitan consultas adicionales a la base de datos para datos comunes (rol, nombre, ID del usuario).
Desacoplamiento	El servicio que emite el token no tiene por qué ser el mismo que lo valida (útil en arquitecturas de microservicios con RS256).

1.3 JWT vs OAuth2 vs SAML — ¿Cuándo usar cada uno?

Tecnología	Qué es	Cuándo usarlo
JWT	Formato de token (no un protocolo). Define la estructura del token.	Autenticación de APIs REST/GraphQL propia. Control total sobre la emisión y validación. Ideal para aplicaciones SPA + API backend propios.
OAuth2	Protocolo de autorización delegada. Define CÓMO obtener tokens, pero no el formato del token en sí (aunque suele usarse JWT como formato).	Cuando necesitas delegar autorización a terceros ("Iniciar sesión con Google"), o cuando tienes múltiples clientes (web, mobile, third-party) accediendo a tus APIs.
SAML	Protocolo de autenticación federada basado en XML.	Entornos enterprise con Single Sign-On (SSO) corporativo, Active Directory, Okta. Raramente se usa en aplicaciones nuevas orientadas a APIs.

Para este manual usamos **JWT puro con el paquete** `tymon/jwt-auth` porque nuestra arquitectura es una API RESTful propia, sin delegación a terceros ni federación corporativa. Si en el futuro necesitáramos "Login with Google", migraríamos a Laravel Sanctum + Socialite o a una solución OAuth2 como Laravel Passport.

1.4 JWT NO es un mecanismo de sesiones

Este es uno de los malentendidos más comunes entre desarrolladores. JWT es un **token de claims firmado**, no un mecanismo de sesiones. Diferencias fundamentales:

Sesiones tradicionales	JWT
El servidor almacena el estado de la sesión (server-side state)	El token contiene el estado (client-side state)
La sesión se identifica con un ID opaco (cookie de sesión)	El token ES la autorización, no un puntero a ella
Invalidar una sesión es inmediato (borrar del store)	Invalidar un JWT requiere blacklist o esperar a que expire
El servidor controla el ciclo de vida	El token tiene un ciclo de vida autónomo definido por sus claims temporales (<code>exp</code> , <code>nbf</code>)

La confusión nace de que muchas implementaciones usan JWT + refresh tokens para simular sesiones, pero conceptualmente son mecanismos distintos.

2. Instalación del Paquete JWT-Auth

2.1 Composer require

```
composer require tymon/jwt-auth
```

`tymon/jwt-auth` es el paquete canónico para integrar JWT con Laravel. Es mantenido activamente, compatible con Laravel 11, y construye sobre `lcobucci/jwt` para la manipulación de tokens a bajo nivel.

Razones para elegir `tymon/jwt-auth` sobre alternativas: - Integración nativa con el sistema de guards de Laravel (`auth()->guard('api')`) - Middleware incluido para proteger rutas - Blacklist de tokens incorporada - Soporte para refresh tokens con rotación - Manejo de claims personalizados vía la interfaz `JWTSubject`

2.2 Service Providers en Laravel 11

Laravel 11 eliminó `config/app.php` como archivo de configuración de providers. En su lugar, los service providers se registran en `bootstrap/providers.php`.

Para Laravel 11, añade al archivo `bootstrap/providers.php`:

```
<?php

return [
    App\Providers\AppServiceProvider::class,
    Tymon\JWTAuth\Providers\LaravelServiceProvider::class,
];
```

Para Laravel 10 o inferior, añade al array `providers` en `config/app.php`:

```
'providers' => [
    // ... otros providers ...
    Tymon\JWTAuth\Providers\LaravelServiceProvider::class,
],
```

El paquete soporta **auto-discovery** en Laravel, por lo que en la mayoría de los casos Composer lo registrará automáticamente. Sin embargo, ser explícito en el registro evita problemas si el auto-discovery falla o si despliegas en entornos con configuraciones atípicas.

2.3 Publicar configuración

```
php artisan vendor:publish --provider="Tymon\JWTAuth\Providers\LaravelServiceProvider"
```

Este comando copia el archivo de configuración del paquete a `config/jwt.php`. Sin este paso, el paquete usará sus valores por defecto internos, lo cual incluye una clave secreta predecible (punto 3.1).

Salida esperada:

```
Copied File [/vendor/tymon/jwt-auth/config/config.php] To [/config/jwt.php]
Publishing complete.
```

3. Configuración de `config/jwt.php` — Explicación Exhaustiva

El archivo `config/jwt.php` contiene TODAS las claves de configuración del paquete. A continuación se explican las más importantes para este proyecto, agrupadas por sección lógica.

3.1 `secret`

```
'secret' => env('JWT_SECRET'),
```

Qué es: La clave simétrica usada para firmar y verificar los tokens con el algoritmo HMAC (en nuestro caso HS256).

Cómo se genera: Con el comando Artisan `php artisan jwt:secret`, que escribe una cadena aleatoria de 64 caracteres en la variable `JWT_SECRET` del archivo `.env`.

Qué pasa si no se genera: El paquete usará una clave por defecto hardcodeada en su configuración interna. Esto es **extremadamente inseguro** en producción porque cualquiera que conozca la clave por defecto (es pública) podrá generar tokens válidos para tu aplicación.

Seguridad: La entropía de esta clave es la defensa principal contra ataques de fuerza bruta sobre la firma. 64 caracteres alfanuméricos ofrecen ~380 bits de entropía, más que suficiente con HS256.

ADVERTENCIA DE PRODUCCIÓN: Cambiar `JWT_SECRET` en un entorno productivo **invalida instantáneamente TODOS los tokens existentes**. Todos los usuarios serán forzados a re-autenticarse. Planifica cualquier rotación de esta clave con cuidado.

3.2 `keys` (solo para algoritmos asimétricos)

```
'keys' => [
    'public' => env('JWT_PUBLIC_KEY'),
    'private' => env('JWT_PRIVATE_KEY'),
    'passphrase' => env('JWT_PASSPHRASE'),
],
```

Esta sección es relevante **solo cuando se usan algoritmos asimétricos** como RS256, RS384, RS512, EdDSA, o ES256. Dado que este manual usa **HS256 (simétrico)**, estas claves **no se utilizan**. Se dejan documentadas para referencia:

Clave	Algoritmo	Propósito
<code>public</code>	RS256, ES256, EdDSA	Verificar la firma de tokens (la posee cualquier servicio que necesite validar).
<code>private</code>	RS256, ES256, EdDSA	Firmar tokens (SOLO la posee el servicio de autenticación).
<code>passphrase</code>	RS256, ES256	Frase de contraseña para desbloquear la clave privada si está protegida.

¿Cuándo usar RS256 en lugar de HS256?: Cuando tienes múltiples servicios que necesitan validar tokens pero solo uno debe poder emitirlos. El servicio de auth firma con la clave privada; los demás servicios validan con la pública (que no puede generar tokens). En arquitecturas de microservicios esto es una ventaja de seguridad. Para monolitos o APIs

simples, HS256 es suficiente y más simple.

3.3 `ttl` — Time To Live del Access Token

```
'ttl' => env('JWT_TTL', 60),
```

Qué es: Tiempo de vida del access token, en **minutos**.

Nuestro valor: `JWT_TTL=15` (definido en el `.env` desde la Parte 2). 15 minutos = 900 segundos.

¿Por qué 15 minutos?

TTL	Riesgo si es robado	Experiencia de usuario
5 min	Muy bajo	Pésima — refrescos constantes
15 min	Bajo	Buena — el refresh es transparente para el usuario
60 min	Medio	Excelente — pero 1 hora de ventana para un atacante
24 horas	Alto	No necesita refresh, pero es casi como no tener expiración

15 minutos es el sweet spot: la ventana de ataque es lo suficientemente pequeña para que el daño sea limitado, pero lo suficientemente grande para que el usuario no perciba interrupciones (el refresh token renueva silenciosamente).

El valor por defecto del paquete es 60 minutos. Nosotros lo sobrescribimos explícitamente con nuestra variable de entorno.

3.4 `refresh_ttl` — Ventana de Refresh

```
'refresh_ttl' => env('JWT_REFRESH_TTL', 20160),
```

Qué es: El período de tiempo (en **minutos**) durante el cual un access token expirado puede ser renovado usando un refresh token.

Nuestro valor: `JWT_REFRESH_TTL=10080` (definido en el `.env` desde la Parte 2). 10080 minutos = 7 días = 168 horas.

Cómo funciona la ventana de refresh: El `refresh_ttl` no es el TTL del refresh token en sí, sino la ventana máxima de inactividad que toleramos:

- Si un access token expira en el minuto 0, puede ser refrescado hasta el minuto +10080.
- Después del minuto +10080, el token es irrecuperable y el usuario debe volver a iniciar sesión.
- Mientras el usuario siga activo (refrescando antes de que pasen 7 días), su sesión se mantiene viva indefinidamente.

Analogía: El access token es una “llave de hotel” que caduca cada 15 minutos. El refresh token es la “reserva del hotel” que dura 7 días. Mientras tengas la reserva activa, puedes pedir llaves nuevas.

El valor por defecto del paquete es 20160 minutos (14 días). Nosotros elegimos 7 días como balance entre comodidad y seguridad: suficiente para un usuario que usa la app a diario, pero obliga a re-login tras una semana de inactividad.

3.5 `algo` — Algoritmo de Firma

```
'algo' => env('JWT_ALGO', 'HS256'),
```

Qué es: El algoritmo criptográfico usado para calcular la firma del token.

Valor para este manual: `HS256` (HMAC con SHA-256). Es el valor por defecto y el que usamos.

Algoritmo	Tipo	Clave	Velocidad	Caso de uso
HS256	Simétrico	1 clave secreta	Alta	Monolito, API simple
HS384	Simétrico	1 clave secreta	Media	Mayor seguridad, mismo caso
HS512	Simétrico	1 clave secreta	Baja	Máxima seguridad simétrica
RS256	Asimétrico	Par público/privado	Media	Microservicios
ES256	Asimétrico	Par público/privado	Alta	Microservicios modernos

HS256 es la elección correcta para este proyecto porque: 1. Una sola aplicación (no microservicios). 2. No necesitamos distribuir capacidad de firma vs verificación. 3. Es más simple de configurar y operar. 4. La clave de 64 caracteres ofrece entropía más que suficiente.

3.6 `required_claims` — Claims Obligatorios

```
'required_claims' => [  
  'iss',  
  'iat',  
  'exp',  
  'nbf',  
  'sub',  
  'jti',  
],
```

Estos son los claims que `tymon/jwt-auth` exige que estén presentes en CADA token. Si un token no contiene alguno de ellos, es rechazado automáticamente. Explicación de cada uno:

Claim	Nombre completo	Significado	Formato
<code>iss</code>	Issuer	Quién emitió el token. Normalmente la URL del servidor.	String (URL)
<code>iat</code>	Issued At	Timestamp de cuándo se emitió el token. Se usa para calcular la antigüedad.	NumericDate (timestamp Unix)
<code>exp</code>	Expiration	Timestamp de cuándo expira el token. Después de este momento, el token es inválido.	NumericDate (timestamp Unix)
<code>nbf</code>	Not Before	Timestamp antes del cual el token NO debe ser aceptado. Permite emitir tokens para uso futuro.	NumericDate (timestamp Unix)
<code>sub</code>	Subject	Identificador del sujeto del token (el usuario autenticado). Normalmente el ID del usuario.	String o número
<code>jti</code>	JWT ID	Identificador único del token. Fundamental para la blacklist (logout).	String

¿Por qué son obligatorios? - `exp` es la defensa básica contra tokens robados: sin expiración, un token robado es válido para siempre. - `jti` es necesario para invalidar tokens individuales (logout, revocación). - `sub` identifica a quién pertenece el token — sin él, no sabríamos qué usuario está autenticado. - `iat` + `exp` juntos definen la ventana de validez — si falta `iat`, no podemos calcular cuándo se emitió. - `iss` previene que tokens emitidos por otro servidor se usen en este (ataque de confusión de issuer). - `nbf` permite emitir tokens que no son válidos inmediatamente (raro, pero útil en ciertos flujos).

3.7 `persistent_claims`

```
'persistent_claims' => []
```

Claims que **sobreviven al proceso de refresh**. Cuando se refresca un token, todos los claims son regenerados excepto los listados aquí, que se copian del token original al nuevo.

Lo dejamos vacío porque nuestro refresh token regenera todos los claims desde cero. Si en el futuro quisieras que un claim como `device_id` o `session_id` persistiera entre refrescos, lo añadirías aquí.

3.8 `lock_subject`

```
'lock_subject' => true,
```

Qué hace: Cuando es `true`, el claim `sub` (subject = ID del usuario) no puede cambiar durante un refresh. Esto previene un vector de ataque donde un token de un usuario se transforma en token de otro usuario durante el

refresco.

Riesgo de desactivarlo: Un atacante con un token válido podría, durante un refresh, modificar el `sub` para suplantar a otro usuario. Con `lock_subject: true`, el `sub` se bloquea y cualquier intento de modificarlo invalida el refresh.

Recomendación: **dejarlo en `true` siempre**, a menos que tengas un caso de uso muy concreto que requiera migrar tokens entre usuarios (prácticamente inexistente).

3.9 `leeway`

```
'leeway' => env('JWT_LEEWAY', 0),
```

Qué es: Margen de tolerancia en **segundos** para compensar el desfase de reloj entre servidores (clock skew).

Por qué existe: Si el servidor A emite un token con `exp: 1692000900` y el reloj del servidor B va 30 segundos adelantado, B rechazaría el token 30 segundos antes de que realmente expire. `leeway` añade un margen para absorber esta diferencia.

Recomendación: En una arquitectura de un solo servidor, déjalo en 0. Si despliegas en múltiples nodos, configúralo entre 30 y 60 segundos.

3.10 `blacklist_enabled`

```
'blacklist_enabled' => env('JWT_BLACKLIST_ENABLED', true),
```

Qué hace: Habilita el mecanismo de invalidación de tokens. Cuando un token se “bloquea” (logout), su `jti` (JWT ID) se almacena en una blacklist. Cada vez que se valida un token, el paquete consulta si su `jti` está en la blacklist.

Cómo funciona técnicamente: El storage provider (configurado en `providers.storage`) almacena cada `jti` bloqueado hasta que el token habría expirado naturalmente (según `exp`). Esto significa que la blacklist se limpia automáticamente: no necesitas un proceso de limpieza manual.

Caso práctico: El usuario hace logout → el `jti` de su access token va a la blacklist → durante los próximos minutos, si alguien intenta usar ese access token, será rechazado → cuando `exp` se alcanza, el `jti` sale de la blacklist automáticamente.

¿Qué pasa si `blacklist_enabled = false`?: El logout no invalida el access token. El token seguirá siendo válido hasta que expire naturalmente. **No es aceptable para una aplicación con logout real.**

3.11 `blacklist_grace_period`

```
'blacklist_grace_period' => env('JWT_BLACKLIST_GRACE_PERIOD', 0),
```

Qué es: Período de gracia en **segundos** antes de que un token en blacklist sea efectivamente rechazado.

Por qué existe: En sistemas distribuidos, la blacklist puede tener consistencia eventual (un nodo ya la registró, otro todavía no la ve). El grace period da tiempo para que todos los nodos sincronicen antes de empezar a rechazar.

Nuestra configuración: 0 segundos. En una aplicación de un solo servidor, la consistencia es inmediata. Si el usuario hace logout, el token se invalida instantáneamente.

3.12 decrypt_cookies

```
'decrypt_cookies' => false,
```

Qué hace: Permite que el paquete intente extraer el token de cookies cifradas de Laravel.

Nuestra configuración: `false`. Este manual usa el header `Authorization: Bearer <token>` para transportar el JWT, no cookies. El uso de cookies con JWT introduce vulnerabilidades CSRF y va contra la filosofía stateless de JWT.

3.13 providers

```
'providers' => [  
    'jwt' => Tymon\JWTAuth\Providers\JWT\Lcobucci::class,  
    'auth' => Tymon\JWTAuth\Providers\Auth\Illuminate::class,  
    'storage' => Tymon\JWTAuth\Providers\Storage\Illuminate::class,  
],
```

Cada provider es una pieza intercambiable del ecosistema JWT. Explicación detallada:

Provider	Clase	Responsabilidad
jwt	Lcobucci	Codificar y decodificar tokens JWT. Maneja la creación del payload, la firma con el algoritmo configurado, la validación de claims (<code>exp</code> , <code>nbf</code> , <code>iat</code>) y la decodificación del token entrante. Es el corazón criptográfico del paquete.
auth	Illuminate	Integrar con el sistema de autenticación de Laravel. Realiza <code>Auth::login()</code> , <code>Auth::user()</code> , <code>Auth::logout()</code> usando el guard JWT. Traduce entre el mundo JWT (claims, tokens) y el mundo Laravel (User models, guards, providers).
storage	Illuminate	Almacenar tokens en la caché de Laravel para la blacklist. Usa el driver de caché configurado en <code>config/cache.php</code> (file, redis, memcached, database). Cada <code>jti</code> bloqueado se almacena con TTL igual al tiempo restante hasta <code>exp</code> .

¿Por qué son intercambiables?: El paquete está diseñado con una arquitectura de proveedores (strategy pattern). En teoría, podrías reemplazar `Lcobucci` por otra librería JWT, o `Illuminate` por un driver de auth personalizado, sin cambiar el resto del código. En la práctica, las implementaciones por defecto cubren el 99% de los casos de uso.

4. Generar JWT_SECRET

4.1 El comando jwt:secret

```
php artisan jwt:secret
```

Salida esperada:

```
jwt-auth secret [4fH7kL9mN2pQ5rS8tV1wX3yZ6aB9cD0eF2gH5iJ8kL1mN4oP7qR0sT3uV6wX9yZ] set successfully.
```

4.2 Qué hace exactamente

1. Genera una cadena aleatoria de 64 caracteres alfanuméricos con `Str::random(64)`.
2. Busca la clave `JWT_SECRET` en tu archivo `.env`.
3. Si existe, reemplaza su valor con la nueva clave.
4. Si no existe, añade `JWT_SECRET=<clave>` al final del archivo.
5. Limpia la caché de configuración para que el cambio surta efecto inmediato.

4.3 Verificación

```
cat .env | grep JWT_SECRET
```

Deberías ver algo como:

```
JWT_SECRET=4fH7kL9mN2pQ5rS8tV1wX3yZ6aB9cD0eF2gH5iJ8kL1mN4oP7qR0sT3uV6wX9yZ
```

4.4 ADVERTENCIA DE PRODUCCIÓN

Si rotas `JWT_SECRET` en producción: - **TODOS los tokens emitidos con la clave anterior son invalidados instantáneamente** porque la firma ya no coincide. - **Todos los usuarios conectados serán desconectados** (sus tokens serán rechazados como inválidos). - Esto incluye tanto access tokens como refresh tokens. - Planifica la rotación durante ventanas de bajo tráfico. - Considera un período de transición donde el servidor acepte ambas claves (vieja y nueva) durante unos minutos — esto requiere lógica personalizada que escapa al alcance de este manual pero es práctica estándar en sistemas de alta disponibilidad.

5. Implementar `JWTSubject` en el Modelo `User`

El modelo `App\Models\User` quedó pendiente de completar en la Parte 2. Ahora añadimos los dos métodos requeridos por la interfaz `JWTSubject` del paquete `tymon/jwt-auth`.

5.1 El código

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Notifications\Notifiable;
use Tymon\JWTAuth\Contracts\JWTSubject;
use App\Enums\UserRole;
use App\Enums\UserStatus;

class User extends Authenticatable implements JWTSubject
{
    use HasFactory, Notifiable, SoftDeletes;

    protected $fillable = [
        'name',
        'email',
        'password',
        'role',
        'status',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'role' => UserRole::class,
            'status' => UserStatus::class,
        ];
    }

    /**
     |-----
     | JWTSubject Implementation
     |-----
     */

    /**
     * Get the identifier that will be stored in the subject claim of the JWT.
     */
    public function getJWTIdentifier(): mixed
    {
        return $this->getKey();
    }

    /**
     * Return a key value array, containing any custom claims to be added to the JWT.
     */
    public function getJWTCustomClaims(): array
    {
        return [
            'role' => $this->role->value,
            'name' => $this->name,
        ];
    }
}

```

5.2 `getJWTIdentifier()` — El claim `sub`

Qué devuelve: El valor que irá en el claim `sub` (subject) del payload del JWT.

Qué devolvemos: `$this->getKey()` — la primary key del usuario. En Laravel, `getKey()` devuelve el valor de `$this->primaryKey`, que por defecto es `$this->id`.

Ejemplo: Si el usuario tiene `id = 1`, el payload del JWT tendrá `"sub": 1`.

¿Por qué el ID?: El claim `sub` es la forma canónica de identificar al sujeto de un JWT. Durante la validación, el paquete usa este valor para buscar al usuario en la base de datos con `User::find($sub)`. Es lo que conecta el mundo JWT (claims) con el mundo Laravel (Eloquent models).

Alternativas (AVANZADO, no para este manual): Podrías devolver UUID en lugar de ID autoincremental si tu aplicación usa UUIDs como primary key, o un hash del ID si no quieres exponer IDs secuenciales en los tokens. Para este manual, el ID secuencial es suficiente.

5.3 `getJWTCustomClaims()` — Claims personalizados

Qué devuelve: Un array asociativo de claims que se añaden al payload del token.

Qué devolvemos:

Claim	Valor	Propósito
<code>role</code>	<code>\$this->role->value</code> (<code>"admin"</code> o <code>"user"</code>)	Autorización rápida sin consultar la base de datos. Un middleware puede leer <code>auth()->payload()->get('role')</code> y decidir si permite el acceso a rutas administrativas.
<code>name</code>	<code>\$this->name</code> (<code>"John Doe"</code>)	Mostrar el nombre del usuario en la UI sin hacer una query adicional. Útil para “Bienvenido, John” en el frontend.

5.4 ADVERTENCIA DE SEGURIDAD SOBRE CLAIMS PERSONALIZADOS

EL PAYLOAD DEL JWT ES BASE64-ENCODED, NO ENCRYPTADO. CUALQUIERA PUEDE DECODIFICARLO Y LEERLO.

Esto significa: - **Cualquiera** puede decodificar el token y ver `role`, `name`, y cualquier otro claim que incluyas. - Lo que **no pueden hacer** es modificar esos valores porque la firma dejaría de ser válida. - Pero **pueden leerlos**. Siempre.

Qué NUNCA debes incluir en claims personalizados:

Dato	Razón
<code>password</code> o <code>password_hash</code>	Obvio, pero vale la pena repetirlo. Es información de autenticación, no de autorización.
<code>email</code> (si es considerado PII en tu jurisdicción)	Bajo GDPR y similares, el email es dato personal. Si puedes evitarlo en el payload, mejor. Si lo necesitas para la UI, pondera el trade-off. En este manual NO lo incluimos.
Token de verificación de email	Si alguien lo lee, puede verificar emails ajenos.
<code>remember_token</code>	Nunca sale de la base de datos.
Cualquier secret o API key	El payload no es el lugar para secretos.

Qué Sí es seguro incluir: - `role`, `permissions`: strings simples para autorización. - `name`, `username`: identificadores visibles. - Timestamps de sesión. - Cualquier dato que ya sea público o semi-público en tu aplicación.

Regla de oro: Si no pondrías ese dato en un `<div>` público de tu HTML, no lo pongas en un claim del JWT.

6. Configurar el Guard en `config/auth.php`

El guard es el mecanismo que Laravel usa para autenticar usuarios. Configurar el guard JWT correctamente es lo que permite usar `auth('api')->attempt($credentials)` y `auth('api')->user()` en los controllers.

6.1 El archivo `config/auth.php` modificado

```

<?php

return [

    /*
    |-----
    | Authentication Defaults
    |-----
    */

    'defaults' => [
        'guard' => 'api',
        'passwords' => 'users',
    ],

    /*
    |-----
    | Authentication Guards
    |-----
    */

    'guards' => [
        'web' => [
            'driver' => 'session',
            'provider' => 'users',
        ],

        'api' => [
            'driver' => 'jwt',
            'provider' => 'users',
        ],
    ],

    /*
    |-----
    | User Providers
    |-----
    */

    'providers' => [
        'users' => [
            'driver' => 'eloquent',
            'model' => App\Models\User::class,
        ],
    ],

    /*
    |-----
    | Resetting Passwords
    |-----
    */

    'passwords' => [
        'users' => [
            'provider' => 'users',
            'table' => 'password_reset_tokens',
            'expire' => 60,
            'throttle' => 60,
        ],
    ],

    /*
    |-----
    | Password Confirmation Timeout
    |-----
    */

    'password_timeout' => 10800,
];

```

6.2 Explicación por sección

`defaults.guard` → `'api'`

Define qué guard se usa cuando llamas a `auth()` sin especificar guard:

```
// Con defaults.guard = 'api', estas dos líneas son equivalentes:  
$user = auth()->user();  
$user = auth('api')->user();
```

¿Por qué `api` como default?: Nuestra aplicación es una API RESTful. La gran mayoría de las peticiones se autenticarán vía JWT en el header `Authorization`. Si la aplicación tuviera también una parte web con sesiones, el default sería `web` y en las rutas API se especificaría `auth('api')`.

`guards.api` → `driver: 'jwt'`

El driver `jwt` es proporcionado por `tymon/jwt-auth`. Cuando Laravel encuentra `driver: 'jwt'`, delega en el paquete para resolver al usuario autenticado a partir del token JWT en la petición.

Diferencia con otros drivers de API:

Driver	Mecanismo	Estado	Cuándo usarlo
<code>token</code> (Laravel nativo)	Token simple almacenado en la tabla <code>api_tokens</code> de la DB. Cada petición requiere un query a la DB.	Stateful	APIs internas simples, no necesita claims.
<code>sanctum</code> (Laravel Sanctum)	Token almacenado en DB (hash) o cookie de sesión para SPAs. Soporta SPA + mobile.	Stateful	SPAs mismo dominio + API tokens para third-party.
<code>jwt</code> (tymon/jwt-auth)	Token firmado con claims. Validación sin DB. Blacklist opcional vía caché.	Stateless	APIs RESTful puras, mobile apps, microservicios.

`guards.web` → `driver: 'session'`

Se mantiene por compatibilidad. Si la aplicación tuviera rutas web tradicionales (formularios, vistas Blade), usarían este guard. En nuestra API pura, no se usa pero no conviene eliminarlo por si en el futuro se añade un panel admin con Filament o Nova.

`providers.users`

Define de DÓNDE obtiene Laravel los usuarios.

Campo	Valor	Significado
<code>driver</code>	<code>eloquent</code>	Usa Eloquent ORM para buscar usuarios. La alternativa es <code>database</code> (query builder directo).
<code>model</code>	<code>App\Models\User::class</code>	El modelo Eloquent que representa a los usuarios. Debe implementar <code>JWTSubject</code> .

El provider es compartido entre el guard `web` y el guard `api`. Ambos autentican usuarios del mismo modelo `User`, pero usando mecanismos distintos (session vs JWT). Esto es correcto y deseable: el origen de datos es el mismo.

7. Anatomía de un Token JWT en esta Aplicación

7.1 Token completo (ejemplo)

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOi8vbG9jYWxob3N0IiwiaWF0IjE2OTIwMDAwMDAsImV4cCI6MTY5MjAwMDkwMCwibmJmIjoxNjkyMDAwMDAwLCJqdGkiOiJhMwIyYzNkNGU1ZjYiLCJzdWIiOiJEsInBydiI6IjIzYmQwNzFhNjg4MmNiMTJjM2QwYmE4YmU4YmQyYmI3Iiwicm9sZSI6ImFkbWludIiwibmFtZSI6IkpvaG4gRG9lIn0.SIGNATURE_AQUI
```

7.2 Header decodificado

```
{  
  "typ": "JWT",  
  "alg": "HS256"  
}
```

7.3 Payload decodificado

```
{  
  "iss": "http://localhost:8000",  
  "iat": 1692000000,  
  "exp": 1692000900,  
  "nbf": 1692000000,  
  "jti": "a1b2c3d4e5f6",  
  "sub": 1,  
  "prv": "23bd071a6882cb12c3d0ba8be8bd2bb7",  
  "role": "admin",  
  "name": "John Doe"  
}
```

7.4 Explicación de cada claim en este payload

Claim	Valor ejemplo	Origen	Explicación
iss	http://localhost:8000	Automático (URL de la app)	El issuer. <code>tymon/jwt-auth</code> lo configura desde <code>config/app.url</code> . En producción será tu dominio real.
iat	1692000000	Automático (timestamp actual)	Issued At. Momento exacto de emisión. Formato NumericDate (segundos desde epoch Unix).
exp	1692000900	<code>iat + (JWT_TTL * 60)</code>	Expiration. <code>1692000900 - 1692000000 = 900 segundos = 15 minutos</code> . Coincide con <code>JWT_TTL=15</code> .
nbf	1692000000	Automático (= <code>iat</code>)	Not Before. El token no es válido antes de este momento. Igual a <code>iat</code> porque el token es válido inmediatamente.
jti	a1b2c3d4e5f6	Automático (string aleatorio)	JWT ID. Identificador único de este token específico. Se genera con <code>Str::random()</code> . Esencial para la blacklist.
sub	1	<code>User::getJWTIdentifier()</code>	Subject. El ID del usuario autenticado. El paquete usa <code>User::find(1)</code> para resolverlo.
prv	23bd071a...	Automático (hash del provider)	Provider hash. Claim interno de <code>tymon/jwt-auth</code> que identifica este token como parte del flujo de refresh. Es un hash del modelo User para verificar que el token pertenece al provider correcto.
role	admin	<code>User::getJWTCustomClaims()</code>	Claim personalizado. Rol del usuario desde <code>UserRole</code> enum. Permite autorización rápida sin DB.
name	John Doe	<code>User::getJWTCustomClaims()</code>	Claim personalizado. Nombre del usuario para mostrarlo en la UI.

7.5 Relación con `required_claims`

El paquete valida que los 6 claims configurados en `required_claims` estén presentes. Los claims `role` y `name` son opcionales desde el punto de vista del paquete (no están en `required_claims`), pero SIEMPRE estarán presentes porque `getJWTCustomClaims()` los devuelve.

8. Estrategia de Tokens: Access + Refresh

Esta no es una decisión de implementación menor — es la **arquitectura de seguridad de sesiones** de la aplicación.

8.1 Access Token

Propiedad	Valor
Vida útil	15 minutos (<code>JWT_TTL</code>)
Transporte	Header HTTP: <code>Authorization: Bearer <token></code>
Contenido	Claims de autorización (<code>sub</code> , <code>role</code> , <code>name</code>) + claims estándar
Validación	Firma + claims temporales + blacklist
Uso	Cada petición a endpoints protegidos
Si es robado	El atacante tiene acceso durante máximo 15 minutos

¿Por qué 15 minutos? - Si un atacante intercepta un access token (MITM, XSS, log泄露), el daño está acotado a 15 minutos. - 15 minutos es suficiente para que un usuario real complete cualquier operación sin notar el refresh. - Es lo suficientemente corto para que la rotación de refresh tokens detecte anomalías (ver sección 8.3).

8.2 Refresh Token

Propiedad	Valor
Vida útil	7 días (<code>JWT_REFRESH_TTL</code>)
Transporte	Cuerpo de la petición POST al endpoint <code>/auth/refresh</code>
Contenido	Mismos claims que el access token + claim <code>prv</code> que lo marca como refresh token
Uso	Solo en el endpoint de refresh, una vez, para obtener nuevos tokens
Si es robado	Detectable mediante rotación (ver 8.3)

¿Por qué separar access y refresh?

Separar el token que se usa en CADA petición (alto riesgo de exposición en logs, proxies, headers) del token que se usa UNA SOLA VEZ en un endpoint específico (bajo riesgo) es una práctica estándar de seguridad. Es el mismo principio que separar una llave de uso diario de una llave maestra de emergencia.

8.3 Rotación de Refresh Tokens (Refresh Token Rotation)

Cada vez que el cliente usa un refresh token para obtener nuevos tokens, el servidor:

1. Valida el refresh token entrante.
2. **Emite un NUEVO access token y un NUEVO refresh token.**
3. **Revoca (invalida) el refresh token anterior.**

¿Por qué implementar rotación?

Sin rotación, un refresh token robado es indetectable: el atacante y el usuario legítimo pueden usarlo simultáneamente sin que nadie se dé cuenta.

Con rotación, si un atacante roba un refresh token y lo usa: 1. El servidor emite nuevos tokens y revoca el refresh token usado. 2. Cuando el usuario legítimo intente usar SU refresh token (que es el mismo), el servidor lo rechazará porque ya fue revocado. 3. El servidor **detecta el reuse** y puede forzar logout de todas las sesiones del usuario (invalidando TODOS los refresh tokens activos de ese usuario).

Implementación en este manual: La tabla `refresh_tokens` (sección 9) almacena cada refresh token emitido. Durante el refresh, el token entrante se busca en la tabla: - Si existe y no está revocado → se revoca, se emite uno nuevo. - Si no existe o ya está revocado → **posible ataque de reuse** → respuesta 401 + invalidación de todos los tokens del usuario.

8.4 Flujo Completo de Refresh



9. Almacenamiento de Refresh Tokens — Diseño de la Tabla `refresh_tokens`

La migración formal se creará en la Parte 4, pero DISEÑAMOS aquí la tabla para entender la arquitectura completa antes de implementar.

9.1 Esquema SQL

```

CREATE TABLE refresh_tokens (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT UNSIGNED NOT NULL,
  token VARCHAR(500) NOT NULL,
  expires_at TIMESTAMP NOT NULL,
  revoked_at TIMESTAMP NULL DEFAULT NULL,
  created_at TIMESTAMP NULL DEFAULT NULL,
  updated_at TIMESTAMP NULL DEFAULT NULL,

  CONSTRAINT fk_refresh_tokens_user_id
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE,

  INDEX idx_refresh_tokens_user_id (user_id),
  INDEX idx_refresh_tokens_token (token),
  INDEX idx_refresh_tokens_expires_at (expires_at)
);

```

9.2 Explicación columna por columna

Columna	Tipo	Justificación
id	BIGINT UNSIGNED AUTO_INCREMENT	Clave primaria estándar. <code>BIGINT</code> porque una API con mucho tráfico puede generar millones de tokens.
user_id	BIGINT UNSIGNED	Referencia al usuario dueño del token. Necesario para “cerrar sesión en todos los dispositivos” (revocar todos los tokens de un usuario).
token	VARCHAR(500)	El refresh token completo (string JWT). 500 caracteres dan margen suficiente para un JWT con múltiples claims. Un JWT típico ronda los 200-400 caracteres.
expires_at	TIMESTAMP	Cuándo expira este refresh token (7 días desde emisión). Permite: a) rechazar tokens expirados sin decodificarlos, b) limpiar tokens expirados con un scheduled command.
revoked_at	TIMESTAMP NULL	Soft-revoke. <code>NULL</code> = activo. Una timestamp $\neq$ <code>NULL</code> = revocado en ese momento. Permite auditoría forense (“¿cuándo se revocó este token? ¿fue el usuario o fue por detección de reuse?”).
created_at	TIMESTAMP	Laravel lo maneja automáticamente con <code>\$table->timestamps()</code> .
updated_at	TIMESTAMP	Ídem. Se actualiza cuando se revoca el token.

9.3 Índices

Índice	Columnas	Consulta que acelera
PRIMARY	id	Identificación única.
idx_refresh_tokens_user_id	user_id	<code>SELECT * FROM refresh_tokens WHERE user_id = ?</code> — listar sesiones activas de un usuario.
idx_refresh_tokens_token	token	<code>SELECT * FROM refresh_tokens WHERE token = ?</code> — validar un refresh token entrante. Esta es la consulta MÁS frecuente (cada refresh).
idx_refresh_tokens_expires_at	expires_at	<code>DELETE FROM refresh_tokens WHERE expires_at < NOW()</code> — limpieza programada de tokens expirados.
(implícito)	user_id en FK	El constraint de FK crea automáticamente un índice en user_id en la mayoría de motores.

9.4 FK con ON DELETE CASCADE

```
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```

Si un usuario es eliminado de la tabla `users`, todos sus refresh tokens se eliminan automáticamente. Esto es fundamental: sin `ON DELETE CASCADE`, los refresh tokens de usuarios eliminados quedarían huérfanos y ocuparían espacio para siempre.

9.5 Alternativa evaluada: Redis vs Base de Datos

Criterio	Base de Datos (MySQL)	Redis
Persistencia	✓ Duradera (disco). Los tokens sobreviven a reinicios del servidor.	⚠ Volátil (por defecto). Si Redis se reinicia sin snapshot, todos los refresh tokens se pierden → todos los usuarios son desconectados.
Velocidad	⚠ Consulta SQL (~1ms con índice). Aceptable para la frecuencia de refresh (1 vez cada 15 min por usuario).	✓ Sub-milisegundo. Excelente para alta frecuencia.
Simplicidad operativa	✓ Sin dependencias adicionales. MySQL ya está en el stack.	✗ Necesitas Redis instalado, configurado, monitoreado.
TTL nativo	✗ Necesitas un scheduled command para limpieza.	✓ SETEX con TTL — expira automáticamente.
Auditoría	✓ Puedes consultar SQL histórico: “¿cuántos tokens activos tiene el usuario 42?”	✗ Difícil auditar datos que ya expiraron.
Rotación	✓ Transacción SQL: revocar viejo + insertar nuevo = atómico.	⚠ Sin transacciones. Necesitarías Lua scripting para atomicidad.

Decisión para este manual: Base de Datos (MySQL).

Razones: 1. Este proyecto no justifica Redis. La frecuencia de refresh (cada 15 minutos por usuario) no es un cuello de botella para MySQL. 2. No añadimos dependencia operativa. Un desarrollador puede clonar el repo, `php artisan migrate` y tener todo funcionando. 3. La auditoría de sesiones activas es trivial con SQL. 4. Si en el futuro el tráfico escala, migrar refresh tokens a Redis es un refactor acotado (cambiar el storage en el AuthService, la tabla sigue existiendo para auditoría).

9.6 Estrategia de limpieza

Los refresh tokens expirados o revocados se acumularán en la tabla. Para evitar que crezca indefinidamente, se implementará (en la Parte 5 o 6) un comando de limpieza:

```
// Pseudocódigo del futuro comando
DELETE FROM refresh_tokens
WHERE expires_at < NOW()
OR (revoked_at IS NOT NULL AND revoked_at < NOW() - INTERVAL 30 DAY);
```

Este comando se programará en `app/Console/Kernel.php` para ejecutarse diariamente:

```
$schedule->command('tokens:cleanup')->daily();
```

La ventana de 30 días para tokens revocados permite auditoría forense (¿se revocó este token por el usuario o por detección de ataque de reuse?), pero evita acumulación perpetua.

10. Verificación de la Configuración

10.1 Limpiar caché de configuración

Después de modificar `config/auth.php` y `config/jwt.php`, la caché de configuración de Laravel puede contener valores antiguos:

```
php artisan config:clear
```

Esto elimina `bootstrap/cache/config.php` y fuerza a Laravel a releer todos los archivos de configuración. En producción, después de verificar que todo funciona, vuelve a cachear:

```
php artisan config:cache
```

Nota: `config:cache` no se ejecuta en desarrollo local porque cada cambio requeriría re-cachear. En producción, cachear la configuración mejora el rendimiento (evita leer docenas de archivos PHP en cada petición).

10.2 Probar generación de token con Tinker

```
php artisan tinker
```

Dentro de Tinker:

```
$user = User::first();  
$token = auth('api')->login($user);  
echo $token;  
// Debería mostrar un string JWT como:  
//eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRw...
```

Si esto falla, diagnóstico por orden de probabilidad:

Error	Causa probable	Solución
Class 'Tymon\JWTAuth\Providers\LaravelServiceProvider' not found	El paquete no está instalado o el provider no está registrado.	Verifica <code>composer.json</code> , ejecuta <code>composer dump-autoload</code> , confirma <code>bootstrap/providers.php</code> .
JWT_SECRET is not set	No se ejecutó <code>php artisan jwt:secret</code> .	<code>php artisan jwt:secret</code> .
Method login does not exist	El guard <code>api</code> no está configurado con driver <code>jwt</code> .	Revisa <code>config/auth.php</code> → <code>guards.api.driver = 'jwt'</code> .
Undefined method getJWTIdentifier()	El modelo User no implementa <code>JWTSubject</code> .	Verifica <code>implements JWTSubject</code> y los dos métodos.
Target class [config] does not exist en Tinker	La configuración está cacheada con valores viejos.	<code>php artisan config:clear</code> .

10.3 Decodificar el token en jwt.io

1. Copia el token generado en Tinker.
2. Ve a jwt.io.
3. Pégalo en el campo “Encoded”.
4. Verifica que el payload muestre tus claims personalizados (`role` , `name`).
5. En el campo “Verify Signature”, pega tu `JWT_SECRET` para confirmar que la firma es válida.

ADVERTENCIA DE SEGURIDAD: `jwt.io` es una herramienta de debugging. NUNCA pegues tokens de producción con datos reales de usuarios en herramientas de terceros. Para debugging en producción, usa `php artisan tinker` con `JWTAuth::parseToken()->getPayload()`.

11. Resumen y Puente a Parte 4

11.1 Lo instalado y configurado en esta parte

Elemento	Estado
Paquete <code>tymon/jwt-auth</code>	Instalado vía Composer
Service Provider	Registrado en <code>bootstrap/providers.php</code>
<code>config/jwt.php</code>	Publicado y configurado con HS256, TTL 15min, refresh TTL 7d
<code>JWT_SECRET</code>	Generado con <code>php artisan jwt:secret</code> , almacenado en <code>.env</code>
Guard <code>api</code> en <code>config/auth.php</code>	Configurado con driver <code>jwt</code> y provider <code>users</code>
Modelo <code>User</code>	Implementa <code>JWTSubject</code> con <code>getJWTIdentifier()</code> y <code>getJWTCustomClaims()</code>
Tabla <code>refresh_tokens</code>	Diseñada (migración en Parte 4)

11.2 Estado actual del sistema

En este punto, la aplicación PUEDE generar tokens JWT para cualquier usuario (probado con Tinker), pero NO puede: - Registrar usuarios nuevos (→ Parte 4) - Autenticar usuarios con email/password (→ Parte 5) - Proteger rutas con middleware JWT (→ Parte 5) - Refrescar tokens (→ Parte 5)

11.3 Lo que viene en la Parte 4

La Parte 4 implementará el **registro de usuarios**: - `RegisterRequest` — validación de datos de registro - `RegisterController` — single-action controller con `__invoke` - `AuthService::register()` — lógica de negocio (crear usuario, emitir tokens) - Migración de la tabla `refresh_tokens` - Primeros tests de integración

Decisiones Vinculantes para Partes 4-6

- Estructura del access token:** Payload con claims `iss`, `iat`, `exp`, `nbf`, `jti`, `sub`, `prv` (automáticos) + `role` y `name` (custom). Algoritmo HS256. TTL de 15 minutos.
- Estructura del refresh token:** Mismo payload que el access token, mismo algoritmo, TTL de 7 días (10080 minutos). El claim `prv` lo identifica como refresh token.
- Formato del refresh token en la DB:** Se almacena en la tabla `refresh_tokens` como string JWT completo (`VARCHAR(500)`). Columna `revoked_at` para soft-revoke. Columna `expires_at` para limpieza programada.
- Tabla `refresh_tokens`:** La migración se crea en la Parte 4. Debe incluir FK a `users(id)` con `ON DELETE CASCADE` + índices en `user_id`, `token`, `expires_at`. La columna `token` usa `VARCHAR(500)`.
- Rotación de refresh tokens:** Cada refresh emite un NUEVO refresh token y revoca el anterior. Reuse de refresh token revocado → detección de ataque → revocar TODOS los tokens del usuario → 401.
- Claims personalizados:** `role` (desde `UserRole` enum, valor string) y `name` (desde `$user->name`). NO se incluye `email` en los claims por ser PII bajo GDPR. Si un endpoint necesita el email, consulta la DB.
- Limpieza de tokens:** Comando `tokens:cleanup` programado diariamente. Elimina tokens con `expires_at < NOW()` o `revoked_at < NOW() - 30 days`.
- Guard por defecto:** `api` (driver `jwt`). Todas las rutas protegidas usarán `auth('api')` implícitamente vía `auth()->user()` sin especificar el guard.

Parte 4: Registro de Usuarios

1. Introducción

1.1 Dónde estamos

En las tres partes anteriores construimos la base completa del sistema de autenticación:

Parte	Lo construido	Estado
Parte 1	Proyecto Laravel 11, arquitectura en capas, convenciones de código, estructura de directorios	✓
Parte 2	Variables de entorno, base de datos MySQL, migración <code>users</code> , modelo <code>User</code> , enums <code>UserRole</code> y <code>UserStatus</code>	✓
Parte 3	Paquete <code>typhon/jwt-auth</code> instalado y configurado, guard <code>api</code> con driver <code>jwt</code> , claims del JWT, diseño de tabla <code>refresh_tokens</code>	✓

En este momento, la aplicación **puede** generar tokens JWT para cualquier usuario desde Tinker, pero **no puede** registrar usuarios a través de un endpoint HTTP. Eso es exactamente lo que construiremos aquí.

1.2 Objetivo de esta parte

Implementar el endpoint `POST /api/register` completo, de extremo a extremo:

```
Cliente → Ruta → RegisterRequest → RegisterController → AuthService::register() → DB
```

Al terminar esta parte, un cliente HTTP (curl, Postman, frontend SPA) podrá enviar `name`, `email`, `password` y `password_confirmation` y recibir como respuesta el usuario creado junto con sus access token y refresh token.

1.3 Qué construiremos

1. **Migración** `create_refresh_tokens_table` — diseñada en la Parte 3, ahora la materializamos.
2. **Modelo** `RefreshToken` — encapsula la lógica de dominio de los refresh tokens.
3. `RegisterRequest` — valida los datos de entrada del registro.
4. `AuthService::register()` — lógica de negocio: crear usuario, emitir tokens, guardar refresh token.
5. `RegisterController` — punto de entrada HTTP, single-action.
6. **Ruta en** `routes/api.php` — conecta el endpoint con el controller.
7. **Tests de integración** — verificar que todo funciona y manejar casos de error.

2. Migración de `refresh_tokens`

2.1 Crear la migración

```
php artisan make:migration create_refresh_tokens_table
```

Laravel genera un archivo en `database/migrations/` con el timestamp actual. Sobrescribe su contenido con el siguiente código:

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('refresh_tokens', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();
            $table->string('token', 500);
            $table->timestamp('expires_at');
            $table->timestamp('revoked_at')->nullable();
            $table->timestamps();

            $table->index('token');
            $table->index('expires_at');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('refresh_tokens');
    }
};
```

2.2 Análisis columna por columna

```
$table->id()
```

Genera `BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY`. Como se justificó en la Parte 2, `BIGINT` previene el desbordamiento de `INT` en tablas con millones de registros — y una tabla de tokens puede crecer rápidamente si se emite un refresh token por login y por cada refresh subsecuente.

```
$table->foreignId('user_id')->constrained()->cascadeOnDelete()
```

Esta elegante sintaxis de Laravel equivale a:

```
`user_id` BIGINT UNSIGNED NOT NULL,
CONSTRAINT fk_refresh_tokens_user_id
    FOREIGN KEY (user_id) REFERENCES users(id)
ON DELETE CASCADE
```

Desglose de lo que ocurre:

Método	Qué hace
<code>foreignId('user_id')</code>	Crea una columna <code>BIGINT UNSIGNED</code> llamada <code>user_id</code> . Laravel infiere que es una FK por el sufijo <code>_id</code> .
<code>constrained()</code>	Vincula la FK a la tabla <code>users</code> en la columna <code>id</code> . Laravel deduce <code>users</code> del nombre de la columna (<code>user_id</code> → tabla <code>users</code> , columna <code>id</code>). Si la tabla tuviera otro nombre, usarías <code>constrained('other_table')</code> .
<code>cascadeOnDelete()</code>	Añade <code>ON DELETE CASCADE</code> . Cuando un usuario es eliminado, MySQL borra automáticamente todos sus refresh tokens. Sin esto, quedarían huérfanos ocupando espacio.

Además, `foreignId()` **crea implícitamente un índice** sobre `user_id` en la mayoría de motores de base de datos. Esto acelera `SELECT * FROM refresh_tokens WHERE user_id = ?`, que es la consulta para listar sesiones activas de un usuario y para revocar todos sus tokens ante un ataque de reuse.

`$table->string('token', 500)`

Almacena el refresh token JWT completo.

¿Por qué `VARCHAR(500)` y no `TEXT`?

Tipo	Tamaño máximo	Índice posible	Caso de uso
<code>VARCHAR(500)</code>	500 caracteres	✓ Se puede indexar completo	Tokens JWT (~200-400 chars)
<code>VARCHAR(255)</code>	255 caracteres	✓ Se puede indexar	Podría quedarse corto con claims personalizados
<code>TEXT</code>	65,535 bytes	✗ Solo se puede indexar con prefijo	Nunca necesario para un JWT

500 caracteres es el sweet spot: suficiente para un JWT con payload completo (sub, role, name, claims estándar) y compatible con índices B-tree completos. La consulta más frecuente sobre esta columna — `WHERE token = ?` — se beneficia directamente de un índice sobre la columna completa, no sobre un prefijo.

`$table->timestamp('expires_at')`

Fecha y hora de expiración del refresh token. Se establece en el momento de creación como `now() + JWT_REFRESH_TTL minutos`.

Dos usos críticos:

- Validación sin decodificar el JWT:** `WHERE token = ? AND expires_at > NOW() AND revoked_at IS NULL` — una sola consulta SQL valida el token sin necesidad de que el paquete JWT lo decodifique y verifique claims temporales. Si `expires_at` ya pasó, la fila ni siquiera se devuelve.
- Limpieza programada:** `DELETE FROM refresh_tokens WHERE expires_at < NOW()` — el comando `tokens:cleanup` programado diariamente elimina tokens expirados sin lógica compleja.

`$table->timestamp('revoked_at')->nullable()`

Soft-revoke. Semántica:

Valor	Significado
NULL	El token está activo y puede usarse para refresh
Una timestamp	El token fue revocado en ese momento exacto

¿Por qué soft-revoke (timestamp) en vez de hard-delete?

1. **Auditoría forense:** “¿Se revocó este token porque el usuario cerró sesión, o porque el sistema detectó un ataque de reuse?” La timestamp responde esta pregunta.
2. **Debugging:** Si un usuario reporta “me echaron de la sesión”, puedes consultar `SELECT * FROM refresh_tokens WHERE user_id = ? ORDER BY revoked_at DESC` y ver exactamente qué pasó.
3. **Consistencia con soft deletes:** Igual que `users.deleted_at`, mantenemos el registro histórico sin perder información.

¿Por qué no usar `$table->softDeletes()` de Laravel?

El trait `SoftDeletes` de Eloquent usa `deleted_at` y añade un scope global `WHERE deleted_at IS NULL`. Pero aquí necesitamos una semántica distinta:

- `expires_at` marca el fin natural del ciclo de vida del token.
- `revoked_at` marca una terminación forzosa (logout, ataque detectado).

Son dos conceptos diferentes que merecen dos columnas diferentes. Usar `deleted_at` parasitaría la semántica de soft-delete de Laravel y confundiría a futuros desarrolladores.

`$table->timestamps()`

Genera `created_at` y `updated_at`. `created_at` registra cuándo se emitió el refresh token (debería coincidir aproximadamente con `expires_at - 7 días`). `updated_at` se actualiza cuando se revoca el token.

2.3 Índices — la clave del rendimiento

Índice	Columnas	Origen	Consulta que acelera
PRIMARY	id	<code>\$table->id()</code>	Identificación única de fila
(implícito)	user_id	<code>foreignId()</code> crea índice automáticamente	<code>WHERE user_id = ?</code> — listar sesiones activas de un usuario
<code>refresh_tokens_token_index</code>	token	<code>\$table->index('token')</code>	<code>WHERE token = ?</code> — validar refresh token entrante
<code>refresh_tokens_expires_at_index</code>	expires_at	<code>\$table->index('expires_at')</code>	<code>WHERE expires_at < NOW()</code> — limpieza programada

¿Por qué un índice en `token` ?

Cada petición de refresh incluye un refresh token en el cuerpo. El servidor debe buscar ese token en la base de datos para verificar que existe, no ha expirado y no ha sido revocado:

```
SELECT * FROM refresh_tokens WHERE token = ? LIMIT 1;
```

Sin índice, MySQL haría un full table scan sobre TODOS los refresh tokens emitidos en la historia de la aplicación. Con miles o millones de tokens, esto es inaceptable. Con un índice B-tree sobre `token`, la búsqueda es $O(\log n)$ —

microsegundos incluso con millones de filas.

¿Por qué un índice en `expires_at` ?

El comando de limpieza `DELETE FROM refresh_tokens WHERE expires_at < NOW()` necesita encontrar eficientemente los tokens expirados. Sin índice, es otro full table scan diario. Con índice, solo toca las filas que realmente expiraron.

Nota sobre `VARCHAR(500)` indexado: MySQL limita los índices a 767 bytes por defecto en InnoDB con `utf8mb4` (4 bytes por carácter). $500 \times 4 = 2000$ bytes, que excede 767. Sin embargo, InnoDB automáticamente indexa los primeros 191 caracteres con `utf8mb4` — y 191 caracteres de un token JWT son más que suficientes para distinguir tokens únicos (la probabilidad de colisión en los primeros 191 caracteres de dos JWTs con `jti` aleatorio es infinitesimal).

2.4 Ejecutar la migración

```
php artisan migrate
```

Salida esperada:

```
Migrating: 2025_01_15_000002_create_refresh_tokens_table
Migrated:  2025_01_15_000002_create_refresh_tokens_table (45.23ms)
```

Verifica:

```
php artisan migrate:status
```

```
+-----+-----+-----+-----+
| Ran? | Migration                                          | Batch |
+-----+-----+-----+-----+
| Yes  | 2014_10_12_000000_create_users_table            | 1      |
| Yes  | 2025_01_15_000002_create_refresh_tokens_table    | 2      |
+-----+-----+-----+-----+
```

3. Modelo `RefreshToken`

Crea el archivo `app/Models/RefreshToken.php` :

```

<?php

declare(strict_types=1);

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class RefreshToken extends Model
{
    protected $table = 'refresh_tokens';

    protected $fillable = [
        'user_id',
        'token',
        'expires_at',
        'revoked_at',
    ];

    protected function casts(): array
    {
        return [
            'expires_at' => 'datetime',
            'revoked_at' => 'datetime',
        ];
    }

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    public function isExpired(): bool
    {
        return $this->expires_at->isPast();
    }

    public function isRevoked(): bool
    {
        return $this->revoked_at !== null;
    }

    public function isValid(): bool
    {
        return !$this->isExpired() && !$this->isRevoked();
    }

    public function revoke(): void
    {
        $this->update(['revoked_at' => now()]);
    }
}

```

3.1 Explicación del modelo

`$fillable`

Solo las columnas que deben asignarse masivamente. `user_id`, `token`, y `expires_at` se asignan al crear el token. `revoked_at` se asigna al revocarlo (también vía `$fillable` porque el método `revoke()` usa `update()` que es mass-assignment).

`casts()`

Los casts `datetime` convierten automáticamente los strings de fecha de la base de datos a instancias de `Carbon`:

```

$token = RefreshToken::first();
$token->expires_at->isPast() // true/false - ¿ya expiró?
$token->expires_at->diffForHumans(); // "hace 3 días"
$token->revoked_at?->format('Y-m-d H:i:s'); // null-safe

```

`user(): BelongsTo`

Relación inversa a la que definiremos en el modelo `User`. Un refresh token pertenece a un usuario. Esta relación permite:

```

$token = RefreshToken::where('token', $jwtString)->first();
$token->user; // El modelo User dueño del token
$token->user->email; // Email del dueño

```

Métodos de dominio

Método	Propósito
<code>isExpired()</code>	¿La fecha de expiración ya pasó? Encapsula <code>\$this->expires_at->isPast()</code> .
<code>isRevoked()</code>	¿Se revocó este token? Un simple <code>!== null</code> sobre <code>revoked_at</code> .
<code>isValid()</code>	¿Está activo? Combina las dos comprobaciones anteriores. Útil en el <code>AuthService</code> para validar de un vistazo.
<code>revoke()</code>	Marca el token como revocado estableciendo <code>revoked_at = now()</code> . Encapsula el <code>update()</code> para que el service no tenga que saber detalles de implementación del modelo.

¿Por qué estos métodos en el modelo y no en el Service?

Principio de “Tell, Don’t Ask”: el código cliente no debería preguntar por el estado interno de un objeto para tomar decisiones. En vez de:

```

// ✗ El Service conoce los internos del modelo
if ($token->expires_at > now() && $token->revoked_at === null) {
    // hacer algo
}

```

Hacemos:

```

// ✓ El modelo expone comportamiento, no estado
if ($token->isValid()) {
    // hacer algo
}

```

Si en el futuro `isValid()` incluye más condiciones (límite de usos, verificación de IP), el código cliente no cambia.

4. Actualización del Modelo `User`

El modelo `User` (creado en la Parte 2) necesita la relación con `RefreshToken`. Añade al archivo `app/Models/User.php`:

```
use Illuminate\Database\Eloquent\Relations\HasMany;

// Dentro de la clase User, añade:
public function refreshTokens(): HasMany
{
    return $this->hasMany(RefreshToken::class);
}
```

La relación completa queda así en el modelo `User` :

```
// app/Models/User.php - fragmento relevante
use App\Models\RefreshToken;
use Illuminate\Database\Eloquent\Relations\HasMany;

class User extends Authenticatable implements JWTSUBJECT
{
    // ... (resto del código existente de la Parte 2 + Parte 3) ...

    public function refreshTokens(): HasMany
    {
        return $this->hasMany(RefreshToken::class);
    }
}
```

Esta relación permite las siguientes operaciones en el `AuthService` :

```
// Crear un refresh token para un usuario
$user->refreshTokens()->create([
    'token' => $refreshToken,
    'expires_at' => now()->addMinutes(config('jwt.refresh_ttl'))
]);

// Listar todos los refresh tokens activos de un usuario
$user->refreshTokens()
    ->whereNull('revoked_at')
    ->where('expires_at', '>', now())
    ->get();

// Revocar todos los tokens de un usuario (logout global)
$user->refreshTokens()
    ->whereNull('revoked_at')
    ->update(['revoked_at' => now()]);
```

La última operación — revocar todos los tokens de un usuario — es el mecanismo de defensa ante la detección de reuse de refresh tokens (Parte 5).

5. `RegisterRequest` — Validación de Entrada

5.1 Crear el `FormRequest`

```
php artisan make:request Auth/RegisterRequest
```

Sobrescribe el contenido generado en `app/Http/Requests/Auth/RegisterRequest.php` :

```

<?php

declare(strict_types=1);

namespace App\Http\Requests\Auth;

use Illuminate\Foundation\Http\FormRequest;

class RegisterRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'min:2', 'max:100'],
            'email' => ['required', 'string', 'email:rfc,dns', 'max:255', 'unique:users,email'],
            'password' => ['required', 'string', 'min:8', 'confirmed'],
            'password_confirmation' => ['required', 'string'],
        ];
    }

    public function messages(): array
    {
        return [
            'email.unique' => 'Este correo electrónico ya está registrado.',
            'password.min' => 'La contraseña debe tener al menos :min caracteres.',
            'password.confirmed' => 'Las contraseñas no coinciden.',
        ];
    }
}

```

5.2 Explicación de cada regla

`authorize(): bool`

```

public function authorize(): bool
{
    return true;
}

```

Cualquier persona, autenticada o no, puede registrarse. `true` indica que no hay restricciones de autorización a nivel de FormRequest.

¿Cuándo retornarías `false` ? Si el registro fuera solo por invitación, podrías validar aquí un token de invitación o verificar que el email está en una lista blanca:

```

// Ejemplo hipotético - NO implementado en este manual
public function authorize(): bool
{
    return Invitation::where('email', $this->input('email'))
        ->whereNull('used_at')
        ->exists();
}

```

Para este manual, registro público.

`name` : `['required', 'string', 'min:2', 'max:100']`

Regla	Qué valida	Por qué
<code>required</code>	El campo debe estar presente y no vacío	Sin nombre, no podemos crear el usuario
<code>string</code>	Debe ser un string (no un array, objeto, o número)	Previene ataques de type juggling
<code>min:2</code>	Mínimo 2 caracteres	El nombre más corto razonable (ej: "Li", "Jo"). Menos de 2 caracteres no es un nombre real
<code>max:100</code>	Máximo 100 caracteres	El nombre real más largo registrado ronda los 200 caracteres, pero 100 es un límite práctico que cubre el 99.99% de los casos sin permitir abuso

`email` : `['required', 'string', 'email:rfc,dns', 'max:255', 'unique:users,email']`

`email:rfc,dns`

La regla `email` de Laravel acepta parámetros:

Parámetro	Qué valida
<code>rfc</code>	Valida el formato del email según RFC 5322. Rechaza <code>notanemail</code> , <code>@missingusername.com</code> , <code>spaces in@email.com</code> .
<code>dns</code>	Verifica que el dominio del email tenga registros MX (Mail Exchange) en DNS. Rechaza <code>user@thisdomaindoesnotexist12345.com</code> .
<code>spooof</code>	Verifica que el email no use caracteres Unicode sospechosos para ataques de homógrafo (ej: <code>user@google.com</code> donde las <code>o</code> son letras griegas).
<code>filter</code>	Usa <code>filter_var()</code> de PHP con <code>FILTER_VALIDATE_EMAIL</code> . Es la validación más básica y la que Laravel aplica si no pasas parámetros.
<code>strict</code>	Valida usando <code>filter_var()</code> con <code>FILTER_FLAG_EMAIL_UNICODE</code> + verificación de conformidad con RFC.

Usar `email:rfc,dns` es la validación más estricta que Laravel ofrece sin dependencias externas. Sin embargo, tiene un caveat importante:

El parámetro `dns` puede fallar en desarrollo local si no tienes resolución DNS configurada o si el dominio de prueba usa un proveedor de email temporal que no tiene registros MX. En entornos de desarrollo, considera omitir `dns` o usar `email:rfc` solamente:

```
// Desarrollo (menos estricto)
'email' => ['required', 'string', 'email:rfc', 'max:255', 'unique:users,email'],
```

Para producción, `dns` es una capa adicional de defensa contra bots de registro que usan emails de dominios

inexistentes.

```
unique:users,email
```

Verifica que el email no exista ya en la columna `email` de la tabla `users`. Laravel ejecuta:

```
SELECT COUNT(*) FROM users WHERE email = ? AND deleted_at IS NULL
```

Si el resultado > 0 , la validación falla con el mensaje “Este correo electrónico ya está registrado”.

Dos matices importantes:

1. El `deleted_at IS NULL` **es automático**: Gracias al trait `SoftDeletes` en el modelo `User`, Eloquent añade el scope global. Esto significa que un email de un usuario soft-deleted **no bloquea** el registro de uno nuevo — la consulta `unique` encuentra 0 resultados.
2. **Race condition**: Dos requests simultáneos con el mismo email pueden pasar la validación ambos. La validación de Laravel ocurre ANTES de llegar al Service. Si dos requests llegan al mismo tiempo:

```
Request A: unique:users,email → 0 resultados → OK
Request B: unique:users,email → 0 resultados → OK
Request A: INSERT INTO users → éxito
Request B: INSERT INTO users → ✗ Integrity constraint violation (UNIQUE en DB)
```

La defensa en profundidad es el índice `UNIQUE` sobre `users.email` (creado en la Parte 2). Aunque dos requests pasen la validación de Laravel, MySQL rechaza el segundo INSERT con un error de constraint. El `DB::transaction()` en el `AuthService` (sección 6) capturará esto como una excepción y hará rollback.

`unique` **VS** `exists`: `unique:users,email` verifica que el valor NO existe (es único). `exists:users,email` verifica que SÍ existe (útil para login: “¿existe un usuario con este email?”). Son opuestos. No los confundas.

```
max:255
```

El RFC 5321 permite emails de hasta 254 caracteres. `VARCHAR(255)` en la base de datos + `max:255` en validación cubren el caso máximo. Si alguien envía un email de 256 caracteres, la validación lo rechaza antes de que llegue a la base de datos.

```
password : ['required', 'string', 'min:8', 'confirmed']
```

```
min:8
```

NIST SP 800-63B (Digital Identity Guidelines) recomienda un mínimo de 8 caracteres para contraseñas generadas por humanos. Es el balance entre seguridad y usabilidad.

Longitud mínima	Seguridad	Usabilidad	Caso de uso
6 caracteres	Baja — fuerza bruta viable en minutos	Alta	Aplicaciones de bajo riesgo
8 caracteres	Media — fuerza bruta toma horas/días con rate limiting	Buena	Aplicaciones web estándar (este manual)
12 caracteres	Alta — fuerza bruta no viable	Media	Aplicaciones financieras, healthcare
16+ caracteres	Muy alta	Baja (gestores de contraseñas)	Sistemas críticos

confirmed

La regla `confirmed` de Laravel espera un campo hermano con el sufijo `_confirmation`. Es decir, si el campo es `password`, Laravel busca `password_confirmation` en la request y verifica que ambos valores sean iguales. Si no son iguales o si `password_confirmation` no está presente, la validación falla.

```
// ✓ Válido — password y password_confirmation coinciden
{
  "password": "Secure123!",
  "password_confirmation": "Secure123!"
}

// ✗ Inválido — no coinciden
{
  "password": "Secure123!",
  "password_confirmation": "Secure123"
}

// ✗ Inválido — falta password_confirmation
{
  "password": "Secure123!"
}
```

¿Por qué NO validar complejidad (mayúsculas, números, símbolos)?

NIST SP 800-63B **ya NO recomienda** reglas de complejidad como “al menos una mayúscula, un número y un símbolo”. El razonamiento:

1. **Los usuarios encuentran formas de burlarlas:** `Password1!` cumple todas las reglas de complejidad pero es trivial de adivinar.
2. **Reducen la entropía real:** Al obligar a ciertos patrones, reduces el espacio de búsqueda. Saber que “debe tener al menos un número” elimina todas las contraseñas sin números del espacio de ataque.
3. **Frustran al usuario:** Llevan a comportamientos inseguros como anotar la contraseña o reutilizarla en otros sitios.

Lo que NIST recomienda HOY: - **Longitud mínima** (8 caracteres para claves humanas, 6 para claves generadas aleatoriamente). - **Verificación contra diccionarios de contraseñas comunes** (ej: haveibeenpwned.com). - **No exigir cambios periódicos** de contraseña sin evidencia de compromiso. - **No exigir composición arbitraria** (mayúsculas, números, símbolos).

En la sección 9.2 mencionamos cómo añadir verificación contra haveibeenpwned.com como mejora futura.

`password_confirmation` : ['required', 'string']

El campo `password_confirmation` debe estar presente y ser un string. No necesita `min:8` porque su único propósito es compararse con `password` — si `password` pasa la validación, la confirmación es redundante validarla por separado. Sin embargo, `required` es necesario: si falta, la regla `confirmed` sobre `password` generará un error confuso (“las contraseñas no coinciden” cuando en realidad falta el campo). Con `required` explícito, Laravel da un error más claro: “el campo `password_confirmation` es obligatorio”.

5.3 `messages()` — Personalización de errores

```
public function messages(): array
{
    return [
        'email.unique'      => 'Este correo electrónico ya está registrado.',
        'password.min'      => 'La contraseña debe tener al menos :min caracteres.',
        'password.confirmed' => 'Las contraseñas no coinciden.',
    ];
}
```

Laravel tiene mensajes de error por defecto en inglés. Personalizarlos al español mejora la experiencia de usuario. Los mensajes por defecto que NO personalizamos son suficientemente claros:

Regla	Mensaje por defecto (inglés)	Traducción automática de Laravel (si configuras <code>config/app.php</code> → <code>locale = 'es'</code>)
<code>required</code>	“The name field is required.”	“El campo nombre es obligatorio.”
<code>email</code>	“The email must be a valid email address.”	“El campo email debe ser una dirección de correo electrónico válida.”
<code>min:2</code>	“The name must be at least 2 characters.”	“El campo nombre debe tener al menos 2 caracteres.”

El placeholder `:min` en `'password.min'` se reemplaza dinámicamente por el valor de la regla (`8`). El mensaje resultante es: “La contraseña debe tener al menos 8 caracteres.”

5.4 ¿Qué pasa cuando la validación falla?

Laravel maneja automáticamente la respuesta de error cuando un `FormRequest` falla la validación. El cliente recibe:

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/json

{
  "message": "The email field is required. (and 2 more errors)",
  "errors": {
    "email": [
      "The email field is required."
    ],
    "password": [
      "The password field confirmation does not match."
    ],
    "name": [
      "The name field must be at least 2 characters."
    ]
  }
}
```

El código `422 Unprocessable Content` (antes `422 Unprocessable Entity`) es el estándar HTTP para errores de validación. Laravel lo establece automáticamente — no necesitas hacer nada en el controller.

6. AuthService::register() — Lógica de Negocio

Crea el archivo `app/Services/AuthService.php` con **solo** el método `register()`. Los métodos `login()`, `refreshTokens()` y `logout()` se implementarán en la Parte 5.

```
<?php

declare(strict_types=1);

namespace App\Services;

use App\Enums\UserRole;
use App\Enums\UserStatus;
use App\Models\User;
use Illuminate\Support\Facades\DB;

final class AuthService
{
    public function register(array $data): array
    {
        return DB::transaction(function () use ($data): array {
            $user = User::create([
                'name'      => $data['name'],
                'email'     => $data['email'],
                'password'  => $data['password'],
                'role'      => UserRole::User,
                'status'    => UserStatus::Active,
            ]);

            $accessToken = auth('api')->login($user);
            $refreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);

            $user->refreshTokens()->create([
                'token'      => $refreshToken,
                'expires_at' => now()->addMinutes(config('jwt.refresh_ttl')),
            ]);

            return [
                'user'      => $user,
                'access_token' => $accessToken,
                'refresh_token' => $refreshToken,
                'token_type'  => 'bearer',
                'expires_in' => config('jwt.ttl') * 60,
            ];
        });
    }
}
```

6.1 Explicación detallada

```
final class AuthService
```

```
final class AuthService
```

`final` previene que otra clase extienda `AuthService`. Los services son unidades concretas de lógica de negocio, no diseñadas para herencia. Si necesitas extender comportamiento, usa composición (inyectar otro service como dependencia) o el patrón Strategy.

```
DB::transaction()
```

```
return DB::transaction(function () use ($data): array {
    // ...
});
```

Todo el registro ocurre dentro de una transacción de base de datos. Esto garantiza **atomicidad**: o todas las operaciones se completan exitosamente, o ninguna se aplica.

¿Qué operaciones están dentro de la transacción?

1. `User::create()` — INSERT en `users`
2. `$user->refreshTokens()->create()` — INSERT en `refresh_tokens`

Si la segunda operación falla (por ejemplo, por una violación de FK o un error de constraint), MySQL revierte automáticamente la primera. El usuario no se crea a medias.

Si ocurre una excepción de PHP (no de MySQL) dentro del closure, Laravel hace rollback de la transacción. Esto incluye excepciones lanzadas por el paquete JWT, errores de tipo, o cualquier `Throwable`.

¿Qué NO está dentro de la transacción?

Las llamadas a `auth('api')->login($user)` no tocan la base de datos — generan el token JWT en memoria (codificación + firma). No hay nada que “deshacer” si fallan, pero si fallaran, la excepción haría rollback del `User::create()` anterior.

`User::create()`

```
$user = User::create([
    'name' => $data['name'],
    'email' => $data['email'],
    'password' => $data['password'],
    'role' => UserRole::User,
    'status' => UserStatus::Active
]);
```

Campo	Valor	Quién lo decide
<code>name</code>	Lo que envió el cliente	Cliente
<code>email</code>	Lo que envió el cliente (ya validado)	Cliente
<code>password</code>	Texto plano enviado por el cliente	Cliente

¿Texto plano? **Sí**. Y es correcto. El cast `'password' => 'hashed'` en el modelo `User` (Parte 2) convierte automáticamente el valor a un hash bcrypt antes de guardarlo en la base de datos:

```
// El cast 'hashed' internamente hace esto:
$this->attributes['password'] = Hash::make($value);
```

Esto significa que `$data['password']` llega como `"Secure123!"` pero se guarda como `"$2y$12$LJ3m4ys3GZfnYMz8kVsKa0mxAZjFPqFq7dQ8eY0..."`. El password en texto plano **nunca** se persiste.

Campo	Valor	Quién lo decide
<code>role</code>	<code>UserRole::User</code>	El servidor , no el cliente
<code>status</code>	<code>UserStatus::Active</code>	El servidor , no el cliente

El rol y el estado NO vienen del cliente. Son decisiones del servidor. Todo usuario nuevo es `user` (no `admin`) y

`active` (no `banned`). Si el cliente enviara `"role": "admin"` en el JSON, el `RegisterRequest` ni siquiera lo validaría (no está en `rules()`), y aunque llegara al Service, `$fillable` lo aceptaría. **Pero no lo enviamos a `User::create()`**. Hardcodeamos los valores en el Service.

Esta es la defensa en profundidad contra escalación de privilegios en el registro: 1. `RegisterRequest` no valida `role` → no está en `$request->validated()`. 2. `AuthService` no usa `$data['role']` → lo asigna explícitamente. 3. El modelo `User` tiene `'role'` en `$fillable`, pero como el Service no lo pasa, no se asigna.

¿Por qué no quitar `role` de `$fillable`? Porque otros flujos (actualización de perfil, panel admin) Sí necesitan asignar `role` masivamente. `$fillable` no es una restricción de seguridad por endpoint, es una restricción de seguridad por modelo. La lógica de negocio (qué roles se permiten en cada flujo) vive en el Service, no en el modelo.

`auth('api')->login($user)` — Access Token

```
$accessToken = auth('api')->login($user);
```

Desglose de lo que ocurre:

1. `auth('api')` obtiene el guard `api` configurado en `config/auth.php`.
2. El guard `api` tiene driver `jwt`, proporcionado por `tymon/jwt-auth`.
3. `login($user)` del driver JWT:
 - Llama a `$user->getJWTIdentifier()` → obtiene `1` (el ID del usuario recién creado).
 - Llama a `$user->getJWTCustomClaims()` → obtiene `['role' => 'user', 'name' => 'John Doe']`.
 - Construye el payload con los claims estándar (`iss`, `iat`, `exp`, `nbf`, `jti`, `sub`, `prv`) y los claims personalizados.
 - Firma el token con `JWT_SECRET` usando HS256.
 - Codifica todo a Base64URL.
 - Retorna el string JWT completo.

El TTL de este token es el configurado en `config/jwt.ttl`: 15 minutos (`JWT_TTL=15`). El claim `exp` se calcula como `now() + 900 segundos`.

`login()` **de JWT NO es `auth()->login()` de sesiones de Laravel**. El método `login()` del paquete `tymon/jwt-auth` CREA y FIRMA un token JWT. No inicia una sesión en el servidor. No escribe nada en `sessions` o `cache`. Es una operación puramente criptográfica en memoria.

`auth('api')->setTTL(...)->login($user)` — Refresh Token

```
$refreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);
```

`setTTL()` sobrescribe temporalmente el TTL **solo para esta llamada**. No modifica la configuración global. La secuencia es:

1. `auth('api')->setTTL(10080)` — “para el próximo token que generes, usa 10080 minutos como TTL”.
2. `->login($user)` — genera el token con TTL de 7 días en lugar de 15 minutos.

El resultado es un JWT con `exp = now() + 604800 segundos` (7 días). El resto del payload es idéntico al access token. La diferencia está únicamente en el claim `exp`.

¿Por qué no usar `claims()` para marcar el tipo de token?

Podríamos añadir un claim `token_type: "refresh"` así:

```
$refreshToken = auth('api')
->claims(['token_type' => 'refresh'])
->setTTL(config('jwt.refresh_ttl'))
->login($user);
```

Esto permitiría que el middleware JWT distinga access tokens de refresh tokens y rechace refresh tokens en endpoints normales. Sin embargo, este manual mantiene ambos tokens con la misma estructura (la distinción está en la tabla `refresh_tokens` para los refresh tokens). En la Parte 5, si se requiere esta distinción a nivel de middleware, se puede añadir.

Guardar el refresh token en la base de datos

```
$user->refreshTokens()->create([
    'token' => $refreshToken,
    'expires_at' => now()->addMinutes(config('jwt.refresh_ttl'))
]);
```

- `$user->refreshTokens()` usa la relación `HasMany` definida en el modelo `User`.
- `create()` inserta una nueva fila en `refresh_tokens` con el `user_id` del usuario, el token JWT completo, y la fecha de expiración.
- `now()->addMinutes(config('jwt.refresh_ttl'))` calcula `2025-01-22 10:30:00` (7 días después de `now()`).

El `expires_at` en la base de datos es redundante con el claim `exp` dentro del JWT. Esta redundancia es deliberada: permite validar la expiración con SQL sin decodificar el JWT (más rápido, y útil para el comando de limpieza programada).

Respuesta del método

```
return [
    'user' => $user,
    'access_token' => $accessToken,
    'refresh_token' => $refreshToken,
    'token_type' => 'bearer',
    'expires_in' => config('jwt.ttl') * 60,
];
```

Campo	Valor de ejemplo	Explicación
<code>user</code>	Instancia de <code>User</code>	Al serializarse a JSON, Eloquent aplica <code>\$hidden</code> (excluye <code>password</code> , <code>remember_token</code>) y <code>casts</code> (enum a string, fechas a ISO 8601).
<code>access_token</code>	"eyJ0eXAiOiJKV1Q..."	JWT de 15 minutos de vida.
<code>refresh_token</code>	"eyJ0eXAiOiJKV1Q..."	JWT de 7 días de vida.
<code>token_type</code>	"bearer"	El tipo de token para el header <code>Authorization: Bearer <token></code> . Es una convención, no un mecanismo de seguridad.
<code>expires_in</code>	900	TTL en segundos ($15 \times 60 = 900$). Esto sigue la convención de OAuth2 y permite al frontend calcular cuándo refrescar sin decodificar el JWT.

El campo `expires_in` en segundos (no en minutos) es una convención de APIs OAuth2. Si el frontend usa una librería como `axios` con interceptors para refrescar tokens automáticamente, espera este formato.

7. RegisterController — Punto de Entrada HTTP

Crea el archivo `app/Http/Controllers/Auth/RegisterController.php`:

```
<?php

declare(strict_types=1);

namespace App\Http\Controllers\Auth;

use App\Http\Requests\Auth\RegisterRequest;
use App\Services\AuthService;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Response;

final class RegisterController
{
    public function __invoke(RegisterRequest $request, AuthService $authService): JsonResponse
    {
        $result = $authService->register($request->validated());

        return response()->json([
            'data' => $result,
        ], Response::HTTP_CREATED);
    }
}
```

7.1 Explicación

Single-action controller (`__invoke`)

```
final class RegisterController
{
    public function __invoke(RegisterRequest $request, AuthService $authService): JsonResponse
```

Una clase con un solo método `__invoke` es un **single-action controller**. Enrutar `Route::post('/register', RegisterController::class)` llama automáticamente a `__invoke`. Responsabilidad única: esta clase existe solo para manejar el registro de usuarios. Si necesitas cambiar el login, no tocas este archivo.

Inyección de dependencias

```
public function __invoke(RegisterRequest $request, AuthService $authService): JsonResponse
```

Laravel resuelve automáticamente `RegisterRequest` (inyecta la request HTTP validada) y `AuthService` (lo resuelve del contenedor de servicios). No necesitas `new AuthService()` ni `app()->make(AuthService::class)`. El contenedor de Laravel lo hace por ti.

`AuthService` no tiene dependencias en su constructor, así que el contenedor lo instancia directamente. Si en el futuro `AuthService` requiriera dependencias (ej: `UserRepository`, `TokenService`), el contenedor las resolvería recursivamente. El controller no cambia.

```
$request->validated()
```

```
$result = $authService->register($request->validated());
```

`validated()` retorna **solo los campos que pasaron la validación** (los definidos en `rules()`). No retorna `_token`, `_method`, query params, ni campos adicionales que el cliente haya enviado.

Comparación:

```
// ✗ Peligroso – incluye TODOS los campos de la request
$data = $request->all();

// △ Aceptable si confías en $fillable, pero no es explícito
$data = $request->only(['name', 'email', 'password', 'password_confirmation']);

// ✔ Seguro y explícito – solo lo validado
$data = $request->validated();
```

`validated()` devuelve `password_confirmation` además de `name`, `email`, `password`. El `AuthService::register()` no usa `password_confirmation` (su propósito era solo para la validación `confirmed`), pero no hay problema con que esté en el array — `User::create()` usa `$fillable` para filtrar.

HTTP 201 Created

```
return response()->json([
    'data' => $result,
], Response::HTTP_CREATED);
```

`Response::HTTP_CREATED` es la constante `201` de Laravel. Usar la constante en vez del número mágico:

```
// ✖ Número mágico - ¿qué significa 201?
return response()->json(['data' => $result], 201);

// ✔ Constante con nombre - autoexplicativo
return response()->json(['data' => $result], Response::HTTP_CREATED);
```

HTTP 201 Created es el código semánticamente correcto para la creación exitosa de un recurso:

Código	Significado	Cuándo usarlo
200 OK	La petición fue exitosa	GET, PUT, PATCH, DELETE
201 Created	Se creó un recurso nuevo	POST que resulta en una nueva entidad
202 Accepted	La petición fue aceptada pero no procesada aún	Operaciones asíncronas
204 No Content	Éxito sin cuerpo de respuesta	DELETE exitoso sin datos que devolver

El controller tiene ~5 líneas de lógica. No hay `if`, no hay bucles, no hay acceso a base de datos. Toda la complejidad vive en el `RegisterRequest` (validación) y el `AuthService` (lógica de negocio).

8. Ruta de Registro

Añade en `routes/api.php`:

```
use App\Http\Controllers\Auth\RegisterController;

Route::post('/register', RegisterController::class)
    ->name('auth.register')
    ->middleware('throttle:10,1');
```

8.1 Explicación

Ruta POST a `/register`

Laravel carga `routes/api.php` con el prefijo `/api` (definido en `bootstrap/app.php` en Laravel 11). La ruta completa es `POST /api/register`.

`->name('auth.register')`

Nombrar las rutas permite referenciarlas simbólicamente en otros lugares del código:

```
// En tests:
$response = $this->postJson(route('auth.register'), $data);

// En generar URLs:
$url = route('auth.register'); // "http://localhost:8000/api/register"
```

Si algún día cambias la URL de `/register` a `/signup`, solo modificas la ruta. Todas las referencias por nombre siguen funcionando.

`->middleware('throttle:10,1')`

Rate limiting: máximo 10 intentos de registro por minuto desde una misma IP.

Parámetro	Significado
10	Número máximo de intentos permitidos
1	Ventana de tiempo en minutos

Si un cliente (o bot) intenta más de 10 registros en 1 minuto, Laravel responde:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
Content-Type: application/json

{
  "message": "Too Many Requests"
}
```

¿Por qué rate limiting en el registro?

El endpoint de registro tiene tres vectores de abuso: 1. **Creación masiva de cuentas** (spam, bots) — rate limiting lo frena. 2. **Enumeración de emails** (probar si `x@y.com` ya existe) — rate limiting + mensajes genéricos de error lo mitigan (ver sección 9.3). 3. **Denegación de servicio por agotamiento de recursos** — crear usuarios consume INSERTs, hashéo bcrypt (CPU), y generación de JWT (CPU). 10 por minuto mantiene esto bajo control.

9. Testing del Registro

9.1 Prueba manual con curl

```
curl -X POST http://localhost:8000/api/register \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{
    "name": "John Doe",
    "email": "john@example.com",
    "password": "SecurePass123!",
    "password_confirmation": "SecurePass123!"
  }'
```

Headers explicados:

Header	Propósito
Content-Type: application/json	Indica al servidor que el cuerpo es JSON. Sin esto, Laravel podría interpretar el body como form-data y la validación fallaría.
Accept: application/json	Indica al servidor que el cliente espera respuesta JSON. Si la validación falla, Laravel devolverá JSON con errores en vez de redirigir (comportamiento web por defecto). Crítico para APIs.

Respuesta esperada (éxito):

```

HTTP/1.1 201 Created
Content-Type: application/json

{
  "data": {
    "user": {
      "id": 1,
      "name": "John Doe",
      "email": "john@example.com",
      "role": "user",
      "status": "active",
      "email_verified_at": null,
      "created_at": "2025-01-15T10:30:00.000000Z",
      "updated_at": "2025-01-15T10:30:00.000000Z"
    },
    "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
    "refresh_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
    "token_type": "bearer",
    "expires_in": 900
  }
}

```

Observa que el `user` NO incluye `password` ni `remember_token` — el `$hidden` del modelo los excluyó. Tampoco incluye `deleted_at` porque es `null` y Eloquent omite atributos `null` en la serialización por defecto.

9.2 Prueba con Postman / Insomnia

Crea una colección con la siguiente configuración:

```

Method: POST
URL: http://localhost:8000/api/register

Headers:
  Content-Type: application/json
  Accept: application/json

Body (raw JSON):
{
  "name": "Jane Doe",
  "email": "jane@example.com",
  "password": "SecurePass123!",
  "password_confirmation": "SecurePass123!"
}

```

Guarda la colección. La reutilizarás en las Partes 5 y 6 para probar login, refresh y logout.

9.3 Pruebas con PHPUnit

Crea el archivo `tests/Feature/Auth/RegisterTest.php`:

```

<?php

declare(strict_types=1);

namespace Tests\Feature\Auth;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Facades\Hash;
use Tests\TestCase;

class RegisterTest extends TestCase
{
    use RefreshDatabase;

    public function test_user_can_register(): void
    {
        $response = $this->postJson('/api/register', [
            'name' => 'Test User',
            'email' => 'test@example.com',
            'password' => 'Secure123!',
            'password_confirmation' => 'Secure123!',
        ]);

        $response->assertStatus(201)
            ->assertJsonStructure([
                'data' => [
                    'user' => [
                        'id',
                        'name',
                        'email',
                        'role',
                        'status',
                        'created_at',
                        'updated_at',
                    ],
                    'access_token',
                    'refresh_token',
                    'token_type',
                    'expires_in',
                ],
            ]);

        $this->assertDatabaseHas('users', [
            'email' => 'test@example.com',
        ]);

        $this->assertDatabaseHas('refresh_tokens', [
            'user_id' => 1,
        ]);

        $user = User::where('email', 'test@example.com')->first();
        $this->assertNotNull($user);
        $this->assertNotEquals('Secure123!', $user->password);
        $this->assertTrue(Hash::check('Secure123!', $user->password));
    }
}

```

¿Qué prueba cada aserción?

```
$response->assertStatus(201);
```

Verifica que el endpoint responde con HTTP 201 Created. Si responde 422, algo falló en la validación. Si 500, algo explotó en el Service.

```
->assertJsonStructure([...]);
```

Verifica que la respuesta JSON tiene exactamente la estructura esperada. Si falta `access_token` o `user.id`, el test falla. Esto detecta cambios accidentales en el formato de respuesta.

```
$this->assertDatabaseHas('users', ['email' => 'test@example.com']);
```

Confirma que el usuario realmente se persistió en la base de datos. No solo que el endpoint respondió 201 — el registro se materializó.

```
$this->assertDatabaseHas('refresh_tokens', ['user_id' => 1]);
```

Confirma que se creó un refresh token para el usuario recién registrado. Verifica que la transacción funcionó completa.

```
$this->assertNotEquals('Secure123!', $user->password);  
$this->assertTrue(Hash::check('Secure123!', $user->password));
```

La prueba más importante de todas: el password NO está en texto plano en la base de datos. `assertNotEquals` verifica que el valor guardado es diferente al input. `Hash::check` verifica que el hash corresponde al password original.

¿Por qué ambas aserciones? `assertNotEquals` detecta el caso obvio (error humano). `Hash::check` detecta el caso sutil: el cast `hashed` podría estar roto y guardar un hash de otro string. Si `Hash::check` falla, el password guardado no puede usarse para login.

9.4 Casos de prueba negativos

Añade estos tests al mismo archivo:

```
public function test_registration_fails_with_duplicate_email(): void  
{  
    User::factory()->create(['email' => 'existing@example.com']);  
  
    $response = $this->postJson('/api/register', [  
        'name' => 'Duplicate User',  
        'email' => 'existing@example.com',  
        'password' => 'Secure123!',  
        'password_confirmation' => 'Secure123!',  
    ]);  
  
    $response->assertStatus(422)  
        ->assertJsonValidationErrors(['email']);  
}  
  
public function test_registration_fails_with_short_password(): void  
{  
    $response = $this->postJson('/api/register', [  
        'name' => 'Short Pass',  
        'email' => 'short@example.com',  
        'password' => '1234567',  
        'password_confirmation' => '1234567',  
    ]);  
  
    $response->assertStatus(422)  
        ->assertJsonValidationErrors(['password']);  
}  
  
public function test_registration_fails_without_password_confirmation(): void  
{  
    $response = $this->postJson('/api/register', [  
        'name' => 'No Confirm',  
        'email' => 'noconfirm@example.com',  
        'password' => 'Secure123!',  
    ]);  
  
    $response->assertStatus(422);  
}
```

```

    }

    public function test_registration_fails_with_invalid_email(): void
    {
        $response = $this->postJson('/api/register', [
            'name' => 'Bad Email',
            'email' => 'not-an-email',
            'password' => 'Secure123!',
            'password_confirmation' => 'Secure123!',
        ]);

        $response->assertStatus(422)
            ->assertJsonValidationErrors(['email']);
    }

    public function test_registration_fails_with_empty_name(): void
    {
        $response = $this->postJson('/api/register', [
            'name' => '',
            'email' => 'empty@example.com',
            'password' => 'Secure123!',
            'password_confirmation' => 'Secure123!',
        ]);

        $response->assertStatus(422)
            ->assertJsonValidationErrors(['name']);
    }

    public function test_registration_enforces_rate_limiting(): void
    {
        for ($i = 0; $i < 10; $i++) {
            $this->postJson('/api/register', [
                'name' => "User {$i}",
                'email' => "user{$i}@example.com",
                'password' => 'Secure123!',
                'password_confirmation' => 'Secure123!',
            ]);
        }

        $response = $this->postJson('/api/register', [
            'name' => 'Rate Limited',
            'email' => 'ratelimited@example.com',
            'password' => 'Secure123!',
            'password_confirmation' => 'Secure123!',
        ]);

        $response->assertStatus(429);
    }
}

```

Test	Qué verifica
fails_with_duplicate_email	El índice UNIQUE + regla <code>unique:users,email</code> funcionan
fails_with_short_password	<code>min:8</code> rechaza contraseñas de 7 caracteres
fails_without_password_confirmation	<code>confirmed</code> falla cuando falta <code>password_confirmation</code>
fails_with_invalid_email	<code>email:rfc,dns</code> rechaza strings que no son emails
fails_with_empty_name	<code>required</code> + <code>min:2</code> rechazan strings vacíos
enforces_rate_limiting	<code>throttle:10,1</code> bloquea después de 10 intentos

9.5 Ejecutar los tests

```
php artisan test --filter RegisterTest
```

Salida esperada:

```
PASS Tests\Feature\Auth\RegisterTest
✓ user can register
✓ registration fails with duplicate email
✓ registration fails with short password
✓ registration fails without password confirmation
✓ registration fails with invalid email
✓ registration fails with empty name
✓ registration enforces rate limiting

Tests: 7 passed
```

10. Seguridad Específica del Registro

10.1 Rate Limiting

Ya implementado en la ruta con `throttle:10,1`. Diez registros por minuto es suficiente para un usuario legítimo (nadie abre 10 cuentas en un minuto) pero insuficiente para un bot de registro masivo.

La respuesta 429 incluye el header `Retry-After` para que el frontend sepa cuánto esperar. Si usas Axios, puedes interceptar 429 y mostrar un mensaje al usuario.

10.2 Password Strength

Tres niveles de defensa, de menor a mayor inversión:

Nivel	Qué incluye	Implementado en este manual
Básico	Longitud mínima (8 chars) + confirmación	✓ <code>min:8</code> + <code>confirmed</code>
Intermedio	Verificación contra <code>haveibeenpwned.com</code>	✗ Mejora futura
Avanzado	Diccionario de contraseñas comunes + <code>zxcvbn</code> (estimación de entropía)	✗ No planeado

Mejora futura recomendada: instalar el paquete `arubacao/laravel-password-validation`. Añade una regla de validación que verifica contraseñas contra la API de `haveibeenpwned.com` (sin enviar la contraseña completa, solo los primeros 5 caracteres del hash SHA-1, usando k-anonymity). Ejemplo:

```
// Implementación futura con el paquete
'password' => ['required', 'string', 'min:8', 'confirmed', new PwnedPassword()],
```

10.3 Email Uniqueness y Timing Attacks

El mensaje de error personalizado en `RegisterRequest` revela si un email ya está registrado:

```
{
  "errors": {
    "email": ["Este correo electrónico ya está registrado."]
  }
}
```

Esto permite a un atacante **enumerar** emails registrados en el sistema: prueba `admin@empresa.com` → “ya está registrado” → sabe que existe un admin con ese email. Prueba `noexiste@empresa.com` → 201 Created → sabe que no existe.

¿Es esto aceptable?

Depende del modelo de amenaza de tu aplicación:

Escenario	¿Aceptable?	Mitigación
Aplicación B2C (red social, ecommerce)	Sí, en general	UX de “ese email ya tiene cuenta” es esperado
Aplicación B2B/enterprise	Precaución	Podría revelar qué empresas usan tu servicio
Aplicación de alta seguridad (banca, gobierno)	No	Mensaje genérico + verificación por email

Mitigación parcial que ya tenemos: rate limiting (`throttle:10,1`) limita la velocidad a la que un atacante puede probar emails. Probar 1000 emails tomaría 100 minutos como mínimo.

Mitigación completa (no implementada en este manual): No revelar si el email existe. En vez de “ya está registrado”, siempre responder con éxito y enviar un email de verificación. Si el email ya existe, el email dice “alguien intentó registrar esta cuenta — si no fuiste tú, ignora este mensaje”. El atacante nunca sabe si el email existía o no.

10.4 SQL Injection

El uso exclusivo de Eloquent + bindings en este flujo previene inyección SQL automáticamente. Cada interacción con la base de datos usa prepared statements:

```
// ✔ Seguro – Eloquent con bindings
User::create($data);
$user->refreshTokens()->create([...]);
User::where('email', $email)->first();
```

No se usa `DB::raw()` , `whereRaw()` , ni concatenación de strings con input de usuario. **Si añadieras esto en el futuro, asegúrate de usar bindings explícitos.**

10.5 XSS en el nombre

El campo `name` se almacena tal cual lo envía el cliente. Si un atacante se registra con:

```
{
  "name": "<script>alert('XSS')</script>",
  "email": "xss@evil.com"
}
```

La base de datos guardará el script. Cuando ese nombre se devuelva en respuestas JSON, **el backend no escapa nada** porque JSON no tiene contexto de HTML. La responsabilidad de escapar para HTML es del **frontend**.

En Vue/React, esto se maneja automáticamente (escapan por defecto). Si el frontend usa `innerHTML` o `dangerouslySetInnerHTML`, debe sanear el nombre.

Opción de saneamiento en backend: aplicar `strip_tags()` en el `AuthService`:

```
'name' => strip_tags($data['name']),
```

Esto eliminaría tags HTML del nombre antes de guardarlo. Sin embargo, es una decisión controversial: estás modificando el input del usuario. Un nombre legítimo como `"O'Reilly"` no debería ser afectado, pero `strip_tags()` no lo afecta (solo elimina `<tag>`). Para este manual, no aplicamos `strip_tags()` — confiamos en que el frontend maneje la renderización segura de datos provenientes de la API.

10.6 Mass Assignment — `$fillable` y `role`

El campo `role` está en `$fillable`, pero el `AuthService::register()` lo asigna explícitamente a `UserRole::User`. Si un atacante intenta:

```
{
  "name": "Hacker",
  "email": "hacker@evil.com",
  "password": "password123",
  "password_confirmation": "password123",
  "role": "admin"
}
```

El `RegisterRequest` acepta el campo `role` (no lo valida, pero tampoco lo rechaza — los `FormRequests` no eliminan campos, solo validan los que están en `rules()`). Sin embargo, el controller usa `$request->validated()`, que **retorna solo los campos validados**: `name`, `email`, `password`, `password_confirmation`. El campo `role` no está en `rules()`, así que `validated()` no lo incluye.

El array que llega al Service es:

```
[
  'name' => 'Hacker',
  'email' => 'hacker@evil.com',
  'password' => 'password123',
  'password_confirmation' => 'password123',
]
```

No contiene `role`. El Service hardcodea `'role' => UserRole::User`. El usuario se crea con rol `user` incluso si el atacante envió `"role": "admin"`.

11. Resumen y Puente a la Parte 5

11.1 Lo construido en esta parte

Componente	Archivo	Estado
Migración <code>refresh_tokens</code>	<code>database/migrations/xxxx_create_refresh_tokens_table.php</code>	✓ Creada y ejecutada
Modelo <code>RefreshToken</code>	<code>app/Models/RefreshToken.php</code>	✓ Con casts, helpers y relación a User
Relación en <code>User</code>	<code>app/Models/User.php</code> (actualizado)	✓ <code>refreshTokens()</code> : HasMany
<code>RegisterRequest</code>	<code>app/Http/Requests/Auth/RegisterRequest.php</code>	✓ Validación completa con mensajes en español
<code>AuthService::register()</code>	<code>app/Services/AuthService.php</code>	✓ Lógica de negocio con transacción
<code>RegisterController</code>	<code>app/Http/Controllers/Auth/RegisterController.php</code>	✓ Single-action, 5 líneas de lógica
Ruta	<code>routes/api.php</code>	✓ <code>POST /api/register</code> con rate limiting
Tests	<code>tests/Feature/Auth/RegisterTest.php</code>	✓ 7 tests: 1 positivo, 6 negativos

11.2 Flujo completo de registro

```

1. Cliente → POST /api/register (JSON con name, email, password, password_confirmation)

2. Rate Limiter: ¿más de 10 intentos en 1 minuto?
   → Sí: 429 Too Many Requests
   → No: continuar

3. RegisterRequest::authorize(): ¿el registro es público?
   → Sí (true): continuar
   → No (false): 403 Forbidden

4. RegisterRequest::rules(): validar name, email, password, password_confirmation
   → Falla: 422 Unprocessable Content con errores
   → Pasa: $request->validated()

5. RegisterController → AuthService::register(validatedData)

6. AuthService::register():
   a. DB::transaction() inicia
   b. User::create(name, email, password, role=user, status=active)
      → Cast 'hashed' hashea password automáticamente
   c. auth('api')->login($user) → access token (15 min TTL)
   d. auth('api')->setTTL(10080)->login($user) → refresh token (7 días TTL)
   e. $user->refreshTokens()->create(token, expires_at)
   f. DB::transaction() confirma

7. RegisterController → response()->json(['data' => $result], 201)

8. Cliente recibe 201 Created con user, access_token, refresh_token, token_type, expires_in

```

11.3 Lo que viene en la Parte 5

La [Parte 5](#) implementará:

- `LoginRequest` — validación de credenciales (email + password).
- `AuthService::login()` — verificación de credenciales, validación de estado del usuario (active/inactive/banned), emisión de tokens.
- `AuthService::refreshTokens()` — rotación de refresh tokens con detección de reuse.
- `AuthService::logout()` — invalidación de access token y refresh token.
- `LoginController`, `RefreshTokenController`, `LogoutController` — single-action controllers.
- **Middleware** `JwtAuthenticate` — protección de rutas con validación JWT automática.
- **Rutas protegidas** — el usuario autenticado puede acceder a su perfil y cerrar sesión.

Decisiones Vinculantes para Partes 5-6

1. **El método** `register()` **del** `AuthService` **YA EXISTE**. La Parte 5 debe añadir los métodos `login()`, `refreshTokens()`, y `logout()` a la misma clase `AuthService`. No crear servicios separados.
2. **El modelo** `RefreshToken` **YA EXISTE** con sus métodos de dominio (`isExpired()`, `isRevoked()`, `isValid()`, `revoke()`). La Parte 5 debe USAR estos métodos, no reimplementarlos ni acceder directamente a las columnas.
3. **La relación** `User::refreshTokens()` **YA EXISTE**. La Parte 5 la usa para crear, consultar y revocar refresh tokens.
4. **La tabla** `refresh_tokens` **YA ESTÁ CREADA Y POBLADA** (cada registro crea una fila). La Parte 5 debe usarla para validar refresh tokens entrantes durante el refresh.
5. **El formato de respuesta de registro YA ESTÁ DEFINIDO:**

```
{
  "data": {
    "user": { "id": 1, "name": "...", "email": "...", "role": "user", "status": "active", ... },
    "access_token": "eyJ...",
    "refresh_token": "eyJ...",
    "token_type": "bearer",
    "expires_in": 900
  }
}
```

La Parte 5 debe mantener este formato para login y refresh (misma estructura de `data`).

6. **El rate limiting YA ESTÁ CONFIGURADO** en la ruta de registro (`throttle:10,1`). La Parte 5 debe añadir rate limiting en la ruta de login (recomendado: `throttle:5,1` — 5 intentos por minuto, más restrictivo por ser endpoint de autenticación).
7. **El** `AuthService` **es final**. No puede extenderse. Si la Parte 5 necesita variaciones de comportamiento, debe usar composición o inyección de dependencias.
8. **El rol por defecto para nuevos usuarios es** `UserRole::User`. Hardcodeado en el Service. La Parte 5 debe validar que un usuario `Banned` o `Inactive` NO pueda loguearse, lanzando una excepción específica con mensaje apropiado.
9. **Los tokens JWT generados en registro usan los métodos** `getJWTIdentifier()` **y** `getJWTCustomClaims()` **del modelo** `User` (Parte 3). La Parte 5 debe usar los mismos métodos para generar tokens en login — coherencia en claims entre registro y login.

Parte 5: Login, Autenticación y Middleware JWT

1. Introducción

1.1 Dónde estamos

En las cuatro partes anteriores construimos la base completa del sistema de autenticación:

Parte	Lo construido	Estado
Parte 1	Proyecto Laravel 11, arquitectura en capas, convenciones de código, estructura de directorios	✓
Parte 2	Variables de entorno, base de datos MySQL, migración <code>users</code> , modelo <code>User</code> , enums <code>UserRole</code> y <code>UserStatus</code>	✓
Parte 3	Paquete <code>typon/jwt-auth</code> instalado y configurado, guard <code>api</code> con driver <code>jwt</code> , claims del JWT, diseño de tabla <code>refresh_tokens</code>	✓
Parte 4	Migración <code>refresh_tokens</code> , modelo <code>RefreshToken</code> , <code>RegisterRequest</code> , <code>AuthService::register()</code> , <code>RegisterController</code> , ruta de registro, tests	✓

En este momento, la aplicación **puede** registrar usuarios y emitir tokens JWT, pero **no puede**:

- Autenticar usuarios existentes con email y contraseña (login)
- Proteger rutas con middleware JWT
- Refrescar tokens expirados
- Cerrar sesión de usuarios

Eso es exactamente lo que construiremos aquí.

1.2 Objetivo de esta parte

Implementar el ciclo completo de autenticación:

Login → Access Token → Proteger Rutas → Refresh → Logout

Al terminar esta parte, la aplicación podrá: 1. Autenticar usuarios con email/password y devolver tokens. 2. Proteger rutas con un middleware JWT que rechaza peticiones no autenticadas. 3. Refrescar access tokens expirados usando refresh tokens, con detección de reuse attack. 4. Cerrar sesión invalidando tanto el access token como el refresh token.

1.3 Qué construiremos

1. `LoginRequest` — valida los datos de entrada del login.
2. `AuthService::login()` — lógica de negocio: verificar credenciales, validar estado del usuario, emitir tokens.
3. **Excepciones de dominio** — `AuthenticationException` y `AccountInactiveException`.
4. `LoginController` — punto de entrada HTTP, single-action.

5. **Rutas públicas** — agrupación `auth` con ruta de login y rate limiting.
 6. **Middleware** `JwtAuthenticate` — protege rutas verificando el token JWT en cada petición.
 7. **Rutas protegidas** — agrupación bajo `jwt.auth`.
 8. `AuthService::refreshTokens()` — rotación de refresh tokens con detección de reuse attack.
 9. `AuthService::logout()` — cierre de sesión completo.
 10. `RefreshTokenRequest` — valida el refresh token entrante.
 11. `RefreshTokenController` y `LogoutController` — single-action controllers.
 12. **Endpoint** `/me` — datos del usuario autenticado vía `UserController`.
 13. **Tests de integración** — verificar cada flujo y cada caso de error.
-

2. `LoginRequest` — Validación de Credenciales

2.1 Crear el FormRequest

```
php artisan make:request Auth/LoginRequest
```

Sobrescribe el contenido generado en `app/Http/Requests/Auth/LoginRequest.php`:

```
<?php

declare(strict_types=1);

namespace App\Http\Requests\Auth;

use Illuminate\Foundation\Http\FormRequest;

class LoginRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'email' => ['required', 'string', 'email'],
            'password' => ['required', 'string'],
        ];
    }

    public function messages(): array
    {
        return [
            'email.required' => 'El correo electrónico es obligatorio.',
            'password.required' => 'La contraseña es obligatoria.',
        ];
    }
}
```

2.2 Explicación de cada regla y las reglas AUSENTES

```
authorize(): bool
```

```
public function authorize(): bool
{
    return true;
}
```

Cualquier persona, autenticada o no, puede intentar hacer login. `true` indica que no hay restricciones de autorización a nivel de FormRequest.

Reglas presentes

Regla	Qué valida	Por qué
<code>email: required</code>	El campo debe estar presente y no vacío	Sin email no hay autenticación posible
<code>email: string</code>	Debe ser un string, no un array u objeto	Previene ataques de type juggling
<code>email: email</code>	Formato de email válido según <code>filter_var()</code>	Rechaza <code>notanemail</code> , <code>@missing</code> , <code>spaces in@email</code>
<code>password: required</code>	El campo debe estar presente y no vacío	Sin contraseña no hay autenticación posible
<code>password: string</code>	Debe ser un string, no un array u objeto	Previene type juggling

`email:rfc,dns` — ¿Por qué NO aquí?

En el `RegisterRequest` (Parte 4), usamos `email:rfc,dns` con validación estricta de formato RFC 5322 y verificación DNS de registros MX. En el `LoginRequest`, usamos solo `email` sin parámetros adicionales (equivalente a `filter`).

Razón: el usuario ya se registró. Durante el registro validamos que su email tenía un formato RFC válido y un dominio con registros MX. Re-validarlo en cada login es:

1. **Redundante:** si el email pasó la validación estricta en el registro, no va a dejar de ser válido después.
2. **Más lento:** la verificación DNS (`dns`) hace una consulta de red real (`checkdnsrr()`). En cada login, esto añade entre 50-500ms de latencia innecesaria.
3. **Punto de fallo:** si el servidor DNS del dominio del usuario está caído temporalmente, el login fallaría por un problema externo a nuestra aplicación.

La validación `email` (básica, `filter_var`) es suficiente para asegurar que el string tiene formato de email.

`min:8` para password — ¿Por qué NO aquí?

En el `RegisterRequest`, validamos `password: min:8`. En el `LoginRequest`, NO validamos longitud.

Razón: la contraseña ya fue validada durante el registro. El usuario podría haber creado su cuenta con una contraseña de 8+ caracteres, y esa es la que debe ingresar ahora. Si validáramos `min:8` en el login y el usuario ingresa correctamente una contraseña de 8 caracteres... pasa. Pero si la política de contraseñas cambiara en el futuro y este usuario tuviera una contraseña creada antes de que `min:8` fuera la regla, `min:8` en el login lo bloquearía injustamente.

Además, `min:8` en el login **no tiene valor de seguridad**: no previene brute force porque el atacante igual probará contraseñas de cualquier longitud. La defensa contra brute force es el rate limiting, no la validación de longitud.

`exists:users,email` — ¿Por qué NO aquí?

Esta es una de las reglas más tentadoras de añadir pero una de las más peligrosas:

```
// ✗ NUNCA hagas esto en login
'email' => ['required', 'string', 'email', 'exists:users,email']
```

Problema: User Enumeration Attack

Si validas `exists:users,email`, el atacante recibe respuestas diferentes según si el email existe o no:

```
POST /api/login { email: "admin@empresa.com", password: "wrong" }
→ 422 "The selected email is invalid."
  ↑ El email NO existe → el atacante sabe que "admin@empresa.com" no es un usuario

POST /api/login { email: "john@empresa.com", password: "wrong" }
→ 401 "Credenciales inválidas."
  ↑ El email SÍ existe → el atacante sabe que "john@empresa.com" ES un usuario
```

Esta diferencia de respuestas permite a un atacante enumerar todos los emails registrados en el sistema: prueba emails de una base de datos de leaks, y los que pasan `exists` son cuentas válidas. Luego concentra el brute force solo en esos emails.

En su lugar, validamos la existencia en el Service con un mensaje de error genérico:

```
// ✔ Mensaje genérico – no revela si el email existe
throw new AuthenticationException(
    'Credenciales inválidas.',
    'invalid_credentials'
);
```

El atacante recibe exactamente el mismo error (401 + “Credenciales inválidas”) tanto si el email no existe como si la contraseña es incorrecta. No puede distinguir entre ambos casos.

2.3 `messages()` — Personalización de errores

```
public function messages(): array
{
    return [
        'email.required' => 'El correo electrónico es obligatorio.',
        'password.required' => 'La contraseña es obligatoria.',
    ];
}
```

Solo personalizamos los mensajes `required` porque son los errores más comunes en login. Los errores `string` y `email` tienen mensajes razonables por defecto. La ausencia de un mensaje para `email` no válido es intencional: Laravel ya provee “The email field must be a valid email address” que es suficientemente claro.

3. `AuthService::login()` — Lógica de Negocio

3.1 El método completo

Añade este método a la clase `app/Services/AuthService.php` **existente** (junto al método `register()` de la Parte 4):

```

public function login(array $credentials): array
{
    // Intentar autenticar con el guard JWT
    if (!$token = auth('api')->attempt($credentials)) {
        throw new \App\Exceptions\AuthenticationException(
            'Credenciales inválidas.',
            'invalid_credentials'
        );
    }

    // Obtener el usuario autenticado
    $user = auth('api')->user();

    // Verificar estado del usuario
    if ($user->status !== UserStatus::Active) {
        auth('api')->logout(); // Invalidar el token generado
        throw new \App\Exceptions\AccountInactiveException(
            match($user->status) {
                UserStatus::Inactive => 'Tu cuenta está inactiva. Contacta al administrador.',
                UserStatus::Banned   => 'Tu cuenta ha sido suspendida.',
                default                => 'Tu cuenta no está activa.',
            },
            'account_' . $user->status->value
        );
    }

    // Verificar si el usuario está soft-deleted
    if ($user->trashed()) {
        auth('api')->logout();
        throw new \App\Exceptions\AuthenticationException(
            'Credenciales inválidas.',
            'invalid_credentials'
        );
    }

    // Generar refresh token
    $refreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);

    // Revocar TODOS los refresh tokens anteriores del usuario (one session per user)
    $user->refreshTokens()->whereNull('revoked_at')->update(['revoked_at' => now()]);

    // Guardar nuevo refresh token
    $user->refreshTokens()->create([
        'token'      => $refreshToken,
        'expires_at' => now()->addMinutes(config('jwt.refresh_ttl')),
    ]);

    return [
        'user'          => $user->only(['id', 'name', 'email', 'role', 'status']),
        'access_token'  => $token,
        'refresh_token' => $refreshToken,
        'token_type'    => 'bearer',
        'expires_in'    => config('jwt.ttl') * 60,
    ];
}

```

3.2 Explicación detallada, paso a paso

Paso 1: `auth('api')->attempt($credentials)`

```

if (!$token = auth('api')->attempt($credentials)) {
    throw new \App\Exceptions\AuthenticationException(
        'Credenciales inválidas.',
        'invalid_credentials'
    );
}

```

Este es el corazón del login. `attempt()` es un método del driver JWT proporcionado por `tymon/jwt-auth`. Lo que hace internamente:

1. **Extrae las credenciales:** Toma el array `$credentials` que contiene `['email' => 'john@example.com', 'password' => 'Secure123!']`.
2. **Busca al usuario por el campo identificador:** El driver usa el provider configurado en `config/auth.php` → `providers.users` → `model => App\Models\User::class`. Eloquent ejecuta:

```
SELECT * FROM users WHERE email = 'john@example.com' LIMIT 1;
```

El campo usado para buscar es el **primer elemento del array** `$credentials`, en este caso `email`. Si pasaras `['username' => 'john', 'password' => '...']`, buscaría por `username`. Esta es la razón por la que el orden de las claves en `$credentials` importa: `email` debe ir primero.

3. **Verifica la contraseña:** Si encuentra un usuario, compara la contraseña usando `Hash::check($credentials['password'], $user->password)`. Internamente, esto usa `password_verify()` de PHP. Si el modelo tiene `SoftDeletes`, **automáticamente excluye usuarios soft-deleted** (`WHERE deleted_at IS NULL`).
4. **Si la verificación falla** (usuario no encontrado o contraseña incorrecta):
 - Retorna `false` — **NO lanza excepción**.
 - No distingue entre “email no existe” y “contraseña incorrecta” → esto es deliberado para prevenir user enumeration.
5. **Si la verificación es exitosa:**
 - Llama a `$user->getJWTIdentifier()` → obtiene `1` (el `id` del usuario).
 - Llama a `$user->getJWTCustomClaims()` → obtiene `['role' => 'user', 'name' => 'John Doe']`.
 - Construye el payload con los claims estándar (`iss`, `iat`, `exp`, `nbf`, `jti`, `sub`, `prv`) y los personalizados.
 - Firma el payload con `JWT_SECRET` usando HS256.
 - Codifica todo a Base64URL.
 - Retorna el string JWT completo.

¿Por qué `attempt()` en vez de buscar manualmente y luego `login()` ?

```
// ✗ Enfoque manual - NO usar
$user = User::where('email', $credentials['email'])->first();
if (!$user || !Hash::check($credentials['password'], $user->password)) {
    throw new AuthenticationException('Credenciales inválidas.');
}
$token = auth('api')->login($user);
```

Problemas del enfoque manual:

1. **Timing attack:** `Hash::check()` usa `password_verify()` que es timing-attack safe, pero `User::where('email', ...)->first()` no lo es. Un atacante podría medir tiempos de respuesta para inferir si el email existe (la consulta SQL tarda más si encuentra el usuario que si no). `attempt()` maneja esto internamente con comparaciones de tiempo constante.
2. **Código duplicado:** `attempt()` delega la búsqueda del usuario al provider de Laravel. Si en el futuro cambias el modelo `User` por otro mecanismo de autenticación (LDAP, OAuth), solo cambias el provider, no el código del Service.
3. **Edge cases:** `attempt()` maneja automáticamente `SoftDeletes`, `email_verified_at`, y cualquier scope global que el provider tenga configurado. El código manual tendría que replicar toda esa lógica.
4. **Consistencia:** `attempt()` usa exactamente el mismo mecanismo que `auth('api')->user()` y `auth('api')->login()`. La coherencia entre “cómo se autentica” y “cómo se resuelve el usuario después” está garantizada.

Paso 2: Obtener el usuario autenticado

```
$user = auth('api')->user();
```

Después de `attempt()` exitoso, el usuario queda “logueado” en el contexto del guard `api`. `auth('api')->user()` resuelve al modelo `User` completo desde la base de datos usando el claim `sub` del token recién generado. No genera una segunda consulta innecesaria: el paquete JWT cachea el usuario resuelto en memoria.

Paso 3: Verificar estado del usuario

```
if ($user->status !== UserStatus::Active) {  
    auth('api')->logout();  
    throw new \App\Exceptions\AccountInactiveException(  
        match($user->status) {  
            UserStatus::Inactive => 'Tu cuenta está inactiva. Contacta al administrador.',  
            UserStatus::Banned   => 'Tu cuenta ha sido suspendida.',  
            default              => 'Tu cuenta no está activa.',  
        },  
        'account_' . $user->status->value  
    );  
}
```

Las credenciales son correctas, pero el usuario NO está autorizado para usar el sistema. Diferenciamos dos casos:

Estado	Significado	Mensaje al usuario	Código HTTP	error_code
active	Cuenta normal, puede loguearse	(No aplica — el login prosigue)	—	—
inactive	Cuenta desactivada (por admin o por el usuario)	“Tu cuenta está inactiva. Contacta al administrador.”	403	account_inactive
banned	Cuenta suspendida por violación de términos	“Tu cuenta ha sido suspendida.”	403	account_banned

¿Por qué estos mensajes SÍ revelan el estado del usuario?

Porque esta es información que el usuario LEGÍTIMO necesita. Si tu cuenta fue suspendida, tienes derecho a saberlo. El user enumeration attack NO se aplica aquí porque:

1. El atacante YA proporcionó credenciales correctas (pasó `attempt()`). Ya sabe que el usuario existe.
2. Si las credenciales son incorrectas, nunca llega a esta verificación — recibe “Credenciales inválidas” antes.

Análisis del `match()`:

```
match($user->status) {  
    UserStatus::Inactive => 'Tu cuenta está inactiva. Contacta al administrador.',  
    UserStatus::Banned   => 'Tu cuenta ha sido suspendida.',  
    default              => 'Tu cuenta no está activa.',  
}
```

- `match()` es exhaustivo: si en el futuro añades un nuevo estado (`UserStatus::Pending`) y olvidas añadir un caso aquí, PHP lanza `UnhandledMatchError` en tiempo de ejecución. El `default` captura cualquier estado futuro no previsto.
- `$user->status->value` devuelve el string del enum (`"inactive"`, `"banned"`). Se concatena con `account_` para formar el `errorCode` que el frontend puede usar programáticamente.

`auth('api')->logout()` en caso de error:

El `attempt()` ya generó un access token válido. Si el usuario no está activo, ese token no debe ser usable. `auth('api')->logout()` añade el `jti` de este token a la blacklist (gracias a `blacklist_enabled: true` en `config/jwt.php`). Si alguien intentara usar este token en una petición posterior, el middleware lo rechazaría porque su `jti` está blacklistado.

Paso 4: Verificar soft-delete

```
if ($user->trashed()) {
    auth('api')->logout();
    throw new \App\Exceptions\AuthenticationException(
        'Credenciales inválidas.',
        'invalid_credentials'
    );
}
```

En teoría, `attempt()` NO debería encontrar usuarios soft-deleted porque Eloquent aplica el scope global `SoftDeletes` que añade `WHERE deleted_at IS NULL` a todas las queries. Pero en la práctica, un bug en el provider, una migración mal aplicada, o una condición de carrera podrían hacer que un usuario soft-deleted pase `attempt()`.

Este `if` es defensa en profundidad: si ocurre, el token se invalida y el atacante recibe “Credenciales inválidas” (sin revelar que el usuario existió alguna vez). El mensaje es GENÉRICO — same as wrong password — porque un usuario eliminado no debería ser identificable.

Paso 5: Generar refresh token

```
$refreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);
```

Exactamente la misma lógica que en `register()` (Parte 4, sección 6.1):

1. `setTTL(10080)` sobrescribe temporalmente el TTL a 7 días (10080 minutos).
2. `login($user)` genera un JWT con `exp = now() + 604800 segundos`.
3. El token incluye los mismos claims que el access token (`sub`, `role`, `name`, `prv`), solo cambia `exp`.

Nota sobre `login()` después de `attempt()`: `login()` crea un NUEVO token. No reutiliza el token generado por `attempt()`. Esto significa que el refresh token y el access token son tokens JWT completamente independientes — diferentes `jti`, diferentes `exp`, mismo payload de claims. Es correcto: ambos son emitidos por el mismo usuario, para la misma sesión, pero con diferentes propósitos y TTLs.

Paso 6: Revocar refresh tokens anteriores

```
$user->refreshTokens()->whereNull('revoked_at')->update(['revoked_at' => now()]);
```

Esta línea implementa la estrategia de **una sesión por usuario** (one session per user):

Estrategia	Comportamiento	Ventaja	Desventaja
Una sesión por usuario (este manual)	Cada login invalida la sesión anterior	Simple de implementar, fácil de razonar sobre seguridad	Solo puedes usar la app en un dispositivo a la vez
Múltiples sesiones	Cada login crea una nueva sesión independiente	Multi-dispositivo sin fricción	Más complejo: necesitas <code>device_name</code> , gestión de sesiones, UI de “cerrar otras sesiones”

Para este manual usamos una sesión por usuario. Si el usuario inicia sesión en su laptop y luego en su teléfono, el login en el teléfono revoca el refresh token de la laptop. Cuando la laptop intente refrescar su access token, recibirá 401 y será redirigida al login.

¿Por qué `update()` masivo y no `revoke()` individual?

`revoke()` es un método del modelo `RefreshToken` que revoca UN token:

```
$token->revoke(); // UPDATE refresh_tokens SET revoked_at = NOW() WHERE id = ?
```

`update()` masivo revoca TODOS los tokens activos de una vez:

```
$user->refreshTokens()->whereNull('revoked_at')->update(['revoked_at' => now()]);
// UPDATE refresh_tokens SET revoked_at = NOW() WHERE user_id = ? AND revoked_at IS NULL
```

Usamos `update()` masivo por eficiencia: una sola query SQL en vez de N queries si el usuario tuviera múltiples refresh tokens activos (lo cual no debería ocurrir con una sesión por usuario, pero es una protección adicional).

Paso 7: Guardar nuevo refresh token

```
$user->refreshTokens()->create([
    'token' => $refreshToken,
    'expires_at' => now()->addMinutes(config('jwt.refresh_ttl'))
]);
```

Inserta una nueva fila en `refresh_tokens` con el token JWT y su fecha de expiración. `$user->refreshTokens()` usa la relación `HasMany` definida en el modelo `User` (Parte 4), y `create()` automáticamente asigna `user_id` al valor de `$user->id`.

Paso 8: Respuesta del método

```
return [
    'user' => $user->only(['id', 'name', 'email', 'role', 'status']),
    'access_token' => $token,
    'refresh_token' => $refreshToken,
    'token_type' => 'bearer',
    'expires_in' => config('jwt.ttl') * 60,
];
```

Comparativa con la respuesta de `register()`:

Campo	register()	login()	Diferencia
user	Instancia completa de <code>User</code> (Eloquent serializa)	<code>\$user->only(...)</code> — array con campos seleccionados	<code>login()</code> es más explícito sobre qué campos se exponen
access_token	JWT 15 min	JWT 15 min	Idéntico
refresh_token	JWT 7 días	JWT 7 días	Idéntico
token_type	"bearer"	"bearer"	Idéntico
expires_in	900 (segundos)	900 (segundos)	Idéntico

Ambas respuestas tienen exactamente la misma estructura. El frontend puede tratar el resultado de `register()` y `login()` de forma intercambiable: guarda los tokens y redirige al dashboard.

4. Excepciones de Dominio

Las excepciones personalizadas son una de las herramientas más poderosas para mantener la consistencia en las respuestas de error de una API. En vez de que cada Service o Controller decida cómo formatear un error, centralizamos el formato en la excepción.

4.1 AuthenticationException

Crea el archivo `app/Exceptions/AuthenticationException.php` :

```

<?php

declare(strict_types=1);

namespace App\Exceptions;

use Exception;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class AuthenticationException extends Exception
{
    public function __construct(
        string $message = 'No autenticado.',
        private readonly string $errorCode = 'unauthenticated',
        int $httpCode = 401
    ) {
        parent::__construct($message, $httpCode);
    }

    public function render(Request $request): JsonResponse
    {
        return response()->json([
            'error' => [
                'code' => $this->errorCode,
                'message' => $this->getMessage(),
            ],
        ], $this->getStatusCode() ? 401);
    }

    public function getErrorCode(): string
    {
        return $this->errorCode;
    }
}

```

4.2 AccountInactiveException

Crea el archivo `app/Exceptions/AccountInactiveException.php` :

```

<?php

declare(strict_types=1);

namespace App\Exceptions;

use Exception;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class AccountInactiveException extends Exception
{
    public function __construct(
        string $message = 'Tu cuenta no está activa.',
        private readonly string $errorCode = 'account_inactive',
        int $statusCode = 403
    ) {
        parent::__construct($message, $statusCode);
    }

    public function render(Request $request): JsonResponse
    {
        return response()->json([
            'error' => [
                'code' => $this->errorCode,
                'message' => $this->getMessage(),
            ],
            $this->getStatusCode() ? 403:
        ], $this->getStatusCode() ? 403:

    public function getErrorCode(): string
    {
        return $this->errorCode;
    }
}

```

4.3 Explicación del diseño

¿Por qué excepciones de dominio y no genéricas?

Comparación de enfoques:

```

// ✗ Enfoque con excepción genérica – inconsistente
throw new \Exception('Credenciales inválidas');
// → El controller tiene que catchear y decidir el código HTTP y el formato JSON
// → Cada controller podría formatear el error de forma diferente
// → El mensaje de error no está asociado a un código de error programático

// ✓ Enfoque con excepción de dominio – consistente y semántico
throw new AuthenticationException('Credenciales inválidas.', 'invalid_credentials');
// → La excepción SABE cómo renderizarse a JSON
// → El código HTTP (401) es parte de la excepción
// → El errorCode permite al frontend manejar casos específicos

```

El método `render()`

```

public function render(Request $request): JsonResponse
{
    return response()->json([
        'error' => [
            'code' => $this->errorCode,
            'message' => $this->getMessage(),
        ],
        $this->getStatusCode() ? 401:
    ], $this->getStatusCode() ? 401:
}

```

Laravel llama automáticamente a `render()` cuando una excepción NO es atrapada con `try/catch`. El flujo es:

1. El Service lanza `throw new AuthenticationException(...)`.
2. La excepción sube por la pila de llamadas: Service → Controller → Middleware → Exception Handler de Laravel.
3. Laravel verifica si la excepción tiene método `render()`.
4. Si lo tiene, lo llama y devuelve su resultado como respuesta HTTP.
5. Si no lo tiene, usa el handler por defecto (HTML en web, JSON en API).

No necesitas `try/catch` en el controller. La excepción “sabe” cómo presentarse al cliente.

Códigos HTTP semánticos

Excepción	Código HTTP	Significado
<code>AuthenticationException</code>	401 Unauthorized	“No sé quién eres” o “tus credenciales son incorrectas”. Es un problema de AUTENTICACIÓN.
<code>AccountInactiveException</code>	403 Forbidden	“Sé quién eres (las credenciales son correctas) pero NO tienes permiso para acceder (cuenta inactiva/baneada)”. Es un problema de AUTORIZACIÓN.

Esta distinción entre 401 y 403 es sutil pero importante:

- **401:** No estás autenticado. La solución es proporcionar credenciales válidas.
- **403:** Estás autenticado pero no autorizado. La solución NO es cambiar credenciales, es resolver el estado de tu cuenta.

El campo `errorCode`

```
private readonly string $errorCode = 'unauthenticated';
```

Cada excepción lleva un código de error programático (`error_code` en el JSON de respuesta) que permite al frontend manejar casos específicos con lógica condicional:

```
// Frontend (React/Vue) – pseudocódigo
try {
  const response = await api.post('/login', credentials);
  // éxito
} catch (error) {
  switch (error.response.data.error.code) {
    case 'invalid_credentials':
      // Mostrar mensaje genérico: "Email o contraseña incorrectos"
      break;
    case 'account_inactive':
      // Redirigir a página de "cuenta inactiva – contacta al admin"
      break;
    case 'account_banned':
      // Redirigir a página de "cuenta suspendida"
      break;
    case 'token_expired':
      // Intentar refresh automáticamente
      break;
    case 'token_missing':
      // Redirigir al login
      break;
  }
}
```

Sin `errorCode`, el frontend tendría que hacer string matching sobre el `message`, lo cual es frágil (cambios de texto rompen la lógica) y problemático con i18n (el mensaje cambia según el idioma).

5. LoginController — Punto de Entrada HTTP

Crea el archivo `app/Http/Controllers/Auth/LoginController.php`:

```
<?php

declare(strict_types=1);

namespace App\Http\Controllers\Auth;

use App\Http\Requests\Auth\LoginRequest;
use App\Services\AuthService;
use Illuminate\Http\JsonResponse;

final class LoginController
{
    public function __invoke(LoginRequest $request, AuthService $authService): JsonResponse
    {
        $result = $authService->login($request->validated());

        return response()->json([
            'data' => $result,
        ]);
    }
}
```

5.1 Explicación

Single-action controller (`__invoke`)

```
final class LoginController
{
    public function __invoke(LoginRequest $request, AuthService $authService): JsonResponse
```

Mismo patrón que `RegisterController` (Parte 4). Una clase con una sola responsabilidad: manejar la petición HTTP de login. No hay métodos `store()`, `create()`, `edit()` como en un resource controller tradicional. `Route::post('/login', LoginController::class)` llama automáticamente a `__invoke`.

¿Por qué `200 OK` y no `201 Created`?

Endpoint	Código HTTP	Razón
<code>POST /register</code>	<code>201 Created</code>	Se crea un recurso nuevo (un usuario).
<code>POST /login</code>	<code>200 OK</code>	No se crea un recurso nuevo. Se crea una sesión/token , y las sesiones no son recursos RESTful en el sentido tradicional.
<code>201 Created</code> implica “aquí tienes la URL del nuevo recurso” (header <code>Location</code>). Un token no tiene URL canónica. <code>200</code>		

OK es semánticamente correcto: la petición fue exitosa, aquí está el resultado.

```
$request->validated()
```

```
$result = $authService->login($request->validated());
```

validated() retorna ['email' => 'john@example.com', 'password' => 'Secure123!']. Solo los campos definidos en rules() del LoginRequest. Si el cliente enviara campos adicionales (remember_me, device_name), serían ignorados — no llegan al Service.

Respuesta del controller

```
return response()->json([
    'data' => $result,
]);
```

El controller tiene 3 líneas de lógica. No hay if, no hay try/catch, no hay formateo manual. La excepción de autenticación se lanza desde el Service y Laravel la captura automáticamente llamando a render().

Diferencias clave con RegisterController :

Aspecto	RegisterController	LoginController
Código HTTP	201 Created	200 OK
Service llamado	\$authService->register()	\$authService->login()
Request inyectado	RegisterRequest	LoginRequest
Respuesta	Misma estructura { data: { user, access_token, refresh_token, ... } }	Misma estructura
Excepciones posibles	Errores de validación (422) manejados por FormRequest, errores de BD (500) manejados por Laravel	AuthenticationException (401), AccountInactiveException (403), errores de validación (422)

6. Ruta de Login y Grupo de Rutas Públicas

6.1 Agrupación de rutas públicas

Edita routes/api.php para agrupar las rutas de autenticación pública:

```
<?php

use App\Http\Controllers\Auth\LoginController;
use App\Http\Controllers\Auth\RegisterController;
use Illuminate\Support\Facades\Route;

Route::prefix('auth')->group(function () {
    Route::post('/register', RegisterController::class)
        ->name('auth.register')
        ->middleware('throttle:10,1');

    Route::post('/login', LoginController::class)
        ->name('auth.login')
        ->middleware('throttle:5,1');
});
```

6.2 Explicación

Prefijo `auth`

```
Route::prefix('auth')->group(function () {
    // ...
});
```

Laravel carga `routes/api.php` con el prefijo `/api` (definido en `bootstrap/app.php` en Laravel 11). Con el prefijo `auth`, las rutas completas quedan:

Ruta	Nombre	Middleware
POST <code>/api/auth/register</code>	<code>auth.register</code>	<code>throttle:10,1</code>
POST <code>/api/auth/login</code>	<code>auth.login</code>	<code>throttle:5,1</code>

El prefijo `auth` comunica claramente que estas rutas pertenecen al dominio de autenticación. Si en el futuro añadieras `POST /api/auth/forgot-password` o `POST /api/auth/reset-password`, encajarían naturalmente en este grupo.

Rate limiting en login: `throttle:5,1`

```
->middleware('throttle:5,1');
```

Endpoint	Rate Limit	Razón
Registro	<code>throttle:10,1</code> (10/min)	El registro es más permisivo porque: a) un usuario legítimo normalmente se registra UNA vez, b) los bots de registro necesitan muchas cuentas rápidamente.
Login	<code>throttle:5,1</code> (5/min)	Más restrictivo porque: a) el login es el vector principal de brute force, b) un usuario legítimo puede equivocarse 2-3 veces, pero 5 intentos en un minuto es sospechoso, c) un atacante probando contraseñas de un diccionario hará cientos de intentos.

5 intentos por minuto son suficientes para un usuario real que:

1. Escribe mal su contraseña 2-3 veces (teclado en otro idioma, mayúsculas).
2. La recuerda y la escribe correctamente al 4º intento.

No son suficientes para un atacante que está iterando sobre un diccionario de 10,000 contraseñas comunes (tomaría 2,000 minutos = 33 horas para probar todas contra un solo email).

Si el límite se excede, Laravel responde:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
Content-Type: application/json

{
  "message": "Too Many Requests"
}
```

Naming

```
->name('auth.login');
```

Nombrar las rutas permite referenciarlas simbólicamente:

```
// En tests:
$response = $this->postJson(route('auth.login'), $credentials);

// En generar URLs desde cualquier parte del código:
$loginUrl = route('auth.login'); // "http://localhost:8000/api/auth/login"
```

Si algún día cambias `/login` a `/signin`, solo modificas la ruta. Todas las referencias por nombre siguen funcionando.

7. Middleware `JwtAuthenticate` — Protección de Rutas

El middleware es la pieza que protege las rutas: cada petición a un endpoint protegido pasa por aquí, y si el token no es válido, la petición se rechaza antes de llegar al controller.

7.1 Crear el middleware

```
php artisan make:middleware JwtAuthenticate
```

Sobrescribe el contenido generado en `app/Http/Middleware/JwtAuthenticate.php` :

```

<?php

declare(strict_types=1);

namespace App\Http\Middleware;

use App\Exceptions\AuthenticationException;
use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class JwtAuthenticate
{
    public function handle(Request $request, Closure $next): Response
    {
        try {
            // Intenta autenticar con el guard JWT
            if (!$user = auth('api')->user()) {
                throw new AuthenticationException(
                    'Token no proporcionado o inválido.',
                    'token_missing_or_invalid'
                );
            }

            // Verificar que el usuario sigue estando activo
            // (podría haber sido baneado después de emitir el token)
            if ($user->status !== \App\Enums\UserStatus::Active) {
                auth('api')->logout(); // Invalidar el token
                throw new AuthenticationException(
                    'Tu cuenta ya no está activa.',
                    'account_inactive',
                    403
                );
            }

            // Verificar soft-delete
            if ($user->trashed()) {
                auth('api')->logout();
                throw new AuthenticationException(
                    'Token inválido.',
                    'token_invalid'
                );
            }
        } catch (\Tymon\JWTAuth\Exceptions\TokenExpiredException $e) {
            throw new AuthenticationException(
                'El token ha expirado.',
                'token_expired'
            );
        } catch (\Tymon\JWTAuth\Exceptions\TokenInvalidException $e) {
            throw new AuthenticationException(
                'El token no es válido.',
                'token_invalid'
            );
        } catch (\Tymon\JWTAuth\Exceptions\JWTException $e) {
            throw new AuthenticationException(
                'Token no proporcionado.',
                'token_missing'
            );
        }

        return $next($request);
    }
}

```

7.2 Explicación detallada del flujo

Cada petición a una ruta protegida con `jwt.auth` atraviesa este middleware. El flujo es:

```

Request → auth('api')->user() → ¿token válido?
├── Sí → ¿usuario activo? → ¿no soft-deleted?
│   ├── Sí → next($request) → Controller
│   └── No → 403/401
└── No → ¿por qué falló?
    ├── TokenExpiredException → 401 token_expired
    ├── TokenInvalidException → 401 token_invalid
    └── JWTException → 401 token_missing

```

Paso 1: `auth('api')->user()`

```

if (!$user = auth('api')->user()) {
    throw new AuthenticationException(
        'Token no proporcionado o inválido.',
        'token_missing_or_invalid'
    );
}

```

`auth('api')->user()` le pide al guard JWT que resuelva al usuario autenticado. El driver JWT hace lo siguiente:

1. Busca el token en el header `Authorization: Bearer <token>`.
2. Si no hay header `Authorization`, lanza `JWTException` → capturado en el `catch` inferior.
3. Si hay header, decodifica el token y verifica la firma con `JWT_SECRET`.
4. Si la firma es inválida o el token está malformado, lanza `TokenInvalidException`.
5. Si el token es válido pero el claim `exp` ya pasó, lanza `TokenExpiredException`.
6. Si el `jti` del token está en la blacklist, lanza `TokenInvalidException` (porque `TokenBlacklistedException` es subclase de `TokenInvalidException`).
7. Si todo es correcto, extrae el claim `sub`, busca `User::find($sub)`, y retorna el modelo.

Este `if (!$user = ...)` es un fallback para cualquier caso donde `user()` retorne `null` sin lanzar excepción — extremadamente raro, pero defensivo.

Paso 2: Verificar estado del usuario

```

if ($user->status !== \App\Enums\UserStatus::Active) {
    auth('api')->logout();
    throw new AuthenticationException(
        'Tu cuenta ya no está activa.',
        'account_inactive',
        403
    );
}

```

Esta verificación es crítica. ¿Por qué verificar el estado AQUÍ si ya lo verificamos en `login()`?

Momento	Usuario activo	Token emitido	Token TTL
t=0 (login)	✓ Active	Access token emitido	Expira en t=15min
t=5 (admin banea al usuario)	✗ Banned	Token SIGUE siendo válido criptográficamente	Expira en t=15min
t=10 (request del usuario baneado)	✗ Banned	Token aún no expirado → si no verificamos estado aquí, PASARÍA	—

Sin esta verificación en el middleware, un usuario baneado tendría acceso durante el tiempo restante de su access

token (hasta 15 minutos). Con esta verificación, el acceso se corta inmediatamente (en el siguiente request).

Trade-off: una query extra a la base de datos por request autenticado. `auth('api')->user()` YA hizo una query para resolver al usuario (`User::find($sub)`). El modelo ya está cargado en memoria, así que `$user->status` NO genera una query adicional — es una lectura de un atributo del modelo ya hidratado.

Paso 3: Verificar soft-delete

```
if ($user->trashed()) {  
    auth('api')->logout();  
    throw new AuthenticationException(  
        'Token inválido.',  
        'token_invalid'  
    );  
}
```

Similar al caso de estado: un usuario podría ser soft-deleted después de que su token fue emitido. `trashed()` verifica `deleted_at !== null`.

Mensaje genérico (“Token inválido”) — no revelamos que el usuario fue eliminado.

Paso 4: Captura de excepciones del paquete JWT

```
} catch (\ Tymon\JWTAuth\Exceptions\TokenExpiredException $e) {  
    throw new AuthenticationException(  
        'El token ha expirado.',  
        'token_expired'  
    );  
}  
} catch (\ Tymon\JWTAuth\Exceptions\TokenInvalidException $e) {  
    throw new AuthenticationException(  
        'El token no es válido.',  
        'token_invalid'  
    );  
}  
} catch (\ Tymon\JWTAuth\Exceptions\JWTException $e) {  
    throw new AuthenticationException(  
        'Token no proporcionado.',  
        'token_missing'  
    );  
}
```

Estas excepciones las lanza `auth('api')->user()` internamente cuando el token no es válido. Las capturamos y las convertimos a nuestras `AuthenticationException` de dominio con `errorCode` semántico:

Excepción del paquete	Causa	Nuestra excepción	errorCode
<code>TokenExpiredException</code>	<code>exp</code> del token ya pasó	<code>AuthenticationException</code>	<code>token_expired</code>
<code>TokenInvalidException</code>	Firma inválida, token malformado, o <code>jti</code> en blacklist	<code>AuthenticationException</code>	<code>token_invalid</code>
<code>JWTException</code>	No hay header <code>Authorization</code> , o está vacío	<code>AuthenticationException</code>	<code>token_missing</code>

¿Por qué capturar aquí y no dejar que suban?

Por consistencia en el formato de error. Si no capturáramos estas excepciones, el paquete JWT devolvería su propio formato de error, que es diferente al nuestro ({ "error": { "code": "...", "message": "..." } }). Las capturamos y las convertimos para mantener la consistencia.

Jerarquía de excepciones del paquete:

```
JWTException (base)
├── TokenExpiredException
├── TokenInvalidException
├── TokenBlacklistedException
└── PayloadException
```

El orden de los `catch` importa: `TokenExpiredException` y `TokenInvalidException` extienden `JWTException`, por lo que deben ir ANTES del `catch (JWTException)`. Si el orden fuera inverso, `JWTException` capturaría todas las subclases y nunca llegaríamos a los catches específicos.

Paso 5: Pasar al siguiente middleware/controller

```
return $next($request);
```

Si todo es válido, la petición continúa al siguiente middleware en la cadena (o al controller si este es el último middleware). `$next($request)` es una closure que representa el resto de la cadena de middleware.

7.3 Registrar el middleware en el Kernel

En Laravel 11, los middleware se registran en `bootstrap/app.php`:

```
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__ . '/../routes/web.php',
        api: __DIR__ . '/../routes/api.php',
        commands: __DIR__ . '/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        $middleware->alias([
            'jwt.auth' => \App\Http\Middleware\JwtAuthenticate::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions) {
        //
    })->create();
```

El método `alias()` registra un nombre corto (`jwt.auth`) para la clase del middleware. Esto permite usar `->middleware('jwt.auth')` en las rutas en vez de `->middleware(\App\Http\Middleware\JwtAuthenticate::class)`.

7.4 ¿Por qué NO usar el middleware incluido en `tymon/jwt-auth`?

El paquete incluye su propio middleware (`Tymon\JWTAuth\Http\Middleware\Authenticate`), pero NO lo usamos. Razones:

Aspecto	Middleware del paquete	Nuestro middleware
Formato de error	Formato propio del paquete	Nuestro formato consistente <code>{ error: { code, message } }</code>
Verificación de estado	✗ No verifica <code>UserStatus</code>	✓ Verifica que el usuario siga activo
Verificación de soft-delete	✗ No verifica	✓ Verifica <code>trashed()</code>
Control de mensajes	Mensajes en inglés hardcodedos	Mensajes en español personalizables
<code>errorCode</code>	✗ No proporciona	✓ Cada error tiene un código programático
Extensibilidad	Difícil de extender sin heredar	Código propio — control total

Crear nuestro propio middleware son ~70 líneas y nos da control total sobre el comportamiento, mensajes, y formato de error.

8. Rutas Protegidas

8.1 Agrupación bajo `jwt.auth`

Añade en `routes/api.php`, debajo del grupo `auth`:

```
use App\Http\Controllers\Auth\LogoutController;
use App\Http\Controllers\Auth\RefreshTokenController;
use App\Http\Controllers\UserController;

Route::middleware('jwt.auth')->group(function () {
    // Perfil del usuario autenticado
    Route::get('/me', [UserController::class, 'me'])->name('user.me');

    // Refresh de tokens
    Route::post('/auth/refresh', RefreshTokenController::class)->name('auth.refresh');

    // Logout
    Route::post('/auth/logout', LogoutController::class)->name('auth.logout');
});
```

8.2 Explicación

Agrupación por middleware

```
Route::middleware('jwt.auth')->group(function () {
    // Todas las rutas aquí requieren token JWT válido
});
```

DRY: en vez de repetir `->middleware('jwt.auth')` en cada ruta, lo declaramos una vez para el grupo. Cualquier ruta añadida dentro del grupo hereda automáticamente la protección JWT.

Orden de las rutas en `api.php`

```
// 1. Rutas públicas (sin middleware)
Route::prefix('auth')->group(function () {
    Route::post('/register', ...);
    Route::post('/login', ...);
});

// 2. Rutas protegidas (con middleware jwt.auth)
Route::middleware('jwt.auth')->group(function () {
    Route::get('/me', ...);
    Route::post('/auth/refresh', ...);
    Route::post('/auth/logout', ...);
});
```

El orden importa porque Laravel evalúa las rutas en el orden en que están definidas. Si una ruta pública y una protegida coincidieran (no es el caso aquí), la primera que haga match gana.

`/me` — Usuario autenticado

```
Route::get('/me', [UserController::class, 'me'])->name('user.me');
```

Endpoint fundamental para SPAs: después de login/registro, el frontend necesita los datos del usuario autenticado para mostrar “Bienvenido, John” y decidir qué rutas/componentes mostrar según el rol.

`/me` es un estándar de facto en APIs RESTful para “dame los datos del usuario cuyo token estoy enviando”. Alternativas: `/user`, `/profile`, `/auth/user`. `/me` es semánticamente claro: “yo” = el usuario autenticado.

`/auth/refresh` y `/auth/logout` bajo `jwt.auth`

```
Route::post('/auth/refresh', RefreshTokenController::class)->name('auth.refresh');
Route::post('/auth/logout', LogoutController::class)->name('auth.logout');
```

Estas rutas están bajo el prefijo `auth` pero DENTRO del grupo protegido. La ruta completa es:

Método	Ruta completa	Nombre	¿Requiere token?
GET	<code>/api/me</code>	<code>user.me</code>	✓ Sí
POST	<code>/api/auth/refresh</code>	<code>auth.refresh</code>	✓ Sí
POST	<code>/api/auth/logout</code>	<code>auth.logout</code>	✓ Sí

Wait — ¿refresh requiere token? La respuesta es **sí, requiere access token**. Esto puede parecer contradictorio (si el access token expiró, ¿cómo pasas el middleware JWT?), y es una excelente pregunta.

Excepción para refresh en el middleware:

Para manejar este caso correctamente, el middleware `JwtAuthenticate` DEBE permitir tokens expirados en la ruta de refresh. Añade esta condición al inicio del método `handle()`:

```

public function handle(Request $request, Closure $next): Response
{
    $isRefreshRoute = $request->is('api/auth/refresh');

    try {
        if (!$user = auth('api')->user()) {
            throw new AuthenticationException(
                'Token no proporcionado o inválido.',
                'token_missing_or_invalid'
            );
        }

        // ... resto de verificaciones ...

    } catch (\Symfony\Component\Security\Core\Exception\TokenExpiredException $e) {
        // Permitir tokens expirados SOLO en la ruta de refresh
        if ($isRefreshRoute) {
            return $next($request);
        }

        throw new AuthenticationException(
            'El token ha expirado.',
            'token_expired'
        );
    } catch (\Symfony\Component\Security\Core\Exception\TokenInvalidException $e) {
        throw new AuthenticationException(
            'El token no es válido.',
            'token_invalid'
        );
    } catch (\Symfony\Component\Security\Core\Exception\JWTException $e) {
        throw new AuthenticationException(
            'Token no proporcionado.',
            'token_missing'
        );
    }

    return $next($request);
}

```

La lógica es: si el token expiró PERO estamos en la ruta `/api/auth/refresh`, permitimos que la petición continúe. El `RefreshTokenController` usará el refresh token (que viene en el body, no en el header) para validar la sesión.

¿Por qué no hacer la ruta de refresh pública?

Si la ruta de refresh fuera pública (sin middleware JWT), cualquiera podría enviar refresh tokens sin siquiera tener un access token. Aunque el refresh token está protegido (es un JWT firmado y se valida contra la DB), añadir el access token como capa adicional:

1. Permite identificar al usuario ANTES de validar el refresh token (el claim `sub` del access token expirado sigue siendo legible).
2. El `sub` del access token debe coincidir con el `sub` del refresh token — el middleware fuerza esta consistencia.
3. Añade una capa adicional de defensa: el atacante necesita DOS tokens (access + refresh) en vez de uno solo.

Si prefieres mantener la ruta de refresh pública, omite `$isRefreshRoute` y coloca `POST /auth/refresh` en el grupo de rutas públicas. Ambas opciones son válidas. Este manual usa la ruta protegida con excepción para tokens expirados.

9. `AuthService::refreshTokens()` — Rotación de Tokens con Detección de Reuse Attack

9.1 El método completo

Añade este método a `app/Services/AuthService.php`:

```

use App\Models\RefreshToken;

public function refreshTokens(string $refreshToken): array
{
    // Buscar refresh token en DB
    $storedToken = RefreshToken::where('token', $refreshToken)->first();

    // Validar existencia, expiración y revocación
    if (!$storedToken || $storedToken->isExpired() || $storedToken->isRevoked()) {
        // Si el refresh token está revocado pero no expirado → posible reuse attack
        if ($storedToken && $storedToken->isRevoked() && !$storedToken->isExpired()) {
            // Revocar TODOS los refresh tokens del usuario (compromiso de seguridad)
            $storedToken->user->refreshTokens()
                ->whereNull('revoked_at')
                ->update(['revoked_at' => now()]);

            // Invalidar access token actual
            auth('api')->invalidate(true); // true = forever (no grace period)
        }

        throw new AuthenticationException(
            'Refresh token inválido o expirado.',
            'invalid_refresh_token'
        );
    }

    $user = $storedToken->user;

    // Verificar estado del usuario
    if ($user->status !== UserStatus::Active || $user->trashed()) {
        $storedToken->revoke();
        throw new AccountInactiveException(
            'Tu cuenta no está activa.',
            'account_inactive'
        );
    }

    // Revocar refresh token actual (rotación)
    $storedToken->revoke();

    // Emitir nuevos tokens
    $newAccessToken = auth('api')->login($user);
    $newRefreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);

    // Guardar nuevo refresh token
    $user->refreshTokens()->create([
        'token' => $newRefreshToken,
        'expires_at' => now()->addMinutes(config('jwt.refresh_ttl')),
    ]);

    return [
        'access_token' => $newAccessToken,
        'refresh_token' => $newRefreshToken,
        'token_type' => 'bearer',
        'expires_in' => config('jwt.ttl') * 60,
    ];
}

```

9.2 Explicación detallada de cada paso

Paso 1: Buscar el refresh token en la base de datos

```
$storedToken = RefreshToken::where('token', $refreshToken)->first();
```

Buscamos el refresh token en la tabla `refresh_tokens` por su valor JWT completo. El índice `idx_refresh_tokens_token` (creado en la Parte 4) hace que esta consulta sea $O(\log n)$ incluso con millones de tokens.

¿Por qué buscar en la DB y no solo validar la firma JWT?

El paquete `tymon/jwt-auth` puede validar la firma del refresh token (es un JWT como cualquier otro), pero necesitamos la DB para saber si fue REVOCADO. La revocación es una operación de base de datos — no hay forma de saber desde el token mismo si fue revocado. El JWT dice “soy válido hasta el día X”, pero no dice “fui revocado ayer”.

Paso 2: Validar existencia, expiración y revocación

```
if (!$storedToken || $storedToken->isExpired() || $storedToken->isRevoked()) {  
    // ...  
}
```

Usamos los métodos de dominio del modelo `RefreshToken` (Parte 4):

Método	Qué verifica	SQL equivalente
<code>!\$storedToken</code>	¿Existe el token en la DB?	<code>SELECT ... WHERE token = ?</code> no encontró nada
<code>\$storedToken->isExpired()</code>	¿ <code>expires_at</code> ya pasó?	<code>WHERE expires_at < NOW()</code> → devuelve <code>true</code> si pasó
<code>\$storedToken->isRevoked()</code>	¿ <code>revoked_at</code> no es null?	<code>WHERE revoked_at IS NOT NULL</code> → <code>true</code> si fue revocado

Si cualquiera de estas condiciones es verdadera, el refresh token NO es válido. Pero antes de lanzar la excepción, verificamos si es un ataque de reuse.

Paso 3: Detección de Reuse Attack

```
if ($storedToken && $storedToken->isRevoked() && !$storedToken->isExpired()) {  
    // REUSE ATTACK DETECTADO  
    $storedToken->user->refreshTokens()  
        ->whereNull('revoked_at')  
        ->update(['revoked_at' => now()]);  
  
    auth('api')->invalidate(true);  
}
```

Esta es una de las protecciones más importantes de la rotación de refresh tokens. El razonamiento:

Escenario normal: 1. El usuario legítimo usa su refresh token → el servidor lo revoca y emite uno nuevo. 2. El usuario legítimo usa el NUEVO refresh token → todo bien.

Escenario de ataque: 1. Un atacante roba el refresh token del usuario legítimo (MITM, XSS, leak). 2. El usuario legítimo usa su refresh token → el servidor lo revoca y emite uno nuevo. 3. El atacante intenta usar el refresh token ROBADO (el mismo que el usuario ya usó en el paso 2).

En el paso 3, `$storedToken` EXISTE en la DB pero `isRevoked()` es `true` (el usuario legítimo ya lo usó en el paso 2) y `isExpired()` es `false` (todavía está dentro de los 7 días). Esta combinación — **revocado pero no expirado** — es la firma de un ataque de reuse:

Estado del token	¿Existe?	¿Revocado?	¿Expirado?	Interpretación
Nunca emitido	✗ No	—	—	Token inválido — no hay ataque
Usado normalmente, ya rotado	✓ Sí	✓ Sí	✓ Sí	Token viejo que expiró naturalmente — no hay ataque
Usado, revocado, NO expirado	✓ Sí	✓ Sí	✗ No	POSIBLE ATAQUE DE REUSE
Activo	✓ Sí	✗ No	✗ No	Token válido — refresh normal

¿Qué hacemos cuando detectamos reuse?

1. Revocamos TODOS los refresh tokens del usuario:

```
$storedToken->user->refreshTokens()
->whereNull('revoked_at')
->update(['revoked_at' => now()]);
```

Esto incluye el refresh token que el usuario legítimo recibió en el paso 2. El usuario legítimo es forzado a volver a loguearse. Es una fricción, pero preferible a que el atacante mantenga acceso.

2. Invalidamos el access token actual:

```
auth('api')->invalidate(true); // true = forever (no grace period)
```

`invalidate(true)` añade el `jti` del access token actual a la blacklist con TTL igual al `exp` del token (es decir, permanece blacklisted hasta que habría expirado naturalmente). El parámetro `true` significa “forever” — ignora el `blacklist_grace_period` y rechaza inmediatamente.

¿Qué gana el atacante? NADA.

- El refresh token robado es rechazado (revocado).
- Incluso si el atacante tuviera un access token válido, también es invalidado.
- El usuario legítimo es desconectado, pero puede volver a loguearse con email/password.

¿Qué gana el usuario legítimo? SABER que algo pasó.

La próxima vez que intente usar la app, su refresh token será rechazado → es redirigido al login. Idealmente, en el login se le mostraría un mensaje: “Tu sesión fue cerrada por seguridad. Si no fuiste tú, cambia tu contraseña.” (Esto requiere lógica adicional en el frontend, no cubierta aquí.)

Paso 4: Verificar estado del usuario

```
if ($user->status !== UserStatus::Active || $user->trashed()) {
    $storedToken->revoke();
    throw new AccountInactiveException(
        'Tu cuenta no está activa.',
        'account_inactive'
    );
}
```

Misma verificación que en `login()`, pero en el contexto de refresh. Si el usuario fue baneado después del último

refresh, no debe poder seguir refrescando tokens.

Paso 5: Revocar el refresh token actual

```
$storedToken->revoke();
```

Rotación: el refresh token usado para esta petición es revocado inmediatamente. No puede usarse una segunda vez. Esta es la esencia de la rotación:

```
Refresh Token A → usado → revocado → Refresh Token B emitido
Refresh Token B → usado → revocado → Refresh Token C emitido
Refresh Token A → usado de nuevo → REUSE ATTACK DETECTADO
```

Paso 6: Emitir nuevos tokens

```
$newAccessToken = auth('api')->login($user);
$newRefreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);
```

Misma lógica que en `login()` y `register()`: access token de 15 minutos, refresh token de 7 días.

Paso 7: Guardar nuevo refresh token

```
$user->refreshTokens()->create([
    'token'      => $newRefreshToken,
    'expires_at' => now()->addMinutes(config('jwt.refresh_ttl'))
]);
```

El nuevo refresh token se persiste en la base de datos. Ahora es el ÚNICO refresh token activo para este usuario (una sesión por usuario).

Paso 8: Respuesta del método

```
return [
    'access_token' => $newAccessToken,
    'refresh_token' => $newRefreshToken,
    'token_type'   => 'bearer',
    'expires_in'   => config('jwt.ttl') * 60,
];
```

Comparativa con la respuesta de `login()`:

Campo	login()	refreshTokens()	Diferencia
<code>user</code>	✓ Incluido	✗ No incluido	El refresh no necesita devolver el usuario — el frontend ya lo tiene
<code>access_token</code>	✓	✓	Mismo formato
<code>refresh_token</code>	✓	✓	Mismo formato
<code>token_type</code>	✓	✓	Mismo formato
<code>expires_in</code>	✓	✓	Mismo formato

La respuesta de `refreshTokens()` NO incluye `user` porque el frontend ya tiene los datos del usuario (los obtuvo en el login/registro). Si los necesita actualizados, puede llamar a `GET /me`.

10. `AuthService::logout()` — Cierre de Sesión

10.1 El método completo

Añade este método a `app/Services/AuthService.php`:

```
public function logout(): void
{
    $user = auth('api')->user();

    if ($user) {
        // Revocar todos los refresh tokens del usuario
        $user->refreshTokens()
            ->whereNull('revoked_at')
            ->update(['revoked_at' => now()]);

        // Invalidar el access token actual (blacklist)
        auth('api')->logout();
    }
}
```

10.2 Explicación

¿Por qué dos operaciones de invalidación?

El logout debe cerrar la sesión COMPLETAMENTE. Esto requiere dos acciones:

1. **Revocar refresh tokens:** Sin esto, el usuario podría usar su refresh token para obtener un nuevo access token incluso después de hacer logout. El refresh token seguiría siendo válido criptográficamente y no está blacklistado (la blacklist es solo para access tokens).

```
$user->refreshTokens()
    ->whereNull('revoked_at')
    ->update(['revoked_at' => now()]);
```

2. **Invalidar el access token actual:** Sin esto, el access token seguiría siendo válido por el tiempo que le quede de TTL (hasta 15 minutos). Aunque el usuario ya no puede refrescar, podría seguir usando la API durante el resto de la ventana del access token.

```
auth('api')->logout();
```

`auth('api')->logout()` añade el `jti` del access token actual a la blacklist. Si alguien intenta usar este token en una petición posterior, el middleware `JwtAuthenticate` lo rechazará (porque el `jti` está en la blacklist).

Idempotencia

```
if ($user) {
    // ...
}
```

Llamar `logout()` dos veces no causa errores: - La segunda vez, `$user->refreshTokens()->whereNull('revoked_at')` no

encuentra tokens → `update()` afecta 0 filas. - La segunda vez, `auth('api')->logout()` intenta blacklistear un token que ya está blacklistedo → el paquete ignora duplicados.

¿Por qué no borrar el refresh token de la DB?

Usamos `revoked_at` (soft-revoke) en vez de `DELETE` (hard-delete) por las mismas razones que en el diseño original de la tabla (Parte 4):

1. **Auditoría forense:** ¿El usuario cerró sesión voluntariamente o fue un cierre forzoso por ataque de reuse? `revoked_at` lo registra.
2. **Debugging:** Si un usuario reporta problemas de sesión, puedes consultar el historial de sus refresh tokens.
3. **Consistencia:** El modelo `RefreshToken` usa soft-revoke en todo el sistema. Mezclar hard-delete en logout y soft-revoke en refresh sería inconsistente.

11. RefreshTokenRequest — Validación del Refresh Token

Crea el archivo `app/Http/Requests/Auth/RefreshTokenRequest.php` :

```
<?php

declare(strict_types=1);

namespace App\Http\Requests\Auth;

use Illuminate\Foundation\Http\FormRequest;

class RefreshTokenRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'refresh_token' => ['required', 'string']
        ];
    }

    public function messages(): array
    {
        return [
            'refresh_token.required' => 'El refresh token es obligatorio.',
        ];
    }
}
```

11.1 Explicación

La validación es mínima: el campo `refresh_token` debe estar presente y ser un string. No validamos el formato JWT aquí porque:

1. El formato JWT se valida en el Service cuando `auth('api')` intenta decodificarlo.
 2. Si el token está malformado, el Service lanzará `AuthenticationException` con `invalid_refresh_token`.
 3. Validar formato JWT en el FormRequest requeriría lógica de decodificación duplicada aquí, rompiendo el principio de single responsibility.
-

12. RefreshTokenController y LogoutController

12.1 RefreshTokenController

Crea el archivo `app/Http/Controllers/Auth/RefreshTokenController.php` :

```
<?php

declare(strict_types=1);

namespace App\Http\Controllers\Auth;

use App\Http\Requests\Auth\RefreshTokenRequest;
use App\Services\AuthService;
use Illuminate\Http\JsonResponse;

final class RefreshTokenController
{
    public function __invoke(RefreshTokenRequest $request, AuthService $authService): JsonResponse
    {
        $result = $authService->refreshTokens($request->validated('refresh_token'));

        return response()->json([
            'data' => $result,
        ]);
    }
}
```

Nota: `$request->validated('refresh_token')` extrae solo el valor del campo `refresh_token` del array validado. Es equivalente a `$request->validated()['refresh_token']` pero más limpio.

12.2 LogoutController

Crea el archivo `app/Http/Controllers/Auth/LogoutController.php` :

```
<?php

declare(strict_types=1);

namespace App\Http\Controllers\Auth;

use App\Services\AuthService;
use Illuminate\Http\JsonResponse;

final class LogoutController
{
    public function __invoke(AuthService $authService): JsonResponse
    {
        $authService->logout();

        return response()->json([
            'data' => [
                'message' => 'Sesión cerrada exitosamente.',
            ],
        ]);
    }
}
```

12.3 Explicación

Ambos controllers siguen el mismo patrón: single-action, inyección de dependencias, delegación total al Service.

Controller	Líneas de lógica	Service llamado	Respuesta
RefreshTokenController	1	\$authService->refreshTokens()	{ data: { access_token, refresh_token, token_type, expires_in } }
LogoutController	1	\$authService->logout()	{ data: { message: "Sesión cerrada exitosamente." } }

El `LogoutController` NO devuelve `204 No Content` (que sería semánticamente correcto para un DELETE exitoso) porque:

1. Usamos `POST /auth/logout`, no `DELETE`. Las operaciones de autenticación suelen ser POST.
2. La respuesta con mensaje es más amigable para el frontend: puede mostrar "Sesión cerrada" al usuario.
3. `POST` con body de respuesta es más fácil de manejar en interceptors HTTP que `204 No Content`.

13. Endpoint `/me` — Usuario Autenticado

13.1 `UserController`

Crea el archivo `app/Http/Controllers/UserController.php`:

```
<?php

declare(strict_types=1);

namespace App\Http\Controllers;

use Illuminate\Http\JsonResponse;

final class UserController
{
    public function me(): JsonResponse
    {
        $user = auth('api')->user();

        return response()->json([
            'data' => [
                'user' => $user->only(['id', 'name', 'email', 'role', 'status']),
            ],
        ]);
    }
}
```

13.2 Explicación

`auth('api')->user()`

En este punto, el middleware `JwtAuthenticate` YA verificó que el token es válido y el usuario está activo. `auth('api')->user()` devuelve el modelo `User` ya resuelto (cacheado en memoria por el middleware). No genera una segunda consulta a la base de datos.

`$user->only(...)`

Limitamos los campos expuestos a los necesarios para el frontend:

Campo	Propósito en el frontend
<code>id</code>	Identificador único para operaciones (actualizar perfil, eliminar cuenta)
<code>name</code>	Mostrar “Bienvenido, John” en la UI
<code>email</code>	Mostrar en la página de perfil
<code>role</code>	Control de acceso en el frontend (mostrar/ocultar panel admin)
<code>status</code>	Información para el usuario sobre el estado de su cuenta

No exponemos `password`, `remember_token`, `email_verified_at`, `created_at`, `updated_at`, `deleted_at`. Los campos sensibles ya están en `$hidden` del modelo, pero `only()` es una capa adicional de defensa explícita.

¿Por qué no usar `auth('api')->user()` directamente en la respuesta?

```
// ✗ Expone TODOS los campos visibles del modelo
return response()->json([
    'data' => [
        'user' => auth('api')->user(),
    ],
]);
```

Aunque `$hidden` protegería `password` y `remember_token`, `created_at` y `updated_at` quedarían expuestos. Usar `only()` hace explícito qué campos se exponen, facilitando auditorías de seguridad y documentación de API.

14. Testing del Login, Middleware, Refresh y Logout

14.1 Archivo de tests

Crea el archivo `tests/Feature/Auth/LoginTest.php` :

```
<?php

declare(strict_types=1);

namespace Tests\Feature\Auth;

use App\Enums\UserStatus;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class LoginTest extends TestCase
{
    use RefreshDatabase;

    private string $password = 'Secure123!';

    private function createUser(array $overrides = []): User
    {
        return User::factory()->create(array_merge([
            'email' => 'test@example.com',
            'password' => $this->password,
            'status' => UserStatus::Active,
        ], $overrides));
    }

    private function loginPayload(string $email = 'test@example.com', string $password = null): array
```

```

{
    return [
        'email' => $email,
        'password' => $password ?? $this->password,
    ];
}

// — Tests de Login Exitoso —

public function test_user_can_login_with_valid_credentials(): void
{
    $this->createUser();

    $response = $this->postJson(route('auth.login'), $this->loginPayload());

    $response->assertStatus(200)
        ->assertJsonStructure([
            'data' => [
                'user' => [
                    'id',
                    'name',
                    'email',
                    'role',
                    'status',
                ],
                'access_token',
                'refresh_token',
                'token_type',
                'expires_in',
            ],
        ]);

    $this->assertDatabaseHas('refresh_tokens', [
        'user_id' => 1,
        'revoked_at' => null,
    ]);
}

// — Tests de Credenciales Inválidas —

public function test_login_fails_with_wrong_password(): void
{
    $this->createUser();

    $response = $this->postJson(route('auth.login'), $this->loginPayload(password: 'WrongPassword!'));

    $response->assertStatus(401)
        ->assertJson([
            'error' => [
                'code' => 'invalid_credentials',
                'message' => 'Credenciales inválidas.',
            ],
        ]);
}

public function test_login_fails_with_nonexistent_email(): void
{
    $response = $this->postJson(route('auth.login'), $this->loginPayload(email: 'noexiste@example.com'));

    $response->assertStatus(401)
        ->assertJson([
            'error' => [
                'code' => 'invalid_credentials',
                'message' => 'Credenciales inválidas.',
            ],
        ]);
}

// — Tests de Estado de Cuenta —

public function test_login_fails_with_inactive_account(): void
{
    $this->createUser(['status' => UserStatus::Inactive]);

    $response = $this->postJson(route('auth.login'), $this->loginPayload());
}

```

```

        $response->assertStatus(403)
        ->assertJson([
            'error' => [
                'code' => 'account_inactive',
                'message' => 'Tu cuenta está inactiva. Contacta al administrador.',
            ]
        ]);
    }

    public function test_login_fails_with_banned_account(): void
    {
        $this->createUser(['status' => UserStatus::Banned]);

        $response = $this->postJson(route('auth.login'), $this->loginPayload());

        $response->assertStatus(403)
        ->assertJson([
            'error' => [
                'code' => 'account_banned',
                'message' => 'Tu cuenta ha sido suspendida.',
            ]
        ]);
    }

    // — Tests de Soft-Delete —

    public function test_login_fails_with_soft_deleted_user(): void
    {
        $user = $this->createUser();
        $user->delete(); // Soft delete

        $response = $this->postJson(route('auth.login'), $this->loginPayload());

        $response->assertStatus(401)
        ->assertJson([
            'error' => [
                'code' => 'invalid_credentials',
                'message' => 'Credenciales inválidas.',
            ]
        ]);
    }

    // — Tests de Rate Limiting —

    public function test_login_enforces_rate_limiting(): void
    {
        $this->createUser();

        for ($i = 0; $i < 5; $i++) {
            $this->postJson(route('auth.login'), $this->loginPayload(password: 'wrong'));
        }

        $response = $this->postJson(route('auth.login'), $this->loginPayload(password: 'wrong'));

        $response->assertStatus(429);
    }

    // — Tests de Middleware JWT —

    public function test_protected_route_returns_401_without_token(): void
    {
        $response = $this->getJson(route('user.me'));

        $response->assertStatus(401)
        ->assertJson([
            'error' => [
                'code' => 'token_missing',
            ]
        ]);
    }

    public function test_protected_route_returns_401_with_invalid_token(): void
    {
        $response = $this->getJson(route('user.me'), [
            'Authorization' => 'Bearer invalid.token.here',
        ]);
    }

```

```

        $response->assertStatus(401)
        ->assertJson([
            'error' => [
                'code' => 'token_invalid',
            ]
        ]);
    }

    public function test_protected_route_returns_200_with_valid_token(): void
    {
        $user = $this->createUser();
        $token = auth('api')->login($user);

        $response = $this->getJson(route('user.me'), [
            'Authorization' => 'Bearer ' . $token,
        ]);

        $response->assertStatus(200)
        ->assertJson([
            'data' => [
                'user' => [
                    'id' => $user->id,
                    'email' => $user->email,
                ],
            ],
        ]);
    }

    // — Tests de Refresh Token —

    public function test_can_refresh_tokens(): void
    {
        $user = $this->createUser();
        $accessToken = auth('api')->login($user);
        $refreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);

        $user->refreshTokens()->create([
            'token' => $refreshToken,
            'expires_at' => now()->addMinutes(config('jwt.refresh_ttl')),
        ]);

        $response = $this->postJson(route('auth.refresh'), [
            'refresh_token' => $refreshToken,
        ], [
            'Authorization' => 'Bearer ' . $accessToken,
        ]);

        $response->assertStatus(200)
        ->assertJsonStructure([
            'data' => [
                'access_token',
                'refresh_token',
                'token_type',
                'expires_in',
            ],
        ]);

        // Verificar que el refresh token original fue revocado
        $this->assertDatabaseHas('refresh_tokens', [
            'token' => $refreshToken,
            'user_id' => $user->id,
        ]);
        $this->assertNotNull(
            \App\Models\RefreshToken::where('token', $refreshToken)->first()->revoked_at
        );

        // Verificar que se creó un nuevo refresh token
        $this->assertEquals(
            2,
            \App\Models\RefreshToken::where('user_id', $user->id)->count()
        );
    }

    // — Tests de Reuse Attack —

```

```

public function test_refresh_with_revoked_token_detects_reuse_attack(): void
{
    $user = $this->createUser();
    $refreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);

    $storedToken = $user->refreshTokens()->create([
        'token' => $refreshToken,
        'expires_at' => now()->addMinutes(config('jwt.refresh_ttl')),
    ]);

    // Revocar el token (simulando que ya fue usado)
    $storedToken->revoke();

    $accessToken = auth('api')->login($user);

    // Intentar usar el refresh token revocado
    $response = $this->postJson(route('auth.refresh'), [
        'refresh_token' => $refreshToken,
    ], [
        'Authorization' => 'Bearer ' . $accessToken,
    ]);

    $response->assertStatus(401)
        ->assertJson([
            'error' => [
                'code' => 'invalid_refresh_token',
            ],
        ]);

    // Verificar que TODOS los refresh tokens fueron revocados
    $activeTokens = \App\Models\RefreshToken::where('user_id', $user->id)
        ->whereNull('revoked_at')
        ->count();

    $this->assertEquals(0, $activeTokens, 'Todos los refresh tokens deben estar revocados tras un reuse attack');
}

// — Tests de Logout —

public function test_logout_invalidates_access_token_and_refresh_tokens(): void
{
    $user = $this->createUser();
    $accessToken = auth('api')->login($user);
    $refreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);

    $user->refreshTokens()->create([
        'token' => $refreshToken,
        'expires_at' => now()->addMinutes(config('jwt.refresh_ttl')),
    ]);

    // Hacer logout
    $response = $this->postJson(route('auth.logout'), [], [
        'Authorization' => 'Bearer ' . $accessToken,
    ]);

    $response->assertStatus(200)
        ->assertJson([
            'data' => [
                'message' => 'Sesión cerrada exitosamente.',
            ],
        ]);

    // Verificar que el access token fue invalidado (no puede usarse)
    $meResponse = $this->getJson(route('user.me'), [
        'Authorization' => 'Bearer ' . $accessToken,
    ]);

    $meResponse->assertStatus(401);

    // Verificar que los refresh tokens fueron revocados
    $activeTokens = \App\Models\RefreshToken::where('user_id', $user->id)
        ->whereNull('revoked_at')
        ->count();

    $this->assertEquals(0, $activeTokens, 'Todos los refresh tokens deben estar revocados tras logout');
}

```

```
// — Tests de Validación —

public function test_login_fails_without_email(): void
{
    $response = $this->postJson(route('auth.login'), [
        'password' => 'Secure123!',
    ]);

    $response->assertStatus(422)
        ->assertJsonValidationErrors(['email']);
}

public function test_login_fails_without_password(): void
{
    $response = $this->postJson(route('auth.login'), [
        'email' => 'test@example.com',
    ]);

    $response->assertStatus(422)
        ->assertJsonValidationErrors(['password']);
}
}
```

14.2 Explicación de los tests

Tests de login exitoso — test_user_can_login_with_valid_credentials

```
$response->assertStatus(200)
    ->assertJsonStructure([...]);

$this->assertDatabaseHas('refresh_tokens', [
    'user_id' => 1,
    'revoked_at' => null,
]);
```

Tres verificaciones: 1. **HTTP 200 OK** — login exitoso. 2. **Estructura de respuesta** — incluye `user`, `access_token`, `refresh_token`, `token_type`, `expires_in`. 3. **Refresh token persistido** — existe en la DB, no está revocado, pertenece al usuario correcto.

Tests de credenciales inválidas — test_login_fails_with_wrong_password y test_login_fails_with_nonexistent_email

AMBOS tests verifican exactamente la misma respuesta: 401 + `invalid_credentials`. Esto confirma que NO estamos filtrando user enumeration — el atacante recibe la misma respuesta tanto si el email no existe como si la contraseña es incorrecta.

```
// Email incorrecto
$response = $this->postJson(route('auth.login'), ['email' => 'noexiste@...', 'password' => '...']);
$response->assertStatus(401)->assertJson(['error' => ['code' => 'invalid_credentials']]);

// Contraseña incorrecta
$response = $this->postJson(route('auth.login'), ['email' => 'test@...', 'password' => 'wrong']);
$response->assertStatus(401)->assertJson(['error' => ['code' => 'invalid_credentials']]);

// ¡EXACTAMENTE LA MISMA RESPUESTA!
```

Tests de estado de cuenta — test_login_fails_with_inactive_account y test_login_fails_with_banned_account

```
$response->assertStatus(403)
->assertJson([
    'error' => [
        'code' => 'account_inactive', // o 'account_banned'
        'message' => 'Tu cuenta está inactiva. Contacta al administrador.',
    ],
]);
```

Verificamos: - **403 Forbidden** (no 401) — el usuario está autenticado pero no autorizado. - `errorCode` **específico** — el frontend puede manejar `account_inactive` y `account_banned` de forma diferente. - **Mensaje específico** — el usuario legítimo recibe información útil.

Test de soft-delete — `test_login_fails_with_soft_deleted_user`

El usuario soft-deleted NO debería poder loguearse, y la respuesta debe ser **GENÉRICA** (`invalid_credentials`), igual que email inexistente o contraseña incorrecta). Un usuario eliminado no debería ser identificable.

Tests de middleware JWT

```
// Sin token → token_missing
public function test_protected_route_returns_401_without_token()

// Token inválido → token_invalid
public function test_protected_route_returns_401_with_invalid_token()

// Token válido → 200 + datos del usuario
public function test_protected_route_returns_200_with_valid_token()
```

Estos tests verifican el middleware desde la perspectiva del cliente HTTP: ¿qué recibe el frontend en cada caso?

Tests de refresh token

```
public function test_can_refresh_tokens()
```

Verificaciones: 1. Refresh exitoso → 200 OK. 2. El refresh token original fue revocado (`revoked_at` NO es null). 3. Se creó un nuevo refresh token (2 filas en total: la original revocada + la nueva activa).

Test de reuse attack

```
public function test_refresh_with_revoked_token_detects_reuse_attack()
```

El test más sofisticado de esta suite:

1. Creamos un refresh token y lo revocamos (simulando que ya fue usado).
2. Intentamos refrescar con ese token revocado.
3. Verificamos 401 + `invalid_refresh_token`.
4. **Verificamos que TODOS los refresh tokens del usuario fueron revocados** (defensa ante reuse attack).

```
$activeTokens = \App\Models\RefreshToken::where('user_id', $user->id)
->whereNull('revoked_at')
->count();

$this->assertEquals(0, $activeTokens);
```

Test de logout

```
public function test_logout_invalidates_access_token_and_refresh_tokens()
```

Verifica el cierre completo de sesión: 1. Logout exitoso → 200 OK + mensaje. 2. El access token YA NO funciona para acceder a rutas protegidas (401). 3. Todos los refresh tokens del usuario están revocados.

14.3 Ejecutar los tests

```
php artisan test --filter LoginTest
```

Salida esperada:

```
PASS Tests\Feature\Auth\LoginTest
✓ user can login with valid credentials
✓ login fails with wrong password
✓ login fails with nonexistent email
✓ login fails with inactive account
✓ login fails with banned account
✓ login fails with soft deleted user
✓ login enforces rate limiting
✓ protected route returns 401 without token
✓ protected route returns 401 with invalid token
✓ protected route returns 200 with valid token
✓ can refresh tokens
✓ refresh with revoked token detects reuse attack
✓ logout invalidates access token and refresh tokens
✓ login fails without email
✓ login fails without password
```

```
Tests: 15 passed
```

15. Resumen y Puente a Parte 6

15.1 Lo construido en esta parte

Componente	Archivo	Estado
LoginRequest	app/Http/Requests/Auth/LoginRequest.php	✓ Validación con prevención de user enumeration
AuthenticationException	app/Exceptions/AuthenticationException.php	✓ Con <code>render()</code> , <code>errorCode</code> , HTTP 401
AccountInactiveException	app/Exceptions/AccountInactiveException.php	✓ Con <code>render()</code> , <code>errorCode</code> , HTTP 403
AuthService::login()	app/Services/AuthService.php (añadido)	✓ Verificación de credenciales, estado, soft-delete, emisión de tokens
AuthService::refreshTokens()	app/Services/AuthService.php (añadido)	✓ Rotación con detección de reuse attack
AuthService::logout()	app/Services/AuthService.php (añadido)	✓ Invalidación de access token + revocación de refresh tokens
LoginController	app/Http/Controllers/Auth/LoginController.php	✓ Single-action
RefreshTokenController	app/Http/Controllers/Auth/RefreshTokenController.php	✓ Single-action
LogoutController	app/Http/Controllers/Auth/LogoutController.php	✓ Single-action
RefreshTokenRequest	app/Http/Requests/Auth/RefreshTokenRequest.php	✓ Validación mínima
UserController	app/Http/Controllers/UserController.php	✓ Endpoint <code>/me</code>
JwtAuthenticate middleware	app/Http/Middleware/JwtAuthenticate.php	✓ Protección de rutas con verificación de estado
Rutas públicas	routes/api.php	✓ Agrupación <code>auth</code> con rate limiting
Rutas protegidas	routes/api.php	✓ Agrupación <code>jwt.auth</code>
Tests	tests/Feature/Auth/LoginTest.php	✓ 15 tests

15.2 El AuthService completo

Al finalizar esta parte, `app/Services/AuthService.php` contiene CUATRO métodos:

```

final class AuthService
{
    public function register(array $data): array { ... } // Parte 4
    public function login(array $credentials): array { ... } // Parte 5
    public function refreshTokens(string $refreshToken): array { ... } // Parte 5
    public function logout(): void { ... } // Parte 5
}

```

15.3 Flujo completo de autenticación

```

REGISTRO:
POST /api/auth/register → RegisterRequest → RegisterController → AuthService::register() → 201 + tokens

LOGIN:
POST /api/auth/login → LoginRequest → LoginController → AuthService::login()
├─ Credenciales correctas + usuario activo → 200 + tokens
├─ Credenciales incorrectas → 401 invalid_credentials
├─ Usuario inactivo/baneado → 403 account_inactive/account_banned
└─ Rate limit excedido → 429

RUTAS PROTEGIDAS (cada request):
Request → JwtAuthenticate middleware → auth('api')->user()
├─ Token válido + usuario activo → Controller
├─ Token expirado + ruta refresh → Continuar (permitir refresh)
├─ Token expirado + otra ruta → 401 token_expired
├─ Token inválido/blacklisted → 401 token_invalid
└─ Sin token → 401 token_missing

REFRESH:
POST /api/auth/refresh → JwtAuthenticate (permite expirado) → RefreshTokenRequest
→ RefreshTokenController → AuthService::refreshTokens()
├─ Refresh token válido → revocar viejo + emitir nuevos → 200 + tokens
└─ Refresh token revocado (reuse attack) → revocar todos → 401

LOGOUT:
POST /api/auth/logout → JwtAuthenticate → LogoutController → AuthService::logout()
→ Invalidar access token + revocar refresh tokens → 200

PERFIL:
GET /api/me → JwtAuthenticate → UserController::me() → 200 + datos del usuario

```

15.4 Lo que viene en la Parte 6

La [Parte 6](#) implementará las capas finales de seguridad y operaciones:

- **CORS** — configuración de `config/cors.php` para permitir peticiones desde el frontend SPA.
- **Headers de seguridad** — `X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security`, etc.
- **Rate limiting avanzado** — configuración global en `AppServiceProvider`.
- **Comando** `tokens:cleanup` — limpieza programada de refresh tokens expirados/revocados.
- **Programación en** `Console/Kernel.php` — ejecución diaria del comando de limpieza.
- **Manejo de excepciones global** — personalización del `ExceptionHandler` para mantener consistencia en todas las respuestas de error (no solo las de autenticación).
- **Variables de entorno adicionales** — configuración para producción.
- **Verificación y hardening** — checklist de seguridad antes de desplegar a producción.

Decisiones Vinculantes para Parte 6

1. El `AuthService` **YA TIENE los cuatro métodos:** `register()`, `login()`, `refreshTokens()`, `logout()`. La Parte 6 no

añade más métodos a esta clase.

2. **Las excepciones** `AuthenticationException` y `AccountInactiveException` **YA ESTÁN CREADAS**. La Parte 6 puede extender el `ExceptionHandler` global para atrapar otras excepciones (como `ModelNotFoundException`, `ValidationException`, `ThrottleRequestsException`) y formatearlas con la misma estructura `{ error: { code, message } }`.
3. **El middleware** `JwtAuthenticate` **YA ESTÁ CREADO** y registrado con alias `jwt.auth`. La Parte 6 no modifica este middleware pero puede añadir headers de seguridad a nivel de aplicación.
4. **El formato de respuesta de error YA ESTÁ DEFINIDO:**

```
{ "error": { "code": "...", "message": "..." } }
```

La Parte 6 debe extender este formato a TODAS las respuestas de error (no solo las de autenticación), personalizando el `ExceptionHandler`.

5. **Los rate limiters YA ESTÁN CONFIGURADOS** por ruta (`throttle:10,1` en registro, `throttle:5,1` en login). La Parte 6 puede añadir rate limiting global o para rutas protegidas.
6. **La estrategia one-session-per-user ESTÁ IMPLEMENTADA** en `login()` (revocar refresh tokens anteriores). La Parte 6 no cambia esta estrategia.
7. **La detección de reuse attack ESTÁ IMPLEMENTADA** en `refreshTokens()`. La Parte 6 puede añadir notificaciones al usuario cuando se detecta un reuse (email/SMS: “Actividad sospechosa en tu cuenta”).
8. **La tabla** `refresh_tokens` **acumula registros revocados y expirados**. La Parte 6 DEBE implementar el comando `tokens:cleanup` para limpiarlos periódicamente, según lo diseñado en la Parte 3 (sección 9.6) y Parte 4 (sección 2.3).
9. **Los tests de login YA ESTÁN ESCRITOS** (15 tests). La Parte 6 puede añadir tests de integración adicionales para CORS, headers de seguridad, y el comando de limpieza.
10. **El endpoint** `/me` **usa** `UserController::me()` (método, no `__invoke`). Si la Parte 6 añade más endpoints de usuario (`PUT /me`, `DELETE /me`), deben ir en el mismo controller o en controllers separados single-action — consistencia con el patrón establecido.

Parte 6: Seguridad Avanzada, CORS, Rate Limiting, Limpieza Programada y Testing Integral

1. Introducción

1.1 Dónde estamos

En las cinco partes anteriores construimos un sistema de autenticación completo y funcional:

Parte	Lo construido	Estado
Parte 1	Proyecto Laravel 11, arquitectura en capas, convenciones de código, estructura de directorios	✓
Parte 2	Variables de entorno, base de datos MySQL, migración <code>users</code> , modelo <code>User</code> , enums <code>UserRole</code> y <code>UserStatus</code>	✓
Parte 3	Paquete <code>typon/jwt-auth</code> instalado y configurado, guard <code>api</code> con driver <code>jwt</code> , claims del JWT, diseño de tabla <code>refresh_tokens</code>	✓
Parte 4	Migración <code>refresh_tokens</code> , modelo <code>RefreshToken</code> , <code>RegisterRequest</code> , <code>AuthService::register()</code> , <code>RegisterController</code> , ruta de registro, tests	✓
Parte 5	<code>LoginRequest</code> , <code>AuthenticationException</code> , <code>AccountInactiveException</code> , <code>AuthService::login()</code> , <code>AuthService::refreshTokens()</code> , <code>AuthService::logout()</code> , middleware <code>JwtAuthenticate</code> , controllers de login/refresh/logout, endpoint <code>/me</code> , 15 tests	✓

En este momento, la aplicación tiene un ciclo completo de autenticación JWT funcionando: registro, login, protección de rutas, refresh con detección de reuse attack, y logout. Los usuarios pueden registrarse, autenticarse, acceder a rutas protegidas, refrescar sus tokens y cerrar sesión.

1.2 Objetivo de esta parte

Llevar el sistema del estado “funcional en desarrollo” a “seguro y listo para producción”. Nos enfocamos en hardening de seguridad, operaciones automatizadas, y cobertura de tests exhaustiva.

Al terminar esta parte, el sistema contará con:

1. **CORS** correctamente configurado para la comunicación con el frontend SPA.
2. **Rate limiting avanzado** con limiters personalizados por IP y por usuario.
3. **Security headers HTTP** protegiendo contra clickjacking, MIME sniffing, XSS y otros vectores.
4. **Comando de limpieza programada** de refresh tokens expirados y revocados.
5. **Manejo de errores global** con formato JSON consistente para toda la API.
6. **Logging de eventos de seguridad** estructurado.
7. **Checklist de producción** verificable antes del despliegue.
8. **Batería completa de tests PHPUnit** cubriendo todos los flujos de autenticación.

9. **Preparación para producción** con comandos de optimización.

1.3 Qué NO construiremos en esta parte

- Email verification ni password reset (quedan como próximos pasos recomendados).
- 2FA ni OAuth social login (extensiones futuras).
- Panel de administración de usuarios.
- Documentación de API con Swagger/OpenAPI.

2. Configuración de CORS (Cross-Origin Resource Sharing)

2.1 Qué es CORS y por qué importa

La **Same-Origin Policy** (SOP) es una restricción de seguridad implementada por todos los navegadores modernos. Su regla es simple: un script que se ejecuta en `dominioA.com` NO puede hacer peticiones HTTP a `dominioB.com` a menos que `dominioB.com` lo permita explícitamente.

CORS (Cross-Origin Resource Sharing) es el mecanismo que permite a `dominioB.com` (tu API en `api.tudominio.com`) declarar: “sí, acepto peticiones desde `dominioA.com` (tu frontend SPA en `app.tudominio.com`)”.

Sin CORS correctamente configurado:

- Tu API es **inaccesible** desde cualquier frontend que no esté en el mismo dominio.
- El navegador bloquea las peticiones de login, registro y cualquier endpoint de la API. El error típico en la consola del navegador es:

```
Access to fetch at 'https://api.tudominio.com/api/auth/login' from origin 'https://app.tudominio.com' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

- Si configuras CORS de forma demasiado permisiva (origen `*` con credenciales), expones tu API a ataques CSRF-like desde cualquier otro origen malicioso.

JWT NO protege contra CORS. Son capas completamente diferentes:

Capa	Qué protege	Dónde opera
CORS	Qué orígenes pueden hacer peticiones a tu API	Navegador (Browser-enforced)
JWT	Que el usuario que hace la petición es quien dice ser	Servidor (firma criptográfica)
HTTPS	Que los datos en tránsito no son interceptados ni modificados	Red (cifrado TLS)

Un atacante puede tener un JWT perfectamente válido, pero si CORS está mal configurado, podría usar ese JWT desde cualquier dominio malicioso. A la inversa, CORS bien configurado no protege contra un JWT robado — para eso están la expiración corta y la rotación de refresh tokens.

2.2 Flujo de una petición CORS

Cuando un frontend en `https://app.tudominio.com` hace una petición a `https://api.tudominio.com`, ocurre lo siguiente:

```
1. PREFLIGHT (automático, solo para peticiones "complejas")
Navegador → OPTIONS /api/auth/login
Headers:
  Origin: https://app.tudominio.com
  Access-Control-Request-Method: POST
  Access-Control-Request-Headers: Content-Type, Authorization

Servidor responde:
  Access-Control-Allow-Origin: https://app.tudominio.com
  Access-Control-Allow-Methods: GET, POST, PUT, DELETE
  Access-Control-Allow-Headers: Content-Type, Authorization
  Access-Control-Max-Age: 86400

2. PETICIÓN REAL
Navegador → POST /api/auth/login
Headers:
  Origin: https://app.tudominio.com
  Content-Type: application/json

Servidor responde normalmente + header CORS:
  Access-Control-Allow-Origin: https://app.tudominio.com
```

El preflight (petición `OPTIONS`) es automático: el navegador lo envía antes de la petición real cuando esta es “compleja” (usa métodos distintos de GET/HEAD/POST con content-types simples, o incluye headers como `Authorization`). Nuestra API de autenticación SIEMPRE dispara preflight porque usa `Content-Type: application/json` y `Authorization: Bearer <token>`.

Una petición `GET /api/me` con `Authorization: Bearer <token>` también dispara preflight por el header `Authorization`.

Peticiones simples (GET sin headers personalizados, POST con `Content-Type: application/x-www-form-urlencoded`) NO disparan preflight — el navegador envía la petición directamente y verifica los headers CORS en la respuesta.

2.3 Configuración en Laravel 11

Laravel 11 incluye el middleware `HandleCors` en el stack HTTP por defecto, y usa el archivo `config/cors.php` como configuración. Si el archivo no existe en tu proyecto, publícalo:

```
php artisan config:publish cors
```

Sobrescribe el contenido de `config/cors.php`:

```
<?php

declare(strict_types=1);

return [

    /*
    |-----
    | Cross-Origin Resource Sharing (CORS) Configuration
    |-----
    |
    | Configuración para el middleware HandleCors. Controla qué orígenes,
    | métodos, y headers son aceptados en peticiones cross-origin.
    |
    */

    'paths' => ['api/*', 'sanctum/csrf-cookie'],

    'allowed_methods' => ['*'],

    'allowed_origins' => explode(',', env('CORS_ALLOWED_ORIGINS', 'http://localhost:3000')),

    'allowed_origins_patterns' => [],

    'allowed_headers' => ['*'],

    'exposed_headers' => [],

    'max_age' => 86400,

    'supports_credentials' => false,

];
```

2.4 Explicación de cada clave

paths

```
'paths' => ['api/*', 'sanctum/csrf-cookie'],
```

Qué hace: Especifica qué rutas de la aplicación deben incluir los headers CORS en sus respuestas. Solo las rutas que coinciden con estos patrones activan el middleware CORS.

Por qué `api/*`: Todas nuestras rutas de autenticación viven bajo el prefijo `/api` (definido en `bootstrap/app.php`). Incluir `api/*` cubre `api/auth/login`, `api/auth/register`, `api/me`, `api/auth/refresh`, `api/auth/logout`, y cualquier futura ruta de API.

Por qué NO `*`: Si pusiéramos `*`, las rutas web tradicionales (Blade, vistas) también tendrían headers CORS. Esto es innecesario (las vistas Blade se sirven del mismo dominio) y añade superficie de ataque innecesaria.

allowed_methods

```
'allowed_methods' => ['*'],
```

Qué hace: Lista de métodos HTTP que el servidor acepta en peticiones cross-origin.

*** vs lista explícita:** `*` permite todos los métodos (GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD). Es aceptable y conveniente para una API RESTful que expone múltiples métodos. Si prefieres ser más restrictivo:

```
'allowed_methods' => ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
```

La restricción real de métodos en Laravel viene de las rutas definidas en `routes/api.php` — si no hay una ruta `DELETE /api/users`, el método DELETE simplemente retorna 404, independientemente de lo que CORS permita.

`allowed_origins`

```
'allowed_origins' => explode(',', env('CORS_ALLOWED_ORIGINS', 'http://localhost:3000'));
```

Qué hace: Lista blanca explícita de orígenes (dominios) desde los cuales se aceptan peticiones. Cada origen debe incluir el protocolo (`http://` o `https://`), el dominio, y el puerto si no es el estándar (80/443).

⚠ **NUNCA uses `'*'` en producción.** Aunque para APIs JWT sin cookies `*` es técnicamente aceptable (no hay cookies que proteger), es una mala práctica de seguridad. Siempre debes saber exactamente qué dominios están autorizados a hablar con tu API.

Separamos los orígenes con coma en la variable de entorno:

```
# Desarrollo local (React en 3000, Vite en 5173, Angular en 4200, Next.js en 3000)
CORS_ALLOWED_ORIGINS=http://localhost:3000,http://localhost:5173,http://localhost:4200

# Producción (dominios explícitos de tu organización)
CORS_ALLOWED_ORIGINS=https://app.tudominio.com,https://admin.tudominio.com
```

`explode(',', ...)` convierte el string separado por comas en un array PHP que Laravel espera.

`allowed_origins_patterns`

```
'allowed_origins_patterns' => [],
```

Qué hace: Permite definir patrones regex para orígenes dinámicos. Útil cuando tienes múltiples subdominios.

Ejemplo de uso (para futura referencia):

```
'allowed_origins_patterns' => [
    '#^https://.*\.tudominio\.com$#',
]
```

Esto permitiría `https://app.tudominio.com`, `https://admin.tudominio.com`, `https://staging.tudominio.com`, etc. Sin embargo, `allowed_origins_patterns` es incompatible con `supports_credentials: true` (el navegador no permite wildcards con credenciales). Como este manual usa `supports_credentials: false`, es seguro usar patrones si los necesitas.

Para una API simple con 2-3 orígenes conocidos, `allowed_origins` con lista explícita es más seguro y mantenible que un patrón regex.

`allowed_headers`

```
'allowed_headers' => ['*'],
```

Qué hace: Lista de headers HTTP que el cliente puede enviar en peticiones cross-origin.

*** aquí sí es seguro.** A diferencia de `allowed_origins`, donde `*` es peligroso, en `allowed_headers` `*` simplemente significa “acepto cualquier header que el cliente envíe”. Los headers que tu API realmente procesa están limitados por la lógica de la aplicación (el middleware JWT espera `Authorization`, los controllers esperan `Content-Type: application/json`). Un header adicional como `X-Custom-Thing` será ignorado por Laravel.

Si quisieras ser restrictivo:

```
'allowed_headers' => ['Content-Type', 'Authorization', 'X-Requested-With', 'Accept'],
```

Pero `*` es más práctico y no reduce la seguridad real.

`exposed_headers`

```
'exposed_headers' => [],
```

Qué hace: Por defecto, el navegador solo expone al JavaScript los siguientes headers de la respuesta: `Cache-Control`, `Content-Language`, `Content-Type`, `Expires`, `Last-Modified`, `Pragma`. Si tu API devuelve headers personalizados que el frontend necesita leer (ej: `X-RateLimit-Remaining`, `X-Request-Id`), debes listarlos aquí.

Para esta API de autenticación, lo dejamos vacío. Los headers estándar son suficientes. Si en el futuro implementas rate limiting avanzado con headers informativos, añadirías:

```
'exposed_headers' => ['X-RateLimit-Limit', 'X-RateLimit-Remaining', 'X-Retry-After']
```

`max_age`

```
'max_age' => 86400,
```

Qué hace: Tiempo en **segundos** que el navegador puede cachear los resultados del preflight (`OPTIONS`). Durante este tiempo, el navegador NO vuelve a enviar peticiones preflight para el mismo origen.

Nuestro valor: `86400` (**24 horas**). El valor por defecto de Laravel es `0` (no cachear), lo cual es ineficiente: cada petición `POST /api/auth/login` dispararía un `OPTIONS` previo. Con `86400`, el navegador cachea el resultado del preflight por 24 horas. Solo la PRIMERA petición del día (o después de limpiar caché del navegador) dispara preflight.

Valor	Comportamiento	Recomendación
<code>0</code>	Preflight en cada petición	Solo desarrollo con cambios frecuentes de CORS
<code>86400</code> (24h)	Preflight una vez al día	Producción — balance entre eficiencia y flexibilidad
<code>604800</code> (7d)	Preflight una vez a la semana	Orígenes muy estables, rara vez cambian

`supports_credentials`

```
'supports_credentials' => false,
```

Qué hace: Indica si la API acepta credenciales (cookies, headers de autenticación HTTP, certificados TLS de cliente) en peticiones cross-origen.

⚠ **CRÍTICO:** Cuando `supports_credentials` es `true`: - `allowed_origins` **NO puede ser** `*` — el navegador rechaza la combinación. - El header `Access-Control-Allow-Credentials: true` se envía en la respuesta. - Las cookies `httpOnly`, los headers `Authorization`, y los certificados de cliente se incluyen en peticiones cross-origen.

Nuestra configuración: `false`. ¿Por qué?

Nuestra API usa JWT en el header `Authorization: Bearer <token>`. El token lo envía el frontend manualmente (desde

memoria, no desde una cookie). No dependemos de cookies para la autenticación. Con `supports_credentials: false`:

```
# El frontend envía el token manualmente:  
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

El header `Authorization` se envía en TODAS las peticiones cross-origin, incluso con `supports_credentials: false`. La restricción de credenciales aplica SOLO a mecanismos automáticos del navegador (cookies, HTTP Basic auth, TLS client certificates), no a headers que el JavaScript establece manualmente.

¿Cuándo usar `supports_credentials: true`?

Si decidieras almacenar el refresh token en una httpOnly cookie (como se discutió en la Parte 1, sección 4), necesitarías `supports_credentials: true` para que el navegador envíe automáticamente la cookie en peticiones cross-origin. En ese caso, la configuración sería:

```
'supports_credentials' => true,  
'allowed_origins' => ['https://app.tudominio.com'] // Lista explícita, NUNCA '*'
```

Para este manual, con tokens en memoria + header Authorization manual, `false` es correcto.

2.5 Verificación de CORS

Después de configurar `config/cors.php`, verifica que los headers se envían correctamente usando curl:

```
# Simular preflight  
curl -X OPTIONS http://localhost:8000/api/auth/login \  
-H "Origin: http://localhost:3000" \  
-H "Access-Control-Request-Method: POST" \  
-H "Access-Control-Request-Headers: Content-Type, Authorization" \  
-v  
  
# Deberías ver en la respuesta:  
# Access-Control-Allow-Origin: http://localhost:3000  
# Access-Control-Allow-Methods: GET, POST, PUT, DELETE, ...  
# Access-Control-Allow-Headers: Content-Type, Authorization, ...  
# Access-Control-Max-Age: 86400
```

Si estás usando un frontend React/Vue/Next.js en desarrollo, las peticiones al endpoint de login deberían funcionar sin errores de CORS en la consola del navegador.

3. Rate Limiting Avanzado

3.1 ¿Qué es rate limiting y por qué es crítico?

Rate limiting es el mecanismo que limita cuántas peticiones puede hacer un cliente (IP, usuario, o combinación de ambos) en un período de tiempo. Es una defensa de primera línea contra:

Ataque	Cómo lo mitiga el rate limiting
Brute force en login	Limitar intentos de login por IP + email. 5 intentos/minuto = 7200 intentos/día como máximo, comparado con millones sin límite.
Registro masivo de cuentas falsas	Limitar registros por IP/hora. Un bot que intenta crear 10,000 cuentas falsas es detenido.
Denegación de servicio básica	Limitar peticiones totales por IP. Un atacante no puede saturar tu API con miles de requests/segundo desde una sola IP.
Enumeración de usuarios	Limitar intentos de login por IP. El atacante no puede probar cientos de emails para ver cuáles existen.
Credential stuffing	Limitar intentos de login. Si un atacante tiene una base de datos de emails/contraseñas filtradas, el rate limiting frena la verificación masiva.

Los límites ya configurados en las Partes 4 y 5 (`throttle:10,1` para registro, `throttle:5,1` para login) son un buen punto de partida, pero insuficientes en producción por dos razones:

1. **Son límites por ruta, no por endpoint semántico:** `throttle:5,1` aplica 5 intentos/minuto por IP a la ruta de login. Pero no distingue entre emails (un atacante puede probar 5 contraseñas contra `user1@test.com`, luego 5 contra `user2@test.com`, etc.). Un rate limiter personalizado permite granularidad por IP + email.
2. **No hay rate limiting en rutas protegidas:** endpoints como `/me` o futuros endpoints de datos no tienen protección de rate limiting. Un atacante con un token válido podría hacer miles de requests.

3.2 Rate Limiting por IP vs por Usuario

Estrategia	Cómo funciona	Ventaja	Desventaja
IP-based	Cuenta requests por IP del cliente	Simple, no requiere autenticación. Funciona incluso en login/register.	Un atacante puede rotar IPs (VPN, proxy, botnet). Usuarios detrás de NAT comparten IP (oficina, universidad).
User-based	Cuenta requests por usuario autenticado (ID)	Preciso — identifica al usuario específico. No afecta a otros usuarios en la misma red.	No funciona para rutas públicas (login/register) porque el usuario no está autenticado todavía.
Híbrida	IP + email para login, user ID para rutas protegidas	Lo mejor de ambos mundos	Más compleja de configurar

3.3 Rate limiters personalizados

Crea o modifica `app/Providers/AppServiceProvider.php` :

```

<?php

declare(strict_types=1);

namespace App\Providers;

use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        //
    }

    public function boot(): void
    {
        // Limiter para login: 5 intentos por minuto por IP + email
        RateLimiter::for('login', function (Request $request) {
            $key = 'login:' . $request->ip() . ':' . ($request->input('email') ?? 'no-email');
            return Limit::perMinute(5)->by($key);
        });

        // Limiter para API general: 60 requests por minuto por usuario autenticado
        RateLimiter::for('api', function (Request $request) {
            return Limit::perMinute(60)->by(
                $request->user()?->id ?: $request->ip()
            );
        });

        // Limiter para registro: 3 por hora por IP
        RateLimiter::for('register', function (Request $request) {
            return Limit::perHour(3)->by($request->ip());
        });
    }
}

```

Explicación de cada limiter

login limiter

```

RateLimiter::for('login', function (Request $request) {
    $key = 'login:' . $request->ip() . ':' . ($request->input('email') ?? 'no-email');
    return Limit::perMinute(5)->by($key);
});

```

Clave compuesta IP + email: Esto es más granular que un límite solo por IP. Si un atacante detrás de una VPN intenta brute force, cada email diferente que prueba consume el límite POR SEPARADO. Pero el mismo email no puede intentarse más de 5 veces por minuto desde ninguna IP. Esto equilibra la protección contra brute force dirigido (mismo email, muchas contraseñas) y credential stuffing (muchos emails, una contraseña).

`?? 'no-email'`: Si por alguna razón el campo `email` no está presente en la request, usamos un fallback para que la clave no quede como `login:127.0.0.1:` (que sería la misma para cualquier email faltante).

api limiter

```

RateLimiter::for('api', function (Request $request) {
    return Limit::perMinute(60)->by(
        $request->user()?->id ?: $request->ip()
    );
});

```

User-based con fallback a IP: Si el usuario está autenticado, el límite se aplica por `user_id`. Esto significa que cada usuario autenticado tiene su propio bucket de 60 requests/minuto, independientemente de su IP. Si no está autenticado (raro en rutas protegidas, pero defensivo), el límite se aplica por IP.

60 requests por minuto es generoso: un usuario navegando una SPA que hace polling o carga datos en varias pestañas no debería alcanzarlo en uso normal. Un atacante scrapeando datos o haciendo fuzzing sí lo alcanzará.

`register` `limiter`

```
RateLimiter::for('register', function (Request $request) {  
    return Limit::perHour(3)->by($request->ip());  
});
```

3 registros por hora por IP: Más restrictivo que el límite por defecto (`throttle:10,1` que son 10 por minuto). La razón: un usuario legítimo se registra UNA vez. Si una misma IP intenta registrar más de 3 cuentas en una hora, es casi seguro un bot. Usamos `perHour()` en vez de `perMinute()` para capturar ataques de baja intensidad (un bot que registra una cuenta cada 20 minutos para evadir límites por minuto).

3.4 Aplicar los rate limiters personalizados en las rutas

Modifica `routes/api.php` para usar los limiters personalizados en lugar de los límites inline:

```
<?php  
  
use App\Http\Controllers\Auth\LoginController;  
use App\Http\Controllers\Auth\LogoutController;  
use App\Http\Controllers\Auth\RefreshTokenController;  
use App\Http\Controllers\Auth\RegisterController;  
use App\Http\Controllers\UserController;  
use Illuminate\Support\Facades\Route;  
  
// Rutas públicas de autenticación  
Route::prefix('auth')->group(function () {  
    Route::post('/register', RegisterController::class)  
        ->name('auth.register')  
        ->middleware('throttle:register');  
  
    Route::post('/login', LoginController::class)  
        ->name('auth.login')  
        ->middleware('throttle:login');  
});  
  
// Rutas protegidas  
Route::middleware('jwt.auth')->group(function () {  
    Route::get('/me', [UserController::class, 'me'])  
        ->name('user.me')  
        ->middleware('throttle:api');  
  
    Route::post('/auth/refresh', RefreshTokenController::class)  
        ->name('auth.refresh')  
        ->middleware('throttle:api');  
  
    Route::post('/auth/logout', LogoutController::class)  
        ->name('auth.logout')  
        ->middleware('throttle:api');  
});
```

El middleware `throttle:login` busca el rate limiter registrado con nombre `'login'` en `AppServiceProvider`. Es más semántico y mantenible que `throttle:5,1`.

3.5 Strategy de rate limiting en producción

Para un entorno de producción real, considera estas mejoras adicionales al rate limiting básico:

1. Usar Redis como cache driver para rate limiting

Por defecto, Laravel usa el driver de caché configurado en `CACHE_DRIVER` (normalmente `file` en desarrollo). El driver `file` almacena los contadores de rate limiting en archivos del sistema. Esto es problemático en producción con múltiples instancias:

```
Instancia A: login:192.168.1.5:john@test.com → 3 intentos
Instancia B: login:192.168.1.5:john@test.com → 0 intentos (no comparte estado con A)
```

El atacante solo necesita distribuir sus requests entre instancias para evadir el límite. Con Redis:

```
CACHE_DRIVER=redis
```

Todas las instancias comparten el mismo almacén de rate limiting. 5 intentos son 5 intentos en total, sin importar qué instancia los reciba.

2. Combinar con fail2ban a nivel de servidor

Rate limiting en Laravel devuelve HTTP 429. Fail2ban puede monitorear los logs de acceso (nginx/apache) y banear IPs temporalmente a nivel de firewall:

```
# /etc/fail2ban/jail.local
[laravel-auth]
enabled = true
port    = http,https
filter  = laravel-auth
logpath = /var/log/nginx/access.log
maxretry = 10
findtime = 60
bantime  = 600
```

Esto añade una capa de defensa a nivel de red, antes de que las peticiones lleguen siquiera a PHP.

3. Alertas cuando se alcancen thresholds de rate limiting

Configura monitoreo (Laravel Telescope, Sentry, Datadog, ELK) para alertar cuando el rate limiting se active con frecuencia. Un pico de respuestas 429 en el endpoint de login es un indicador temprano de un ataque de brute force en curso.

4. HTTP Security Headers

Los security headers son directivas que el servidor envía en cada respuesta HTTP para instruir al navegador sobre comportamientos de seguridad. Cada header mitiga una clase específica de ataque.

4.1 Crear el middleware de security headers

```
php artisan make:middleware SecurityHeaders
```

Sobrescribe el contenido generado en `app/Http/Middleware/SecurityHeaders.php` :

```

<?php

declare(strict_types=1);

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class SecurityHeaders
{
    public function handle(Request $request, Closure $next): Response
    {
        $response = $next($request);

        // Prevenir MIME type sniffing
        $response->headers->set('X-Content-Type-Options', 'nosniff');

        // Prevenir clickjacking
        $response->headers->set('X-Frame-Options', 'DENY');

        // Controlar información en el header Referer
        $response->headers->set('Referrer-Policy', 'strict-origin-when-cross-origin');

        // Deshabilitar APIs del navegador innecesarias
        $response->headers->set(
            'Permissions-Policy',
            'geolocation=(), microphone=(), camera=(), usb=()'
        );

        // HSTS - solo en producción con HTTPS
        if (app()->environment('production')) {
            $response->headers->set(
                'Strict-Transport-Security',
                'max-age=31536000; includeSubDomains; preload'
            );
        }

        // Content Security Policy
        $response->headers->set(
            'Content-Security-Policy',
            "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; connect-src 'self'"
        );

        return $response;
    }
}

```

4.2 Registrar el middleware globalmente

En Laravel 11, los middlewares globales se registran en `bootstrap/app.php` :

```

->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'jwt.auth' => \App\Http\Middleware\JwtAuthenticate::class,
    ]);

    $middleware->append(\App\Http\Middleware\SecurityHeaders::class);
})

```

`append()` añade el middleware al final de la cadena global. Esto significa que se ejecuta en CADA request, no solo en las de API. Los security headers son deseables también en rutas web, así que es correcto.

Si quisieras que los headers de seguridad se apliquen SOLO a rutas de API, usarías `$middleware->api()` o `$middleware->prependToGroup('api', ...)`. Pero para headers de seguridad, global es la opción más segura.

4.3 Explicación de cada header

X-Content-Type-Options: nosniff

```
X-Content-Type-Options: nosniff
```

Qué previene: MIME type sniffing. Algunos navegadores (especialmente Internet Explorer y versiones antiguas de Chrome) intentan “adivinar” el tipo de contenido de un recurso ignorando el header `Content-Type`. Esto es peligroso: un atacante podría subir un archivo con extensión `.jpg` que contiene JavaScript. Si el servidor lo sirve como `image/jpeg` pero el navegador “huele” que es JavaScript y lo ejecuta, tienes un XSS.

`nosniff` obliga al navegador a respetar EXACTAMENTE el `Content-Type` declarado por el servidor. Si el servidor dice `image/jpeg`, el navegador lo trata como imagen aunque el contenido sea JavaScript — y las imágenes no se ejecutan.

X-Frame-Options: DENY

```
X-Frame-Options: DENY
```

Qué previene: Clickjacking. Un atacante podría embeber tu aplicación en un `<iframe>` invisible en su sitio malicioso, superponer elementos visuales, y engañar al usuario para que haga clic en “Eliminar cuenta” o “Transferir fondos” creyendo que está interactuando con otra cosa.

`DENY` impide que tu aplicación sea embebida en CUALQUIER `<iframe>`, incluso de tu propio dominio. Alternativas:

Valor	Efecto
<code>DENY</code>	No puede ser embebida en ningún iframe (recomendado para APIs)
<code>SAMEORIGIN</code>	Puede ser embebida en iframes del mismo dominio (útil si tu app usa iframes propios)
<code>ALLOW-FROM https://example.com</code>	Solo permite un origen específico (obsoleto, usar CSP <code>frame-ancestors</code>)

Para una API RESTful que devuelve JSON, el clickjacking no es un vector de ataque relevante (no hay UI que clickear). Pero el header es barato de incluir y protege cualquier ruta web que pudiera existir (páginas de error de Laravel, horizon dashboard, telescope en desarrollo).

Referrer-Policy: strict-origin-when-cross-origin

```
Referrer-Policy: strict-origin-when-cross-origin
```

Qué controla: Cuánta información de la URL se envía en el header `Referer` cuando el usuario navega de tu sitio a otro.

`strict-origin-when-cross-origin` : - Mismo origen (tudominio.com → tudominio.com): envía la URL COMPLETA (incluyendo path y query string). - Cross-origin (tudominio.com → otrodominio.com): envía SOLO el origen (`https://tudominio.com`), sin path ni query string. - Downgrade HTTPS → HTTP: NO envía nada.

¿Por qué importa en una API? Si tu API redirige (301/302) a alguna URL, o si sirves documentación HTML, el header `Referer` podría filtrar información. Por ejemplo, si un endpoint redirige a una URL externa con un token en el query string (`/redirect?token=abc123`), el `Referer` expondría ese token. `strict-origin-when-cross-origin` evita filtrar el path y

query string en navegaciones cross-origin.

Permissions-Policy

```
Permissions-Policy: geolocation=(), microphone=(), camera=(), usb=()
```

Qué hace: Deshabilita APIs del navegador potentes que tu aplicación no necesita. Es una declaración de “mi aplicación NO usa geolocalización, micrófono, cámara, ni acceso USB”.

Sintaxis `=()` : significa “no permitido para ningún origen, ni siquiera el propio”. Es un deny-all explícito.

Esto es relevante si tu API alguna vez sirve HTML (páginas de error, dashboard, documentación). Si un atacante encontrara una forma de inyectar JavaScript en esas páginas, `Permissions-Policy` limita qué APIs del navegador puede explotar.

Strict-Transport-Security (HSTS)

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

Qué hace: Instruye al navegador a que SOLO acceda a este dominio mediante HTTPS, incluso si el usuario escribe `http://` en la barra de direcciones o hace clic en un enlace `http://`.

`max-age=31536000` : El navegador recuerda esta instrucción por 1 año (31,536,000 segundos). Durante ese año, CUALQUIER intento de acceder vía HTTP es automáticamente convertido a HTTPS por el navegador (interno, sin pasar por el servidor).

`includeSubDomains` : La regla aplica a TODOS los subdominios (`api.tudominio.com`, `app.tudominio.com`, `admin.tudominio.com`). Sin esto, un atacante podría crear un subdominio malicioso que use HTTP.

`preload` : Permite incluir tu dominio en la lista de preload de HSTS mantenida por Google y usada por Chrome, Firefox, Safari. Una vez en la lista, el navegador SABE que tu dominio es HTTPS-only incluso ANTES de la primera visita. Esto elimina por completo el ataque de SSL stripping en la primera conexión. Para incluirte, debes someter tu dominio en hstspreload.org.

⚠ **ADVERTENCIA:** HSTS con `max-age` alto es un compromiso. Si lo activas y luego tienes problemas con tu certificado HTTPS (expiró, configuración incorrecta), los usuarios no podrán acceder a tu sitio durante el tiempo que dure el `max-age` — el navegador rechazará cualquier conexión HTTP. Por eso solo lo activamos en `production`. Y si usas preload, la salida es aún más difícil (requiere pasar por el proceso de remoción).

```
if (app()->environment('production')) {
  $response->headers->set(
    'Strict-Transport-Security',
    'max-age=31536000; includeSubDomains; preload'
  );
}
```

En desarrollo (`APP_ENV=local`), HSTS está desactivado. `localhost` rara vez tiene HTTPS configurado, y un HSTS accidental en `localhost` podría bloquearte el acceso a otros proyectos locales.

Content-Security-Policy (CSP)

```
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; connect-src 'self'
```

Qué hace: Define una whitelist de fuentes de contenido que el navegador puede cargar. Es la defensa MÁS POTENTE

contra XSS: incluso si un atacante logra inyectar un `<script>` en tu página, CSP puede bloquear su ejecución.

Desglose de directivas:

Directiva	Valor	Significado
<code>default-src</code>	<code>'self'</code>	Política por defecto: solo cargar recursos del mismo origen. Todas las demás directivas heredan de esta a menos que se sobrescriban.
<code>script-src</code>	<code>'self'</code>	Solo ejecutar scripts del mismo origen. Bloquea <code><script src="https://evil.com/malware.js"></code> e inline scripts (<code><script>alert(1)</script></code>).
<code>style-src</code>	<code>'self' 'unsafe-inline'</code>	Permitir estilos del mismo origen + estilos inline (<code><style></code> y <code>style=""</code>). <code>'unsafe-inline'</code> se incluye porque Laravel y muchas librerías frontend usan estilos inline. Idealmente se eliminaría usando nonces o hashes.
<code>img-src</code>	<code>'self' data:</code>	Imágenes del mismo origen + data URIs (imágenes embebidas en base64).
<code>font-src</code>	<code>'self'</code>	Fuentes tipográficas solo del mismo origen.
<code>connect-src</code>	<code>'self'</code>	Conexiones XHR/WebSocket/fetch solo al mismo origen. Bloquea exfiltración de datos vía <code>fetch('https://evil.com/steal?data=' + token)</code> .

Para una API pura que devuelve JSON, CSP es menos crítico. CSP protege contra XSS en contenido HTML. Si tu API solo devuelve `application/json`, el navegador no ejecuta JSON como HTML. Sin embargo, si tu aplicación incluye: - Páginas de error HTML de Laravel (modo debug) - Horizon dashboard - Telescope - Documentación HTML servida por la API

CSP las protege. Dado que es barato de incluir y no causa problemas, lo añadimos.

Nota sobre CSP y SPAs: Si tu SPA React/Vue/Next.js se sirve desde `app.tudominio.com` y tu API desde `api.tudominio.com`, el CSP se aplica a las respuestas de la API, no a la SPA. La SPA debe configurar su propio CSP desde el servidor que la sirve (nginx, Vercel, Netlify). Esta configuración protege las respuestas HTML que la API pueda generar.

5. Manejo de Errores Global

En las Partes 4 y 5 creamos excepciones de dominio (`AuthenticationException`, `AccountInactiveException`) con su propio método `render()`. Pero el resto de excepciones de Laravel (modelos no encontrados, errores 500, errores de validación en endpoints no-autenticados) todavía usan el formato por defecto de Laravel, que es diferente a nuestro formato `{ "error": { "code": "...", "message": "..." } }`.

El objetivo de esta sección es **uniformizar TODAS las respuestas de error** de la API.

5.1 Configuración en `bootstrap/app.php`

Laravel 11 introdujo un nuevo enfoque para el manejo de excepciones a través del método `withExceptions()` en `bootstrap/app.php`. Amplía la sección de excepciones del archivo:

```

<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use App\Exceptions\AuthenticatonException;
use App\Exceptions\AccountInactiveException;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Illuminate\Validation\ValidationException;
use Symfony\Component\HttpKernel\Exception\NotFoundHttpException;
use Illuminate\Auth\Access\AuthorizationException;
use Throwable;

return Application::configure(basePath: dirname(__DIR__))
->withRouting(
    web: __DIR__ . '/../routes/web.php',
    api: __DIR__ . '/../routes/api.php',
    commands: __DIR__ . '/../routes/console.php',
    health: '/up',
)
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'jwt.auth' => \App\Http\Middleware\JwtAuthenticate::class,
    ]);

    $middleware->append(\App\Http\Middleware\SecurityHeaders::class);
});
->withExceptions(function (Exceptions $exceptions) {
    // Forzar respuestas JSON en rutas API
    $exceptions->shouldRenderJsonWhen(function (Request $request) {
        return $request->is('api/*') || $request->expectsJson();
    });

    // No reportar excepciones esperadas del dominio de autenticación
    $exceptions->dontReport([
        AuthenticationException::class,
        AccountInactiveException::class,
    ]);

    // Modelo no encontrado (404)
    $exceptions->render(function (NotFoundHttpException $e, Request $request) {
        if ($request->is('api/*')) {
            return response()->json([
                'error' => [
                    'code' => 'not_found',
                    'message' => 'El recurso solicitado no existe.',
                ],
            ], 404);
        }
    });

    // Errores de autorización (403) - Policies, Gates
    $exceptions->render(function (AuthorizationException $e, Request $request) {
        if ($request->is('api/*')) {
            return response()->json([
                'error' => [
                    'code' => 'forbidden',
                    'message' => 'No tienes permiso para realizar esta acción.',
                ],
            ], 403);
        }
    });

    // Errores de validación (422) - mantener estructura de Laravel con code adicional
    $exceptions->render(function (ValidationException $e, Request $request) {
        if ($request->is('api/*')) {
            return response()->json([
                'error' => [
                    'code' => 'validation_failed',
                    'message' => 'Los datos proporcionados no son válidos.',
                ],
                'errors' => $e->errors(),
            ], 422);
        }
    });
});

```

```

});

// Reportar en producción con contexto mínimo
$exceptions->reportable(function (Throwable $e) {
    if (app()->environment('production')) {
        Log::error($e->getMessage(), [
            'exception' => get_class($e),
            'file'       => $e->getFile(),
            'line'       => $e->getLine(),
            'user_id'    => auth('api')->id(),
            'url'        => request()->fullUrl(),
            'method'     => request()->method(),
            'ip'         => request()->ip()
        ]);
    }
});
})->create();

```

5.2 Explicación de cada sección

shouldRenderJsonWhen

```

$exceptions->shouldRenderJsonWhen(function (Request $request) {
    return $request->is('api/*') || $request->expectsJson();
});

```

Qué hace: Instruye a Laravel a que TODAS las excepciones en rutas `/api/*` se rendericen como JSON, incluso excepciones inesperadas como `ErrorException` o `PDOException`. Sin esto, una excepción no capturada en una ruta API podría devolver HTML (la página de error de Laravel), lo cual es inútil para un cliente que espera JSON.

`$request->expectsJson()`: Detecta el header `Accept: application/json`. Es una buena práctica que todo cliente de API envíe este header. Si el cliente lo envía pero la ruta no empieza con `api/`, igual le damos JSON.

dontReport

```

$exceptions->dontReport([
    AuthenticationException::class,
    AccountInactiveException::class,
]);

```

Qué hace: Excluye estas excepciones de los logs de Laravel.

¿Por qué? `AuthenticationException` se lanza en CADA intento de login fallido. Si un atacante está haciendo brute force, los logs se llenarían de cientos de entradas “Credenciales inválidas”. Estas excepciones son eventos esperados del dominio de negocio, no errores del sistema. No necesitan llenar los logs.

Si quisieras registrar intentos de login fallidos para auditoría de seguridad, lo harías en el `AuthService::login()` con `Log::warning()`, no mediante el reporte automático de excepciones.

Render personalizado para `NotFoundHttpException` (404)

```

$exceptions->render(function (NotFoundHttpException $e, Request $request) {
    if ($request->is('api/*')) {
        return response()->json([
            'error' => [
                'code' => 'not_found',
                'message' => 'El recurso solicitado no existe.',
            ],
        ], 404);
    }
});

```

Unifica el formato de errores 404. Cuando un cliente pide `GET /api/users/99999` y el usuario no existe, la respuesta es:

```
{
  "error": {
    "code": "not_found",
    "message": "El recurso solicitado no existe."
  }
}
```

En vez del formato por defecto de Laravel para `ModelNotFoundException` (que varía entre versiones).

Render personalizado para `AuthorizationException` (403)

```
$exceptions->render(function ($e, Request $request) {
    if ($request->is('api/*')) {
        return response()->json([
            'error' => [
                'code' => 'forbidden',
                'message' => 'No tienes permiso para realizar esta acción.',
            ],
        ], 403);
    }
});
```

Prepara el terreno para cuando implementes Policies y Gates de autorización. Actualmente no hay lógica de autorización más allá de “está autenticado”, pero cuando añadas `$this->authorize('update', $post)` en futuros controllers, las excepciones de autorización seguirán este formato.

Render personalizado para `ValidationException` (422)

```
$exceptions->render(function ($e, Request $request) {
    if ($request->is('api/*')) {
        return response()->json([
            'error' => [
                'code' => 'validation_failed',
                'message' => 'Los datos proporcionados no son válidos.',
            ],
            'errors' => $e->errors(),
        ], 422);
    }
});
```

Mantiene la estructura de errores de validación de Laravel (`errors` con array de mensajes por campo) pero añade nuestro envoltorio `error` con `code` y `message`. El frontend puede:

1. Verificar `error.code === 'validation_failed'` para saber que es un error de validación.
2. Iterar `errors` para mostrar mensajes específicos por campo.
3. Usar `error.message` como resumen general.

Reporte en producción

```

        $exceptions->reportable(function (Throwable $e) {
            if (app()->environment('production')) {
                Log::error($e->getMessage(), [
                    'exception' => get_class($e),
                    'file'      => $e->getFile(),
                    'line'      => $e->getLine(),
                    'user_id'   => auth('api')->id(),
                    'url'       => request()->fullUrl(),
                    'method'    => request()->method(),
                    'ip'        => request()->ip(),
                ]);
            }
        });
    });
}
});

```

En producción, logueamos excepciones con contexto, SIN stack traces completos. Las stack traces pueden contener rutas absolutas del servidor, fragmentos de código, y parámetros de funciones que potencialmente incluyen datos sensibles. El contexto que guardamos es suficiente para debugging:

Campo	Propósito
exception	Clase de la excepción (<code>PDOException</code> , <code>ErrorException</code> , etc.)
file + line	Archivo y línea donde ocurrió. Suficiente para localizar el código.
user_id	Usuario autenticado (si hay). Saber “el usuario 42 causó este error” acelera el debugging.
url + method	Qué endpoint y método HTTP fallaron.
ip	IP del cliente, útil para identificar ataques.

En desarrollo (`APP_ENV=local`), Laravel muestra la página de error completa con stack trace, variables, y entorno. Esto es invaluable para debugging y es seguro en `localhost` .

6. Logging de Eventos de Seguridad

Los logs son los ojos del sistema en producción. Sin logs, un ataque puede estar ocurriendo durante horas sin que lo sepas. Pero los logs también deben ser selectivos: demasiada información genera ruido, muy poca deja puntos ciegos.

6.1 Eventos a registrar

Añade (o confirma que ya están) los siguientes logs en `App\Services\AuthService.php` :

```

use Illuminate\Support\Facades\Log;

// En AuthService::login() – después de autenticación exitosa
Log::info('User logged in', [
    'user_id' => $user->id,
    'ip'      => request()->ip(),
    'user_agent' => request()->userAgent()
]);

// En AuthService::login() – después de fallo de credenciales
Log::warning('Failed login attempt', [
    'email' => $credentials['email'],
    'ip'    => request()->ip(),
    'user_agent' => request()->userAgent()
]);

// En AuthService::refreshTokens() – detección de reuse attack
Log::alert('Refresh token reuse detected – possible token theft', [
    'user_id' => $storedToken->user->id,
    'token_id' => $storedToken->id,
    'ip'      => request()->ip(),
    'user_agent' => request()->userAgent()
]);

// En AuthService::logout() – cierre de sesión
Log::info('User logged out', [
    'user_id' => $user->id,
    'ip'      => request()->ip(),
]);

// En AuthService::register() – nuevo registro
Log::info('New user registered', [
    'user_id' => $user->id,
    'email'   => $user->email,
    'ip'      => request()->ip(),
]);

```

6.2 Niveles de log (PSR-3)

Laravel usa la especificación PSR-3 de niveles de log. La elección del nivel es semántica y determina qué herramientas de monitoreo alertarán:

Nivel	Cuándo usarlo	Ejemplo en esta app
debug	Información detallada para desarrollo	Valores de variables durante el proceso de login
info	Eventos normales, operaciones esperadas	Login exitoso, registro exitoso, logout
notice	Eventos normales pero significativos	Usuario cambió su contraseña
warning	Eventos anómalos, no necesariamente errores	Login fallido, rate limit alcanzado
error	Errores que requieren atención	Excepción no capturada, error de base de datos
critical	Errores críticos que afectan la operación	Servicio de cache caído, no se pueden validar tokens
alert	Acción inmediata requerida	Detección de reuse attack (posible token robado)
emergency	Sistema inutilizable	Base de datos caída, aplicación no arranca

6.3 Estructura de los logs

Cada entrada de log sigue la misma estructura: un mensaje descriptivo en inglés (convención estándar para logs) y un array de contexto con datos estructurados.

```
log:warning('Failed login attempt', [
  'email'      => $credentials['email'],
  'ip'         => request()->ip(),
  'user_agent' => request()->userAgent(),
]);
```

Ventajas del contexto estructurado:

1. **Parseable:** Herramientas como ELK, Datadog, y Grafana Loki pueden indexar los campos del contexto para búsquedas como `level:warning AND email:"john@test.com"`.
2. **Consistente:** Todos los logs de seguridad tienen los mismos campos (`email`, `ip`, `user_agent`, `user_id` donde aplica).
3. **Privacidad:** El mensaje es genérico. El contexto contiene los detalles. Si necesitas redactar logs para GDPR, es más fácil filtrar campos del contexto que parsear mensajes de texto libre.

6.4 Estrategia de monitoreo recomendada

Para un sistema en producción, los logs deben ser:

1. **Centralizados:** Todos los logs de todas las instancias van a un mismo lugar (ELK stack, Datadog, Grafana Loki, CloudWatch). Buscar en archivos `.log` de cada servidor no escala.
2. **Alertados:** Configura alertas en tu sistema de monitoreo para:
 - `level:alert` → notificación inmediata (posible ataque en curso).
 - `level:warning` + alta frecuencia en login → posible brute force attack.
 - `level:error` → revisión diaria, posible bug.
3. **Retenidos:** Define una política de retención. Logs de seguridad (login, logout, refresh) deberían retenerse al menos 90 días. Logs de debug pueden rotarse cada 7 días.

7. Comando de Limpieza de Refresh Tokens (Scheduled Task)

7.1 El problema

La tabla `refresh_tokens` crece con cada login y cada refresh. Cada fila ocupa aproximadamente:

- `id` : 8 bytes (BIGINT)
- `user_id` : 8 bytes (BIGINT)
- `token` : ~400 bytes (JWT típico en VARCHAR(500))
- `expires_at` : ~4 bytes (TIMESTAMP)
- `revoked_at` : ~4 bytes (TIMESTAMP)
- `created_at` : ~4 bytes (TIMESTAMP)
- `updated_at` : ~4 bytes (TIMESTAMP)

Total: ~432 bytes por fila.

En una aplicación con 10,000 usuarios activos que se loguean una vez al día y refrescan tokens 4 veces al día (uso típico de una sesión de 8 horas con TTL de 15 minutos), se generan ~50,000 filas por día — aproximadamente 21.6 MB diarios. En un año, ~7.8 GB solo en refresh tokens. La mayoría son tokens ya expirados o revocados que no sirven para

nada.

7.2 Crear el comando

```
php artisan make:command CleanupExpiredTokens
```

Sobrescribe el contenido generado en `app/Console/Commands/CleanupExpiredTokens.php` :

```
<?php

declare(strict_types=1);

namespace App\Console\Commands;

use App\Models\RefreshToken;
use Illuminate\Console\Command;

class CleanupExpiredTokens extends Command
{
    protected $signature = 'tokens:cleanup
                            {--days=30 : Días de retención para tokens revocados (default: 30)}';

    protected $description = 'Elimina refresh tokens expirados y revocados antiguos de la base de datos.';

    public function handle(): int
    {
        $retentionDays = (int) $this->option('days');

        // Tokens expirados: eliminación inmediata
        $expiredCount = RefreshToken::where('expires_at', '<', now())->delete();
        $this->info("Tokens expirados eliminados: {$expiredCount}");

        // Tokens revocados: eliminar solo los que tienen más de N días desde su revocación
        $revokedCount = RefreshToken::whereNotNull('revoked_at')
            ->where('revoked_at', '<', now()->subDays($retentionDays))
            ->delete();
        $this->info("Tokens revocados eliminados (>{$retentionDays} días): {$revokedCount}");

        $totalDeleted = $expiredCount + $revokedCount;
        $this->info("Total eliminados: {$totalDeleted}");

        return self::SUCCESS;
    }
}
```

7.3 Explicación del comando

Signature y opciones

```
protected $signature = 'tokens:cleanup
                        {--days=30 : Días de retención para tokens revocados (default: 30)}';
```

- `tokens:cleanup` : nombre del comando. Se ejecuta con `php artisan tokens:cleanup` .
- `--days=30` : opción con valor por defecto. Define cuántos días conservar los tokens revocados antes de eliminarlos. Personalizable en cada ejecución: `php artisan tokens:cleanup --days=60` .

Tokens expirados: eliminación inmediata

```
$expiredCount = RefreshToken::where('expires_at', '<', now())->delete();
```

¿Por qué eliminación inmediata? Un token que ya expiró naturalmente (`expires_at` pasó) no tiene valor de auditoría. No fue revocado — simplemente caducó. No hay “quién lo revocó” ni “por qué”. Ocupa espacio sin propósito. Se elimina sin período de gracia.

El índice `idx_refresh_tokens_expires_at` (creado en la Parte 4) hace que esta consulta sea eficiente: MySQL usa el índice para encontrar las filas con `expires_at < NOW()` sin escanear toda la tabla.

Tokens revocados: retención de N días

```
$revokedCount = RefreshToken::whereNotNull('revoked_at')
    ->where('revoked_at', '<', now()->subDays($retentionDays))
    ->delete();
```

¿Por qué retener tokens revocados? Un token revocado fue invalidado antes de su expiración natural, por una de estas razones:

Razón de revocación	Quién lo revocó	Valor de auditoría
Rotación normal (refresh)	El sistema, durante <code>refreshTokens()</code>	Bajo — es operación normal
Logout	El usuario, durante <code>logout()</code>	Medio — “el usuario cerró sesión a las X:YZ”
Reuse attack detectado	El sistema, durante <code>refreshTokens()</code> (segundo uso)	Alto — posible ataque de seguridad

La retención de 30 días (configurable) permite auditoría forense: si un usuario reporta actividad sospechosa, puedes consultar `SELECT * FROM refresh_tokens WHERE user_id = ? ORDER BY revoked_at DESC` y ver exactamente cuándo y por qué se revocaron sus tokens.

Después de 30 días, los tokens revocados se eliminan. El valor de auditoría decae con el tiempo — un token revocado hace 6 meses rara vez es relevante.

Output del comando

```
$this->info("Tokens expirados eliminados: {$expiredCount}");
$this->info("Tokens revocados eliminados (>{$retentionDays} días): {$revokedCount}");
$this->info("Total eliminados: {$totalDeleted}");

return self::SUCCESS;
```

El comando informa cuántos tokens eliminó en cada categoría. Esto es útil para monitoreo: si un día elimina 1 millón de tokens expirados, sabes que el sistema tiene alto volumen de refresh. Si elimina 0 consistentemente, tal vez el comando no se está ejecutando.

`self::SUCCESS` es la constante `0` (código de salida exitoso). Si quisiéramos indicar fallo, retornaríamos `self::FAILURE` (1).

7.4 Programar el comando

En Laravel 11, la programación de tareas se configura en `routes/console.php` o en `bootstrap/app.php` con el método `withSchedule()`. Usa este último para tener toda la configuración de la aplicación en un solo archivo.

Amplía `bootstrap/app.php` para incluir la programación:

```

use Illuminate\Console\Scheduling\Schedule;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: 'up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        // ... middleware configuration ...
    })
    ->withExceptions(function (Exceptions $exceptions) {
        // ... exception handling ...
    })
    ->withSchedule(function (Schedule $schedule) {
        $schedule->command('tokens:cleanup')
            ->dailyAt('03:00')
            ->timezone('America/Mexico_City')
            ->runInBackground()
            ->withoutOverlapping();
    })
    ->create();

```

Explicación de la programación

- `dailyAt('03:00')` : Ejecuta diariamente a las 3:00 AM. Esta es típicamente la hora de menor tráfico en la mayoría de aplicaciones. Ajusta la zona horaria según tu ubicación.
- `timezone('America/Mexico_City')` : Define la zona horaria para la programación. Sin esto, `03:00` se interpreta en UTC. Ajusta a tu zona horaria.
- `runInBackground()` : Ejecuta el comando en un proceso separado. Si la limpieza tarda más de un minuto, no bloquea otras tareas programadas.
- `withoutOverlapping()` : Si la ejecución anterior aún no ha terminado a las 3:00 AM del día siguiente, no inicia una nueva. Previene múltiples instancias del comando corriendo simultáneamente (lo cual no causaría problemas en esta operación, pero es una buena práctica).

7.5 Configurar el cron en el servidor

Laravel necesita un único cron job en el servidor que ejecute el scheduler cada minuto. El scheduler interno de Laravel decide qué tareas corren según su programación.

Añade esta línea al crontab del usuario que ejecuta la aplicación:

```
* * * * * cd /ruta/a/tu/proyecto && php artisan schedule:run >> /dev/null 2>&1
```

O si usas Laravel Forge, Ploi, o similar, esta entrada se configura automáticamente.

Verificar el cron y el scheduler

Puedes probar que el comando de limpieza funciona sin esperar a las 3 AM:

```
php artisan tokens:cleanup
```

Para probar el scheduler completo sin cron:

```
php artisan schedule:run
```

Esto ejecuta todas las tareas que deberían correr en este minuto. Durante el día, solo verás “No scheduled commands

are ready to run.” porque `tokens:cleanup` está programado para las 3 AM.

8. Factories para Testing

Las factories permiten generar datos de prueba de forma consistente en los tests. `UserFactory` ya fue creada en la Parte 2 y completada en la Parte 4. Aquí la refinamos con estados adicionales útiles para los tests de seguridad.

8.1 UserFactory completa

Sobrescribe (o confirma el contenido de) `database/factories/UserFactory.php` :

```
<?php

declare(strict_types=1);

namespace Database\Factories;

use App\Models\UserRole;
use App\Models\UserStatus;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Facades\Hash;

/**
 * @extends \Illuminate\Database\Eloquent\Factories\Factory<\App\Models\User>
 */
class UserFactory extends Factory
{
    public function definition(): array
    {
        return [
            'name' => fake()->name(),
            'email' => fake()->unique()->safeEmail(),
            'email_verified_at' => now(),
            'password' => Hash::make('password'),
            'role' => UserRole::User,
            'status' => UserStatus::Active,
            'remember_token' => null,
        ];
    }

    /**
     * Usuario con rol de administrador.
     */
    public function admin(): static
    {
        return $this->state(fn (array $attributes) => [
            'role' => UserRole::Admin,
        ]);
    }

    /**
     * Usuario con cuenta inactiva.
     */
    public function inactive(): static
    {
        return $this->state(fn (array $attributes) => [
            'status' => UserStatus::Inactive,
        ]);
    }

    /**
     * Usuario baneado.
     */
    public function banned(): static
    {
        return $this->state(fn (array $attributes) => [
            'status' => UserStatus::Banned,
        ]);
    }
}
```

```

    }

    /**
     * Usuario con email sin verificar.
     */
    public function unverified(): static
    {
        return $this->state(fn (array $attributes) => [
            'email_verified_at' => null,
        ]);
    }

    /**
     * Usuario soft-deleted.
     */
    public function trashed(): static
    {
        return $this->state(fn (array $attributes) => [
            'deleted_at' => now(),
        ]);
    }
}

```

8.2 Explicación de cada estado

Cada método de estado retorna una factory con atributos modificados mediante `state()`. Esto permite composición fluida:

```

// Usuario admin inactivo (estados combinados)
User::factory()->admin()->inactive()->create();

// Usuario baneado con email sin verificar
User::factory()->banned()->unverified()->create();

// Usuario estándar activo (default)
User::factory()->create();

// Crear 10 usuarios activos
User::factory()->count(10)->create();

```

`Hash::make('password')` **en la factory:** La factory crea usuarios directamente en la base de datos con `create()`. El password debe estar hashado ANTES de insertarse. Aunque el cast `'hashed'` del modelo `User` hashlea automáticamente en `save()` / `create()` de Eloquent, usar `Hash::make()` explícitamente en la factory es más seguro: si alguna vez cambia el cast, la factory sigue funcionando.

`'remember_token' => null`: Explícitamente inicializa el `remember_token` como `null`. Aunque Laravel lo hace por defecto, ser explícito evita sorpresas.

9. Testing Integral de Autenticación

Esta sección consolida y expande los tests de las Partes 4 y 5, añadiendo casos que no fueron cubiertos anteriormente. Los tests se escriben en sus propios archivos, organizados por funcionalidad.

9.1 Archivos de test

Crea los siguientes archivos (algunos ya existen de las partes anteriores; aquí los expandimos):

```

tests/Feature/Auth/
├─ LoginTest.php      ← Tests de login, middleware, refresh, logout (Parte 5 + nuevos)
├─ RegisterTest.php   ← Tests de registro (Parte 4 + nuevos)
└─ SecurityTest.php   ← NUEVO: Tests de rate limiting, CORS, headers de seguridad

```

9.2 LoginTest (expandido desde Parte 5)

El archivo `tests/Feature/Auth/LoginTest.php` ya contiene 15 tests de la Parte 5. Añade los siguientes tests adicionales:

```

<?php

declare(strict_types=1);

// =====
// Añadir estos tests a tests/Feature/Auth/LoginTest.php
// (manteniendo los 15 tests existentes de la Parte 5)
// =====

// — Tests adicionales de Login —

public function test_login_sets_expires_in_seconds_correctly(): void
{
    $this->createUser();

    $response = $this->postJson(route('auth.login'), $this->loginPayload());

    $response->assertStatus(200);

    $expectedExpiresIn = config('jwt.ttl') * 60; // 15 min * 60 = 900 segundos
    $this->assertEquals(
        $expectedExpiresIn,
        $response->json('data.expires_in'),
        "expires_in debe ser {$expectedExpiresIn} segundos"
    );
}

public function test_login_token_can_access_protected_routes(): void
{
    $this->createUser();

    $loginResponse = $this->postJson(route('auth.login'), $this->loginPayload());
    $accessToken = $loginResponse->json('data.access_token');

    $response = $this->getJson(route('user.me'), [
        'Authorization' => 'Bearer ' . $accessToken,
    ]);

    $response->assertStatus(200)
        ->assertJsonStructure([
            'data' => [
                'user' => ['id', 'name', 'email', 'role', 'status']
            ],
        ]);
}

public function test_login_revokes_previous_refresh_tokens(): void
{
    $this->createUser();

    // Primer login
    $firstLogin = $this->postJson(route('auth.login'), $this->loginPayload());
    $firstLogin->assertStatus(200);

    // Verificar que hay UN refresh token activo
    $this->assertDatabaseCount('refresh_tokens', 1);
    $this->assertEquals(
        1,
        \App\Models\RefreshToken::where('user_id', 1)->whereNull('revoked_at')->count()
    );

    // Segundo login

```

```

$this->postJson(route('auth.login'), $this->loginPayload())->assertStatus(200);

// Verificar que el primer refresh token fue revocado
$this->assertDatabaseCount('refresh_tokens', 2);
$this->assertEquals(
    1,
    \App\Models\RefreshToken::where('user_id', 1)->whereNull('revoked_at')->count(),
    'Solo debe haber un refresh token activo después del segundo login'
);
}

// — Tests adicionales de Refresh —

public function test_refresh_with_expired_token_fails(): void
{
    $user = $this->createUser();
    $accessToken = auth('api')->login($user);

    // Crear refresh token ya expirado
    $expiredRefreshToken = auth('api')->setTTL(-1)->login($user); // TTL negativo = ya expiró
    $user->refreshTokens()->create([
        'token' => $expiredRefreshToken,
        'expires_at' => now()->subMinute(), // expiró hace 1 minuto
    ]);

    $response = $this->postJson(route('auth.refresh'), [
        'refresh_token' => $expiredRefreshToken,
    ], [
        'Authorization' => 'Bearer ' . $accessToken,
    ]);

    $response->assertStatus(401)
        ->assertJson([
            'error' => [
                'code' => 'invalid_refresh_token',
            ],
        ]);
}

public function test_refresh_for_inactive_user_fails(): void
{
    $user = $this->createUser(['status' => \App\Enums\UserStatus::inactive]);
    $accessToken = auth('api')->login($user);
    $refreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);
    $user->refreshTokens()->create([
        'token' => $refreshToken,
        'expires_at' => now()->addMinutes(config('jwt.refresh_ttl')),
    ]);

    $response = $this->postJson(route('auth.refresh'), [
        'refresh_token' => $refreshToken,
    ], [
        'Authorization' => 'Bearer ' . $accessToken,
    ]);

    $response->assertStatus(403)
        ->assertJson([
            'error' => [
                'code' => 'account_inactive',
            ],
        ]);
}

public function test_refresh_without_existing_token_in_db_fails(): void
{
    $user = $this->createUser();
    $accessToken = auth('api')->login($user);

    // Refresh token generado pero NO guardado en la DB
    $orphanRefreshToken = auth('api')->setTTL(config('jwt.refresh_ttl'))->login($user);

    $response = $this->postJson(route('auth.refresh'), [
        'refresh_token' => $orphanRefreshToken,
    ], [
        'Authorization' => 'Bearer ' . $accessToken,
    ]);
}

```

```

        $response->assertStatus(401)
        ->assertJson([
            'error' => [
                'code' => 'invalid_refresh_token',
            ],
        ]);
    }

    // ——— Tests adicionales de Logout ———

    public function test_logout_revokes_only_current_user_tokens(): void
    {
        $user1 = $this->createUser(['email' => 'user1@test.com']);
        $user2 = $user->factory()->create([
            'email' => 'user2@test.com',
            'password' => $this->password,
            'status' => \App\Enums\UserStatus::Active,
        ]);

        // Login de user1 → refresh token para user1
        $login1 = $this->postJson(route('auth.login'), [
            'email' => 'user1@test.com',
            'password' => $this->password,
        ]);

        // Login de user2 → refresh token para user2
        $login2 = $this->postJson(route('auth.login'), [
            'email' => 'user2@test.com',
            'password' => $this->password,
        ]);

        // Logout de user1
        $this->postJson(route('auth.logout'), [], [
            'Authorization' => 'Bearer ' . $login1->json('data.access_token'),
        ])->assertStatus(200);

        // Verificar: tokens de user1 están revocados, tokens de user2 siguen activos
        $this->assertEquals(
            0,
            \App\Models\RefreshToken::where('user_id', $user1->id)->whereNull('revoked_at')->count(),
            'Tokens de user1 deben estar revocados'
        );

        $this->assertEquals(
            1,
            \App\Models\RefreshToken::where('user_id', $user2->id)->whereNull('revoked_at')->count(),
            'Tokens de user2 deben seguir activos'
        );
    }

    // ——— Tests del endpoint /me ———

    public function test_me_endpoint_returns_only_selected_fields(): void
    {
        $user = $this->createUser();
        $token = auth('api')->login($user);

        $response = $this->getJson(route('user.me'), [
            'Authorization' => 'Bearer ' . $token,
        ]);

        $response->assertStatus(200);

        $userData = $response->json('data.user');

        // Campos que DEBEN estar presentes
        $this->assertArrayHasKey('id', $userData);
        $this->assertArrayHasKey('name', $userData);
        $this->assertArrayHasKey('email', $userData);
        $this->assertArrayHasKey('role', $userData);
        $this->assertArrayHasKey('status', $userData);

        // Campos que NO deben estar expuestos
        $this->assertArrayNotHasKey('password', $userData);
        $this->assertArrayNotHasKey('remember_token', $userData);
    }

```

```

        $this->assertArrayNotHasKey('deleted_at', $userData);
        $this->assertArrayNotHasKey('email_verified_at', $userData);
        $this->assertArrayNotHasKey('created_at', $userData);
        $this->assertArrayNotHasKey('updated_at', $userData);
    }

```

9.3 RegisterTest (expandido desde Parte 4)

El archivo `tests/Feature/Auth/RegisterTest.php` ya contiene tests de la Parte 4. Añade estos tests adicionales:

```

<?php

declare(strict_types=1);

// =====
// Añadir estos tests a tests/Feature/Auth/RegisterTest.php
// (manteniendo los tests existentes de la Parte 4)
// =====

use App\Enums\UserRole;
use App\Enums\UserStatus;
use Illuminate\Support\Facades\Hash;

public function test_password_is_hashed_after_registration(): void
{
    $response = $this->postJson('/api/auth/register', [
        'name' => 'Hash Test',
        'email' => 'hash@example.com',
        'password' => 'Secure123!',
        'password_confirmation' => 'Secure123!',
    ]);

    $response->assertStatus(201);

    $user = \App\Models\User::where('email', 'hash@example.com')->first();
    $this->assertNotNull($user);
    $this->assertNotEquals('Secure123!', $user->password);
    $this->assertTrue(Hash::check('Secure123!', $user->password));
}

public function test_registration_defaults_role_to_user(): void
{
    $response = $this->postJson('/api/auth/register', [
        'name' => 'Role Test',
        'email' => 'role@example.com',
        'password' => 'Secure123!',
        'password_confirmation' => 'Secure123!',
    ]);

    $response->assertStatus(201);

    $this->assertDatabaseHas('users', [
        'email' => 'role@example.com',
        'role' => UserRole::User->value,
    ]);

    $user = \App\Models\User::where('email', 'role@example.com')->first();
    $this->assertEquals(UserRole::User, $user->role);
}

public function test_registration_defaults_status_to_active(): void
{
    $response = $this->postJson('/api/auth/register', [
        'name' => 'Status Test',
        'email' => 'status@example.com',
        'password' => 'Secure123!',
        'password_confirmation' => 'Secure123!',
    ]);

    $response->assertStatus(201);

    $this->assertDatabaseHas('users', [
        'email' => 'status@example.com',
    ]);
}

```

```

        'status' => UserStatus::Active->value
    ));
}

public function test_registration_ignores_role_in_request(): void
{
    // Aunque el cliente envíe "role": "admin", debe ser ignorado
    $response = $this->postJson('/api/auth/register', [
        'name' => 'Attacker',
        'email' => 'attacker@example.com',
        'password' => 'Secure123!',
        'password_confirmation' => 'Secure123!',
        'role' => 'admin', // <-- Intentando escalar privilegios
    ]);

    $response->assertStatus(201);

    $user = \App\Models\User::where('email', 'attacker@example.com')->first();
    $this->assertEquals(UserRole::User, $user->role, 'El rol debe ser User, no Admin');
}

```

9.4 SecurityTest (NUEVO)

Crea el archivo `tests/Feature/Auth/SecurityTest.php` :

```

<?php

declare(strict_types=1);

namespace Tests\Feature\Auth;

use App\Enums\UserStatus;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class SecurityTest extends TestCase
{
    use RefreshDatabase;

    private string $password = 'Secure123!';

    // — Tests de Rate Limiting —

    public function test_login_rate_limit_blocks_after_5_attempts(): void
    {
        $user = User::factory()->create([
            'email' => 'test@example.com',
            'password' => $this->password,
            'status' => UserStatus::Active,
        ]);

        // 5 intentos con contraseña incorrecta
        for ($i = 0; $i < 5; $i++) {
            $this->postJson(route('auth.login'), [
                'email' => 'test@example.com',
                'password' => 'wrong',
            ])->assertStatus(401);
        }

        // 6° intento debe ser bloqueado
        $this->postJson(route('auth.login'), [
            'email' => 'test@example.com',
            'password' => $this->password, // contraseña CORRECTA, pero debe ser bloqueado igual
        ])->assertStatus(429);
    }

    public function test_registration_rate_limit_blocks_after_3_per_hour(): void
    {
        // 3 registros exitosos
        for ($i = 0; $i < 3; $i++) {
            $this->postJson('/api/auth/register', [
                'name' => "User {$i}",
            ]);
        }
    }
}

```

```

        'email'          => "user{$i}@example.com",
        'password'       => $this->password,
        'password_confirmation' => $this->password,
    ]->assertStatus(201);
}

// 4º intento debe ser bloqueado
$this->postJson('/api/auth/register', [
    'name'          => 'Blocked',
    'email'         => 'blocked@example.com',
    'password'       => $this->password,
    'password_confirmation' => $this->password,
])->assertStatus(429);
}

public function test_user_enumeration_prevention(): void
{
    $user = User::factory()->create([
        'email'      => 'exists@example.com',
        'password'   => $this->password,
        'status'     => UserStatus::Active,
    ]);

    // Email que NO existe
    $noExists = $this->postJson(route('auth.login'), [
        'email'      => 'noexists@example.com',
        'password'   => 'irrelevant',
    ]);

    // Email que Sí existe, contraseña incorrecta
    $existsWrongPassword = $this->postJson(route('auth.login'), [
        'email'      => 'exists@example.com',
        'password'   => 'wrongpassword',
    ]);

    // Ambos deben devolver EXACTAMENTE la misma respuesta
    $noExists->assertStatus(401)
        ->assertJson(['error' => ['code' => 'invalid_credentials']]);

    $existsWrongPassword->assertStatus(401)
        ->assertJson(['error' => ['code' => 'invalid_credentials']]);

    $this->assertEquals(
        $noExists->json(),
        $existsWrongPassword->json(),
        'Las respuestas para email inexistente y contraseña incorrecta deben ser idénticas'
    );
}

// — Tests de Security Headers —

public function test_security_headers_are_present(): void
{
    $user = User::factory()->create([
        'password'   => $this->password,
        'status'     => UserStatus::Active,
    ]);

    $token = auth('api')->login($user);

    $response = $this->getJson(route('user.me'), [
        'Authorization' => 'Bearer ' . $token,
    ]);

    $response->assertHeader('X-Content-Type-Options', 'nosniff');
    $response->assertHeader('X-Frame-Options', 'DENY');
    $response->assertHeader('Referrer-Policy', 'strict-origin-when-cross-origin');
    $response->assertHeader('Content-Security-Policy');
    $response->assertHeader('Permissions-Policy');
}

public function test_hsts_header_not_present_in_local(): void
{
    $user = User::factory()->create([
        'password'   => $this->password,
        'status'     => UserStatus::Active,
    ]);

```

```

    ));

    $token = auth('api')->login($user);

    $response = $this->getJson(route('user.me'), [
        'Authorization' => 'Bearer ' . $token,
    ]);

    // En entorno local, HSTS NO debe estar presente
    $this->assertFalse(
        $response->headers->has('Strict-Transport-Security'),
        'HSTS no debe enviarse en entorno de desarrollo'
    );
}

// — Tests de CORS —

public function test_cors_headers_are_present(): void
{
    $response = $this->withHeaders([
        'Origin' => 'http://localhost:3000',
        'Access-Control-Request-Method' => 'POST',
        'Access-Control-Request-Headers' => 'Content-Type, Authorization',
    ]->options('/api/auth/login');

    $response->assertStatus(200);

    $corsOrigin = $response->headers->get('Access-Control-Allow-Origin');

    // En entorno de test, config('cors.allowed_origins') puede ser ['*'] o la lista de .env
    // Verificamos que el header existe y tiene un valor no vacío
    $this->assertNotNull($corsOrigin, 'Access-Control-Allow-Origin debe estar presente');
    $this->assertNotEmpty($corsOrigin);

    $response->assertHeader('Access-Control-Allow-Methods');
    $response->assertHeader('Access-Control-Max-Age');
}

// — Tests de Soft-Delete en autenticación —

public function test_soft_deleted_user_cannot_access_protected_routes(): void
{
    $user = User::factory()->create([
        'password' => $this->password,
        'status' => UserStatus::Active,
    ]);

    $token = auth('api')->login($user);

    // Verificar acceso normal
    $this->getJson(route('user.me'), [
        'Authorization' => 'Bearer ' . $token,
    ]->assertStatus(200);

    // Soft-delete al usuario
    $user->delete();

    // Ahora el token debe ser rechazado
    $this->getJson(route('user.me'), [
        'Authorization' => 'Bearer ' . $token,
    ]->assertStatus(401)
    ->assertJson(['error' => ['code' => 'token_invalid']]));
}

public function test_banned_user_token_is_rejected_by_middleware(): void
{
    $user = User::factory()->create([
        'password' => $this->password,
        'status' => UserStatus::Active,
    ]);

    $token = auth('api')->login($user);

    // Verificar acceso normal
    $this->getJson(route('user.me'), [
        'Authorization' => 'Bearer ' . $token,
    ]->assertStatus(200);

    // Banned user token is rejected by middleware
    $this->getJson(route('user.me'), [
        'Authorization' => 'Bearer ' . $token,
    ]->assertStatus(401)
    ->assertJson(['error' => ['code' => 'token_invalid']]));
}

```

```

    ])->assertStatus(200);

    // Banear al usuario
    $user->update(['status' => UserStatus::Banned]);

    // Ahora el token debe ser rechazado
    $this->getJson(route('user.me'), [
        'Authorization' => 'Bearer ' . $token,
    ])->assertStatus(403) // o 401 según implementación del middleware
    ->assertJson(['error' => ['code' => 'account_inactive']]);
}

// — Tests de Validación de Errores JSON —

public function test_404_errors_follow_json_format(): void
{
    $response = $this->getJson('/api/nonexistent-endpoint');

    $response->assertStatus(404)
        ->assertJsonStructure([
            'error' => ['code', 'message'],
        ])
        ->assertJson([
            'error' => [
                'code' => 'not_found',
            ],
        ]);
}

public function test_422_errors_include_error_code(): void
{
    $response = $this->postJson(route('auth.login'), [
        // Falta el campo email intencionalmente
        'password' => 'anything',
    ]);

    $response->assertStatus(422)
        ->assertJsonStructure([
            'error' => ['code', 'message'],
            'errors',
        ])
        ->assertJson([
            'error' => [
                'code' => 'validation_failed',
            ],
        ]);
}

// — Tests de Comando de Limpieza —

public function test_cleanup_command_removes_expired_tokens(): void
{
    $user = User::factory()->create(['status' => UserStatus::Active]);

    // Crear token expirado
    \App\Models\RefreshToken::create([
        'user_id' => $user->id,
        'token' => 'expired-token-jwt-string',
        'expires_at' => now()->subDay(), // expiró ayer
    ]);

    // Crear token activo
    \App\Models\RefreshToken::create([
        'user_id' => $user->id,
        'token' => 'active-token-jwt-string',
        'expires_at' => now()->addDays(7),
    ]);

    $this->assertDatabaseCount('refresh_tokens', 2);

    $this->artisan('tokens:cleanup')
        ->expectsOutputToContain('Tokens expirados eliminados: 1')
        ->assertExitCode(0);

    $this->assertDatabaseCount('refresh_tokens', 1);
    $this->assertDatabaseHas('refresh_tokens', ['token' => 'active-token-jwt-string']);
}

```

```

    $this->assertDatabaseMissing('refresh_tokens', ['token' => 'expired-token-jwt-string']);
}

public function test_cleanup_command_removes_old_revoked_tokens(): void
{
    $user = User::factory()->create(['status' => UserStatus::Active]);

    // Token revocado hace 31 días
    \App\Models\RefreshToken::create([
        'user_id' => $user->id,
        'token' => 'old-revoked-token',
        'expires_at' => now()->addDays(5), // todavía no expira por fecha natural
        'revoked_at' => now()->subDays(31),
    ]);

    // Token revocado recientemente (hace 5 días)
    \App\Models\RefreshToken::create([
        'user_id' => $user->id,
        'token' => 'recent-revoked-token',
        'expires_at' => now()->addDays(5),
        'revoked_at' => now()->subDays(5),
    ]);

    $this->assertDatabaseCount('refresh_tokens', 2);

    $this->artisan('tokens:cleanup')
        ->expectsOutputToContain('Tokens revocados eliminados (>30 días): 1')
        ->assertExitCode(0);

    $this->assertDatabaseCount('refresh_tokens', 1);
    $this->assertDatabaseMissing('refresh_tokens', ['token' => 'old-revoked-token']);
    $this->assertDatabaseHas('refresh_tokens', ['token' => 'recent-revoked-token']);
}

public function test_cleanup_command_respects_custom_retention_days(): void
{
    $user = User::factory()->create(['status' => UserStatus::Active]);

    // Token revocado hace 10 días
    \App\Models\RefreshToken::create([
        'user_id' => $user->id,
        'token' => 'ten-days-revoked',
        'expires_at' => now()->addDays(5),
        'revoked_at' => now()->subDays(10),
    ]);

    // Con 15 días de retención, el token de 10 días NO debe eliminarse
    $this->artisan('tokens:cleanup --days=15')
        ->expectsOutputToContain('Tokens revocados eliminados (>15 días): 0')
        ->assertExitCode(0);

    $this->assertDatabaseHas('refresh_tokens', ['token' => 'ten-days-revoked']);

    // Con 5 días de retención, el token de 10 días SÍ debe eliminarse
    $this->artisan('tokens:cleanup --days=5')
        ->expectsOutputToContain('Tokens revocados eliminados (>5 días): 1')
        ->assertExitCode(0);

    $this->assertDatabaseMissing('refresh_tokens', ['token' => 'ten-days-revoked']);
}
}

```

9.5 Ejecutar todos los tests

```
php artisan test
```

Salida esperada combinando todas las partes:

```

PASS Tests\Feature\Auth\RegisterTest
  ✓ user can register
  ✓ registration fails with duplicate email
  ✓ registration fails with short password
  ✓ registration fails without password confirmation
  ✓ registration fails with invalid email
  ✓ registration fails with empty name
  ✓ registration enforces rate limiting
  ✓ password is hashed after registration
  ✓ registration defaults role to user
  ✓ registration defaults status to active
  ✓ registration ignores role in request

PASS Tests\Feature\Auth\LoginTest
  ✓ user can login with valid credentials
  ✓ login fails with wrong password
  ✓ login fails with nonexistent email
  ✓ login fails with inactive account
  ✓ login fails with banned account
  ✓ login fails with soft deleted user
  ✓ login enforces rate limiting
  ✓ protected route returns 401 without token
  ✓ protected route returns 401 with invalid token
  ✓ protected route returns 200 with valid token
  ✓ can refresh tokens
  ✓ refresh with revoked token detects reuse attack
  ✓ logout invalidates access token and refresh tokens
  ✓ login fails without email
  ✓ login fails without password
  ✓ login sets expires_in seconds correctly
  ✓ login token can access protected routes
  ✓ login revokes previous refresh tokens
  ✓ refresh with expired token fails
  ✓ refresh for inactive user fails
  ✓ refresh without existing token in db fails
  ✓ logout revokes only current user tokens
  ✓ me endpoint returns only selected fields

PASS Tests\Feature\Auth\SecurityTest
  ✓ login rate limit blocks after 5 attempts
  ✓ registration rate limit blocks after 3 per hour
  ✓ user enumeration prevention
  ✓ security headers are present
  ✓ hsts header not present in local
  ✓ cors headers are present
  ✓ soft deleted user cannot access protected routes
  ✓ banned user token is rejected by middleware
  ✓ 404 errors follow json format
  ✓ 422 errors include error code
  ✓ cleanup command removes expired tokens
  ✓ cleanup command removes old revoked tokens
  ✓ cleanup command respects custom retention days

Tests: 47 passed

```

10. Checklist de Seguridad para Producción

Antes de desplegar a producción, verifica cada punto de esta lista. Un solo elemento sin marcar puede ser la diferencia entre un sistema seguro y una brecha de seguridad.

☒ Ambiente y Configuración

- ☐ `APP_ENV=production` en `.env`
- ☐ `APP_DEBUG=false` en `.env`
- ☐ `APP_KEY` es una clave aleatoria fuerte (generada con `php artisan key:generate`)

- ☐ `JWT_SECRET` es una clave aleatoria fuerte de al menos 64 caracteres (generada con `php artisan jwt:secret`)
- ☐ `DB_PASSWORD` usa una contraseña fuerte — NO `root` , NO vacía, NO `password`
- ☐ Todas las contraseñas y secretos en `.env` son únicos (no se repiten entre entornos)
- ☐ `.env` NO está en el repositorio (está en `.gitignore`)
- ☐ `.env.example` está actualizado con todas las variables necesarias pero SIN valores reales
- ☐ El usuario de base de datos en producción tiene permisos acotados (no es `root`)
- ☐ HTTPS está configurado y funcionando (certificado TLS válido)
- ☐ Todas las peticiones HTTP son redirigidas a HTTPS (nginx/apache)

☒ Optimización de Laravel

- ☐ `php artisan config:cache` ejecutado
- ☐ `php artisan route:cache` ejecutado
- ☐ `php artisan view:cache` ejecutado
- ☐ `php artisan event:cache` ejecutado
- ☐ `composer install --optimize-autoloader --no-dev` ejecutado
- ☐ Dependencias de desarrollo NO están instaladas en producción
- ☐ `composer audit` ejecutado — sin vulnerabilidades conocidas en dependencias

☒ CORS

- ☐ `CORS_ALLOWED_ORIGINS` contiene SOLO los orígenes del frontend en producción (NO `*`)
- ☐ `supports_credentials` es `false` (usamos `Authorization` header, no cookies)
- ☐ `max_age` configurado (86400 recomendado)
- ☐ Verificado que las peticiones desde el frontend no generan errores CORS en consola

☒ Rate Limiting

- ☐ El rate limiter de login está activo (`throttle:login`)
- ☐ El rate limiter de registro está activo (`throttle:register`)
- ☐ Las rutas protegidas tienen rate limiting por usuario (`throttle:api`)
- ☐ El cache driver para rate limiting es `redis` (o `database`) en producción, no `file`

☒ Security Headers

- ☐ El middleware `SecurityHeaders` está registrado globalmente
- ☐ `X-Content-Type-Options: nosniff` presente
- ☐ `X-Frame-Options: DENY` presente
- ☐ `Strict-Transport-Security` activo en producción
- ☐ `Content-Security-Policy` configurado
- ☐ Verificado con securityheaders.com (objetivo: grade A+)

☒ Base de Datos

- ☐ Las migraciones están actualizadas en producción (`php artisan migrate:status`)
- ☐ La base de datos tiene backups automáticos configurados
- ☐ `utf8mb4` es el charset de la base de datos y las tablas

- ☐ Los índices están creados (especialmente `refresh_tokens.token` y `refresh_tokens.expires_at`)

☒ Contraseñas y Autenticación

- ☐ bcrypt cost factor es adecuado (10+ rounds — verificar en `config/hashing.php`)
- ☐ Longitud mínima de contraseña es 8+ caracteres
- ☐ Login tiene protección anti-brute force (rate limiting por IP + email)
- ☐ No se revelan mensajes específicos en errores de login (user enumeration prevention)
- ☐ Cuentas inactivas/baneadas no pueden autenticarse
- ☐ Usuarios soft-deleted son tratados como inexistentes en autenticación
- ☐ Refresh tokens rotan correctamente (cada refresh emite un nuevo token)
- ☐ Reuse de refresh token es detectado y TODOS los tokens del usuario son revocados
- ☐ Logout invalida tanto access token (blacklist) como refresh tokens (revoke en DB)

☒ JWT Específico

- ☐ TTL de access token es corto (≤ 15 minutos)
- ☐ TTL de refresh token es razonable (≤ 14 días)
- ☐ Claims del JWT NO contienen datos sensibles (sin password hash, sin email si es PII crítico)
- ☐ Blacklist de tokens está habilitada (`blacklist_enabled: true`)
- ☐ Blacklist usa un storage confiable (Redis recomendado en producción)
- ☐ Limpieza de refresh tokens programada con `tokens:cleanup` ejecutándose diariamente

☒ Monitoreo y Logs

- ☐ Logs de errores configurados con rotación (no crecen indefinidamente)
- ☐ Logs contienen contexto estructurado pero NO stack traces completos
- ☐ Eventos de seguridad (login exitoso/fallido, reuse attack) son registrados
- ☐ Monitoreo configurado para alertar sobre picos de 401/429 en login
- ☐ Alertas configuradas para detección de reuse attack
- ☐ Canal de comunicación para incidentes de seguridad definido

☒ Tests

- ☐ Todos los tests pasan en el entorno de producción/staging
- ☐ Tests de rate limiting verifican bloqueos
- ☐ Tests de user enumeration prevention verifican respuestas idénticas
- ☐ Tests de middleware verifican que tokens inactivos/baneados son rechazados
- ☐ Tests del comando de limpieza verifican eliminación correcta
- ☐ Cobertura de código $\geq 80\%$ en el dominio de autenticación

11. Preparación para Producción — Resumen de Comandos

Ejecuta estos comandos en orden como parte de tu pipeline de deploy:

```
# 1. Instalar dependencias de producción (sin dev)
composer install --optimize-autoloader --no-dev

# 2. Verificar que .env tiene las variables de producción
php artisan about

# 3. Cachear configuración (IMPORTANTE: hacerlo DESPUÉS de verificar .env)
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache

# 4. Ejecutar migraciones pendientes (--force para producción)
php artisan migrate --force

# 5. Verificar que las migraciones están aplicadas
php artisan migrate:status

# 6. Verificar seguridad de dependencias
composer audit

# 7. Ejecutar tests (idealmente en pipeline CI/CD antes del deploy)
php artisan test

# 8. Limpiar cachés antiguas
php artisan optimize:clear # Solo si necesitas regenerar cachés

# 9. Reiniciar colas (si usas queue workers)
php artisan queue:restart
```

Verifica el estado del sistema después del deploy:

```
# Ver variables críticas de .env
cat .env | grep -E "APP_ENV|APP_DEBUG|APP_KEY|DB_PASSWORD|JWT_SECRET"

# Verificar que config:cache se aplicó
php artisan config:clear # Si muestra "Configuration cache cleared", estaba cacheado
php artisan config:cache # Volver a cachear

# Verificar rutas registradas
php artisan route:list | grep auth

# Verificar que el cron está activo
crontab -l | grep schedule
```

12. Conclusión del Manual Completo

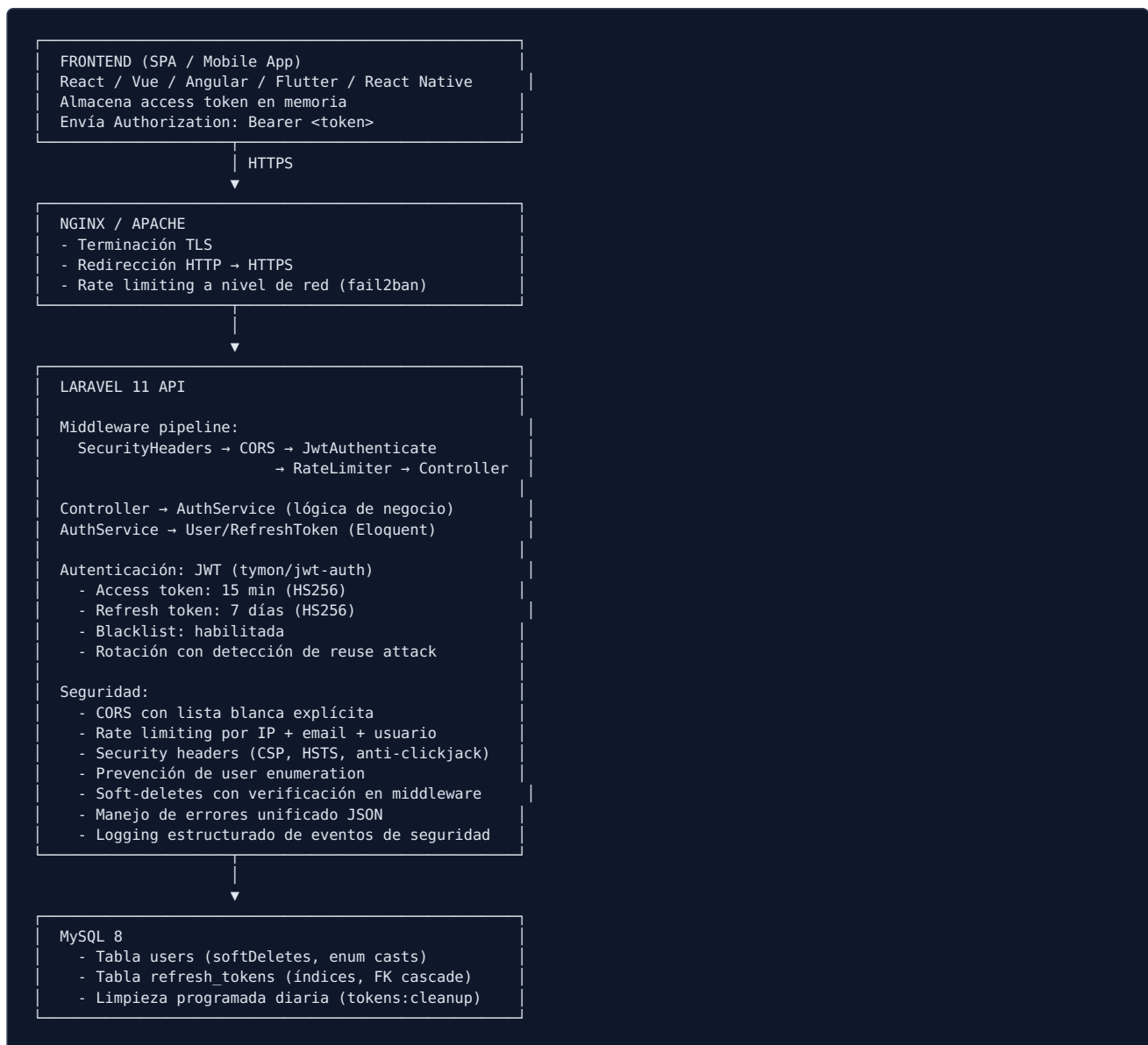
12.1 Lo Construido en las 6 Partes

A lo largo de este manual, construiste desde cero un sistema de autenticación y autorización completo, seguro, testeado y listo para producción. Recapitulemos el camino recorrido:

Parte	Contenido	Archivos Clave
Parte 1	Fundamentos: arquitectura en capas, stack tecnológico, decisiones de seguridad (bcrypt, HS256, claims JWT, CORS conceptual), estructura de directorios, convenciones de código, tipado estricto, enums, formato JSON de respuesta	Estructura del proyecto

Parte 2	Variables de entorno (<code>.env</code>), conexión a MySQL 8, migración <code>users</code> , modelo <code>User</code> con <code>JWTSubject</code> , <code>SoftDeletes</code> , casts (<code>hashed</code> , <code>UserRole</code> , <code>UserStatus</code>), enums <code>UserRole</code> y <code>UserStatus</code>	<code>config/database.php</code> , <code>.env</code> , <code>User.php</code> , <code>UserRole.php</code> , <code>UserStatus.php</code> , migración <code>create_users_table</code>
Parte 3	Instalación de <code>tymon/jwt-auth</code> , configuración de <code>config/jwt.php</code> (HS256, TTL 15min, refresh TTL 7d), generación de <code>JWT_SECRET</code> , implementación de <code>getJWTIdentifier()</code> y <code>getJWTCustomClaims()</code> , guard <code>api</code> con driver <code>jwt</code> , diseño de tabla <code>refresh_tokens</code>	<code>config/jwt.php</code> , <code>config/auth.php</code> , <code>bootstrap/providers.php</code>
Parte 4	Migración <code>refresh_tokens</code> , modelo <code>RefreshToken</code> con helpers (<code>isExpired()</code> , <code>isRevoked()</code> , <code>isValid()</code> , <code>revoke()</code>), <code>RegisterRequest</code> , <code>AuthService::register()</code> , <code>RegisterController</code> , ruta <code>POST</code> <code>/api/auth/register</code> , tests de registro	<code>refresh_tokens migration</code> , <code>RefreshToken.php</code> , <code>RegisterRequest.php</code> , <code>AuthService.php</code> , <code>RegisterController.php</code> , <code>RegisterTest.php</code>
Parte 5	<code>LoginRequest</code> (con prevención de user enumeration), <code>AuthenticationException</code> , <code>AccountInactiveException</code> , <code>AuthService::login()</code> , <code>AuthService::refreshTokens()</code> (rotación con reuse attack detection), <code>AuthService::logout()</code> , middleware <code>JwtAuthenticate</code> , controllers de login/refresh/logout, endpoint <code>/me</code> , <code>UserController</code> , rutas públicas y protegidas, 15 tests	<code>LoginRequest.php</code> , <code>AuthenticationException.php</code> , <code>AccountInactiveException.php</code> , <code>AuthService.php</code> (completado), <code>JwtAuthenticate.php</code> , <code>LoginController.php</code> , <code>RefreshTokenController.php</code> , <code>LogoutController.php</code> , <code>UserController.php</code> , <code>LoginTest.php</code>
Parte 6 (esta)	CORS (<code>config/cors.php</code>), rate limiting avanzado con limiters personalizados por IP/email/usuario, security headers HTTP (CSP, HSTS, X- Frame-Options, etc.), manejo de errores global con formato unificado, logging de eventos de seguridad, comando <code>tokens:cleanup</code> programado, <code>UserFactory</code> con estados, batería completa de tests (47 tests), checklist de producción, comandos de deploy	<code>config/cors.php</code> , <code>AppServiceProvider.php</code> , <code>SecurityHeaders.php</code> , <code>bootstrap/app.php</code> (excepciones + scheduler), <code>CleanupExpiredTokens.php</code> , <code>UserFactory.php</code> , <code>LoginTest.php</code> (expandido), <code>RegisterTest.php</code> (expandido), <code>SecurityTest.php</code>

12.2 Stack Final



12.3 El lector ahora tiene

- Un sistema de autenticación funcional con registro, login, logout, refresh de tokens y endpoint de perfil.
- Middleware JWT que protege rutas verificando estado del usuario en cada request.
- Rotación de refresh tokens que detecta y mitiga ataques de token theft.
- Rate limiting por IP, email, y usuario autenticado.
- Seguridad perimetral: CORS, security headers, HTTPS forced via HSTS.
- Manejo de errores consistente con formato `{ error: { code, message } }` en TODA la API.
- Comando programado de limpieza de tokens expirados/revocados.
- 47 tests PHPUnit cubriendo flujos positivos, negativos, edge cases y escenarios de seguridad.
- Checklist verificable para despliegue a producción.
- Logging estructurado para monitoreo y respuesta a incidentes.

12.4 Próximos pasos recomendados

Este manual cubre el núcleo de autenticación. Funcionalidades que típicamente se añaden después:

Funcionalidad	Complejidad	Descripción
Email verification	Media	Verificar que el email del usuario es real antes de activar la cuenta. Laravel incluye <code>MustVerifyEmail</code> . Requiere configuración de mail (SMTP, Mailgun, SES).
Password reset	Media	Flujo de “olvidé mi contraseña”: email con link firmado, formulario de nueva contraseña. Laravel incluye <code>PasswordBroker</code> .
Two-Factor Authentication (2FA)	Alta	Añadir TOTP (Google Authenticator) o SMS como segundo factor. Paquetes: <code>laravel/fortify</code> , <code>pragmarx/google2fa</code> .
OAuth Social Login	Media-Alta	“Iniciar sesión con Google/Facebook/GitHub”. Paquete: <code>laravel/socialite</code> . Requiere registrar la app en cada provider.
Sesiones múltiples	Media	Permitir múltiples dispositivos simultáneos. Necesitas <code>device_id</code> en refresh tokens y UI de “cerrar otras sesiones”.
Rate limiting con Redis	Baja	Cambiar <code>CACHE_DRIVER=redis</code> para rate limiting compartido entre instancias.
Notificaciones de seguridad	Media	Email/SMS al usuario cuando se detecta login desde nueva IP, reuse attack, o cambio de contraseña.
Panel de administración	Alta	Interfaz para gestionar usuarios (banear, reactivar, eliminar, ver sesiones activas). Usar Filament, Nova, o construir custom.
Documentación de API	Baja-Media	Documentar endpoints con OpenAPI/Swagger. Paquete: <code>scribe</code> o <code>scramble</code> .
CI/CD Pipeline	Media	GitHub Actions / GitLab CI para correr tests automáticamente en cada push/PR y desplegar a producción.

12.5 Referencias y lecturas adicionales

- [JWT.io](#) — Debugger de tokens, introducción a JWT.

- [RFC 7519 — JSON Web Token](#) — La especificación oficial de JWT.
- [OWASP Authentication Cheat Sheet](#) — Guía completa de seguridad en autenticación.
- [OWASP Forgot Password Cheat Sheet](#) — Para cuando implementes password reset.
- [NIST SP 800-63B — Digital Identity Guidelines](#) — Estándar del gobierno de EE.UU. sobre autenticación digital.
- [Laravel Authentication Docs](#) — Documentación oficial de autenticación en Laravel.
- [tymon/jwt-auth Wiki](#) — Documentación del paquete JWT.
- [Laravel Rate Limiting](#) — Documentación oficial de rate limiting.
- [MDN Web Docs — CORS](#) — Explicación exhaustiva de CORS.
- [MDN Web Docs — Content Security Policy](#) — Referencia de CSP.
- [SecurityHeaders.com](#) — Escáner de security headers.
- [HSTS Preload](#) — Para incluir tu dominio en la lista de preload de HSTS.

Decisiones Vinculantes para Futuras Extensiones

Este documento es la Parte 6 de 6 del manual. Las siguientes decisiones tomadas aquí deben ser respetadas por cualquier extensión futura del sistema:

1. **El formato de respuesta de error en TODA la API** es `{ "error": { "code": "...", "message": "..." } }`. Cualquier nuevo endpoint o excepción debe seguir este formato.
2. **Los rate limiters personalizados** (`login`, `register`, `api`) están registrados en `AppServiceProvider`. Nuevos endpoints que necesiten rate limiting deben usar estos limiters existentes o registrar nuevos en el mismo provider.
3. **El middleware `SecurityHeaders` es global.** Cualquier nuevo endpoint, web o API, recibe estos headers automáticamente.
4. **El comando `tokens:cleanup` está programado diariamente a las 03:00.** Si se añaden nuevas tablas con datos temporales que necesitan limpieza (password reset tokens, email verification tokens), deben integrarse en este comando o crear comandos separados.
5. **Los logs de seguridad** (login exitoso/fallido, reuse attack, logout) están implementados en `AuthService`. Cualquier nuevo método de autenticación (OAuth, 2FA) debe seguir el mismo patrón de logging estructurado.
6. **La `UserFactory` contiene estados** `admin`, `inactive`, `banned`, `unverified`, `trashed`. Cualquier nuevo estado de usuario debe añadirse como un método de estado en la factory.
7. **Los tests están organizados por funcionalidad** (`RegisterTest`, `LoginTest`, `SecurityTest`). Nuevos tests deben seguir esta organización y usar `RefreshDatabase`.
8. **El password por defecto en la factory es** `'password'` (hasheado con `Hash::make()`). Cualquier helper de test que cree usuarios debe poder especificar un password custom o usar este default.